

Meaning Over Geometry: Criterion-Driven Text Clustering and Evaluation with Large Language Models

Candidate number: SNHW9

MSc Data Science and Machine Learning[1]

Academic supervisor: Dr. Carlo Ciliberto

Industry supervisors: Dr. Aji Ghose and Patrycja Śliwiak

14 September 2026

University College London

[1]**Disclaimer:** This report is submitted as part of the requirements for the MSc Data Science and Machine Learning at UCL. It is substantially the result of my own work except where explicitly indicated in the text. The report will be distributed to the internal and external examiners, but thereafter may not be copied or distributed except with permission from the author.

Abstract

Text is the most prevalent form of unstructured data in industry and makes up the vast majority of what enterprises record, analyse and act upon. This data routinely accumulates in enormous volumes with no inherent organisation, far beyond what manual inspection can cover. Clustering is often the first step in analysing this incoming data since it provides initial insight into a corpus's underlying structure and organises it into manageable semantic groups. These methods, however, face a major limitation. Clustering lacks a single correct solution because corpora can inherently be organised across diverse semantic clustering criteria such as domain, intent, or emotion. An ideal corpus clustering, therefore, depends on specific criteria that standard clustering algorithms do not integrate. Evaluation of text clustering faces the same problem, as existing methods are blind to the intended criterion.

This thesis asks whether the natural language reasoning ability of Large Language Models, given a one-sentence statement of that criterion, can evaluate and ultimately produce a semantically coherent clustering of an unlabelled corpus. We answer this question by introducing two new algorithms: the codebook judge and the CLEAR algorithm. The codebook judge evaluates any candidate clustering without the need for ground-truth labels by generating a criterion-conditioned reference clustering and scoring candidates against it. Its scores reach a median Spearman correlation of +0.87 with benchmark-label agreement across 19 corpora, after accounting for what geometric label-free metrics measure.

The clustering algorithm, **CLEAR** (**C**lustering with **L**LM **E**valuation **A**nd **R**epair), constructs eight candidate clusterings over the encoded documents and uses a Large Language Model to select the strongest candidate and apply post-processing. When compared against 19 benchmarks' best published result, with the encoder and language model matched to those of the record holder, it wins 8 of 19 benchmarks in normalised mutual information and 9 of 19 in clustering accuracy, at a cost of roughly one cent per 100 clustered documents.

Keywords: text clustering, large language models, label-free evaluation, LLM-as-a-judge, clustering criteria, unsupervised learning

Acknowledgements

This thesis represents a collaborative research effort between University College London and Chattermill through UCL's Industry Exchange Network. I am deeply grateful to my internal and external supervisors, Dr. Aji Ghose and Dr. Carlo Ciliberto, for heir exceptional mentorship, continued encouragement, and guidance in shaping the overall research project. I would also like to extend specific thanks to Patrycja Śliwiak at Chattermill for her critical technical assistance and guidance throughout the project. Their combined depth of knowledge and the resources they provided were foundational to the results of this project. Finally, I am truly thankful to my family and friends for their unconditional love.

Contents

Acknowledgements **i**

List of Figures **v**

List of Tables **vi**

1 Introduction **1**
1.1 Text Clustering . . . 1
1.1.1 Classical Methods . . . 1
1.1.2 Challenges and Motivations . . . 2
1.2 LLM-in-the-Loop Text Clustering . . . 2
1.2.1 Significance . . . 3
1.3 Motivation . . . 3
1.4 Research Questions and Objectives . . . 3
1.4.1 Research Question 1 . . . 3
1.4.2 Research Question 2 . . . 4
1.4.3 Research Question 3 . . . 4
1.4.4 Research Question 4 . . . 4

2 Related Work **5**
2.1 A Brief History of Text Representation . . . 5
2.1.1 Sparse Features and Static Word Vectors . . . 5
2.1.2 Transformer-Based Sentence Encoders . . . 6
2.1.3 Instruction-Tuned Encoders . . . 6
2.1.4 The MTEB Benchmark . . . 7
2.2 Classical Text Clustering . . . 7
2.2.1 Clustering Algorithms . . . 7
2.2.2 Geometric Assumptions of Clustering Algorithms on Text Embeddings . . . 8
2.2.3 Estimating the Number of Clusters . . . 10
2.3 Evaluation of Clusterings . . . 10
2.3.1 Label-Free Evaluation Metrics . . . 10
2.3.2 Label-Based Evaluation Metrics . . . 11
2.3.3 Discrepancies Between Label-Free and Label-Based Evaluation on Text . . . 12
2.4 LLM-in-the-Loop Text Clustering . . . 12

2.4.1 Improving the Representation Before Clustering . . . 13
2.4.2 Guiding Assignment During Clustering . . . 15
2.4.3 Refining Clusters After Clustering . . . 16
2.4.4 LLM-Native Clustering . . . 17
2.5 Benchmarks for Text Clustering . . . 17
2.5.1 The Standard Evaluation Suite . . . 17
2.5.2 Benchmark Coverage and Comparability Across Methods . . . 18
2.5.3 The State of the Art . . . 19
2.6 LLMs as Evaluators . . . 19
2.6.1 The LLM-as-a-Judge Paradigm . . . 19
2.6.2 Documented Biases and Failure Modes . . . 20
2.7 Research Gap . . . 21

3 Experiments **22**
3.1 Research Question 1 . . . 22
3.1.1 The Codebook Judge . . . 23
3.1.2 Corpora . . . 24
3.1.3 Candidate Clusterings . . . 25
3.1.4 Statistical Protocol . . . 25
3.2 Research Question 2 . . . 27
3.2.1 Experimental Design . . . 27
3.2.2 Design Rationale . . . 29
3.2.3 Statistical Protocol . . . 29
3.3 Research Question 3 . . . 31
3.3.1 The CLEAR Algorithm . . . 31
3.3.2 Baselines and Analyses . . . 34
3.4 Research Question 4 . . . 35
3.4.1 Matched and Unmatched Configurations . . . 36
3.5 Implementation . . . 36

4 Results and Analysis **37**
4.1 Research Question 1 . . . 37
4.1.1 Overall Validity Across Judge LLMs . . . 37
4.1.2 Per-Benchmark Patterns . . . 39
4.2 Research Question 2 . . . 40
4.2.1 Estimation Accuracy . . . 40
4.2.2 Calibration . . . 42
4.3 Research Question 3 . . . 42
4.3.1 Clustering Quality . . . 42
4.3.2 Improvement over the Geometric Baseline . . . 44
4.3.3 Selected Candidates . . . 44
4.3.4 The Contribution of the Repair Step . . . 44
4.4 Research Question 4 . . . 45

4.4.1 Against the Published State of the Art . . . 46
4.4.2 Configuration Ablation Study . . . 46
4.4.3 The Judge as Selector . . . 46

5 Discussion **48**
5.1 Interpretation of Findings . . . 49
5.1.1 Research Question 1 . . . 49
5.1.2 Research Question 2 . . . 50
5.1.3 Research Question 3 . . . 51
5.1.4 Research Question 4 . . . 53
5.2 Token Cost . . . 54

6 Conclusion and Future Work **55**
6.1 Conclusion . . . 55
6.2 Contributions . . . 56
6.3 Future Work . . . 56
Data and Code Availability . . . 57

Bibliography **58**

A Literature Survey **69**
A.1 Compared Methods and Benchmarks . . . 69
A.2 Published Results by Method and Benchmark . . . 70

B Experimental Configuration **73**
B.1 Clustering Criteria . . . 73
B.2 Matched Configurations for the State-of-the-Art Comparison . . . 73
B.3 Random Seeds . . . 74

C Prompt Templates **75**
C.1 The Codebook Judge . . . 75
C.2 Granularity Estimators . . . 76
C.3 The CLEAR algortitm . . . 77

D Supplementary Results **79**
D.1 Research Question 1 . . . 79
D.2 Research Question 2 . . . 82
D.3 Research Question 3 . . . 82

Declaration of Generative AI Use **84**

List of Figures

1 The LLM-in-the-loop algorithm roster . . . 13
2 The benchmark suite . . . 18
3 State of the art per benchmark . . . 20

4 The CLEAR pipeline . . . 32

5 Judge–AMI correlation by judge LLM . . . 38
6 Calibration of inferred granularity . . . 41
7 CLEAR quality per benchmark . . . 43
8 CLEAR improvement over k-means . . . 43
9 The contribution of the repair step . . . 45
10 CLEAR against the state of the art . . . 47
11 Encoder and LLM matching factorial . . . 48
12 Candidate selection strategies . . . 48

13 Published accuracy by method and benchmark . . . 71
14 Published NMI by method and benchmark . . . 72

15 Codebook-discovery prompt . . . 75
16 Assignment prompt . . . 75
17 Direct granularity prompt . . . 76
18 Probe granularity prompt . . . 76
19 Same-category vote prompt . . . 76
20 Sequential accumulation prompts . . . 77
21 Rewrite prompt . . . 77
22 Exemplar prompt . . . 78
23 Cluster naming prompt . . . 78
24 Batched classification prompt . . . 78

25 Per-benchmark judge validity . . . 80

List of Tables

1 Judge validity across judge LLMs 38
2 Granularity estimation scoreboard 40
3 Judge candidate selections 44

4 Expected token cost by stage 54

5 The LLM-in-the-loop algorithm roster 69
6 Benchmarks compared against in this thesis 70

7 Criterion sentences per benchmark 73
8 Matched encoder and LLM per record holder 74

9 Wilcoxon tests against the silhouette baseline 82
10 Granularity estimation scoreboard, gpt-4o-mini 82
11 Granularity estimation scoreboard, gpt-3.5-turbo 82
12 CLEAR per-benchmark scores 83

Chapter 1

Introduction

1.1 Text Clustering

Text clustering is the unsupervised organisation of unstructured documents into semantically cohesive groups. Natural language data remains one of the most abundant types of unstructured data in industry and handling massive, unorganised corpora of documents has become a central problem as industry and academia rely more on Large Language Models (LLMs), since LLMs both consume curated text corpora and produce new text at a scale no human team can read, let alone organise.

When confronted with a raw corpus of text, the fundamental first question is consistently "What is its underlying structure?" Clustering answers this by partitioning the data into manageable categories in a completely unsupervised way and is therefore essential as a first step in many analytics pipelines. Typical use cases include everything from grouping news articles into stories and organising customer feedback by issue, to analysing large collections of chatbot conversations and clustering legal documents in litigation, and even exposing coordinated misinformation campaigns on social media.

The fundamental difficulty of applying clustering algorithms to text data is that multiple ways to cluster a corpus can make sense semantically. The same collection of documents can be clustered by different factors such as intent, domain, topic, sentiment, etc. These clustering criteria can partition the same corpus in completely different, yet equally semantically valid ways.

1.1.1 Classical Methods

The classical approach to text clustering is a two-step process. First, an encoder is used to transform a sentence or document into a fixed-dimensional vector, where semantically similar sentences are placed close together in representation space and semantically dissimilar ones far apart. In the second stage, a classical clustering algorithm partitions these vectors into a certain number of groups (or clusters) purely based on the collective geometry of the embedded vectors in their representation space. Both stages are underpinned by substantial fields of research and are explored in detail in §2.1 and §2.2.

The field of computational text representation dates back to 1975 with "bag-of-words" methods [Salton et al., 1975], and has since evolved through learned word vectors to more modern

transformer-based methods. The second stage draws on a longer-established field of research that is not specific to text embeddings but can be applied to any kind of multi-dimensional data and encompasses an array of algorithms developed over many years, including k-means, agglomerative linkage, density methods, and graph community detection.

1.1.2 Challenges and Motivations

These two-stage classical methods completely separate semantic representation from the clustering aspect of text clustering. The clustering algorithm only interacts with the vector coordinates. It never reads the documents and has no notion of what the groups it produces semantically represent, so whatever the encoder failed to express cannot be recovered later. This division between the semantic and geometric stages, along with the broader task of unsupervised text clustering, results in three main challenges.

The first challenge is that of underdetermination. Since a corpus can be clustered by many possible clustering criteria (e.g. topic, domain, etc.), an encoder will always group that corpus by many different criteria at once. The clustering algorithm then groups the data by whichever criteria happen to dominate the geometry, which is rarely aligned with user intent. Aside from fine-tuning the encoder itself, there is no direct way to encode the desired clustering criterion into the geometry.

The second issue is granularity (the number of clusters). Most clustering algorithms require knowledge of the number of clusters k in advance. Those that can infer the number of clusters (§2.2.3) suffer from the same issue of underdetermination, where the intended granularity of the clustering is inherently ambiguous and no semantically superior granularity exists. These clustering algorithms start from the prior assumption that one correct granularity exists. This assumption does not hold when clustering text. The granularity of a corpus is defined by the analyst, and there is no underlying rule that determines this value.

The third challenge with classical unsupervised text clustering is that there is no valid way to evaluate the quality of a clustering in production. The only available substitutes are label-free geometric metrics (§2.3.1), which reward clusters that are compact and well-separated in the embedding space. However, it has been well-documented that these break down in the high-dimensional and complex embeddings of text corpora and consistently disagree with human-referenced quality [Petukhova et al., 2025, Wang and Ye, 2025].

These are the main shortcomings of classical text clustering that the wider field on which this work builds, LLM-in-the-loop clustering, attempts to overcome.

1.2 LLM-in-the-Loop Text Clustering

LLM-in-the-loop text clustering algorithms attempt to bridge the gap between semantic representation and geometric clustering by leveraging the natural language reasoning ability of LLMs. Use cases for LLMs in these algorithms include improving representation before clustering (§2.4.1), guiding assignment during clustering (§2.4.2), and refining cluster boundaries after clustering (§2.4.3). Some methods are even fully LLM-native, where all geometric clustering aspects of text clustering are removed, and the LLM does all the work.

1.2.1 Significance

Since an LLM receives its instructions in the form of natural language, an LLM-in-the-loop method could allow for a way to insert the clustering criterion into the clustering pipeline and thereby address the issues described above. Little to no research has exploited this possibility, although the issue is often acknowledged [von Luxburg et al., 2012, Su et al., 2023, Zhang et al., 2023b]. No published method takes an arbitrary one-sentence criterion and steers a whole pipeline by it. These methods are also evaluated with benchmark labels that a deployed system does not have, so there is no way to evaluate their performance in production by relying on non-geometric signals.

1.3 Motivation

This work was carried out in collaboration with Chattermill, a company whose service analyses customer feedback for large consumer businesses. In this context, the issues with current LLM-in-the-loop methods are even more apparent. A client with a fully unlabelled corpus but a known clustering criterion would have no way of inserting this criterion or even evaluating how well a clustering worked outside of manual checking. For example, while a client may need customer complaints grouped by specific product issues, a standard algorithm might default to clustering them by general user intent if this dominates the generated geometry.

This work addresses these shortcomings in the current literature. Evaluation is addressed first to establish label-free metrics that reliably align with human judgment. Granularity is then addressed to determine whether the number of clusters k can be inferred or must be provided as a user-controlled input. A clustering algorithm is then designed using the results of the first two research questions and is compared to the published state of the art as a clustering method against a variety of benchmarks.

1.4 Research Questions and Objectives

1.4.1 Research Question 1

Given only the criterion underlying a human-generated clustering of a corpus, do the scores of an LLM acting as a judge correlate with human judgement? Is any such correlation more than a restatement of what label-free geometric metrics already measure?

On real-world unsupervised datasets, clustering algorithms are not provided with labels to evaluate performance, so there is no easy and reliable way to compare one clustering to another. The only available label-free metrics are geometry-based (§2.3.1), and mostly reward compact, well-separated geometry, which does not reflect how natural language data is distributed in embedding space. Therefore, these geometry-based metrics have been shown to disagree with human-referenced quality on text (§2.3.3).

LLMs have been validated as evaluators of generated text [Zheng et al., 2023], but have so far been shown to correlate with human judgement only to a limited extent on clustering evaluation tasks [Hoyle et al., 2025]. A simple correlation with human judgement does not, by

itself, definitively indicate agreement with human annotators. If the LLM judge's scores turn out to be nothing more than a proxy for geometric compactness and separation, the evaluator will fail in exactly the cases where the label-free geometric metrics already fail. As such, we control for these label-free geometric metrics.

1.4.2 Research Question 2

To what extent can LLMs, given a clustering criterion, infer the granularity of purpose-built human taxonomies from unlabelled data?

Most clustering algorithms require prior knowledge of the number of clusters k. Classical methods (§2.2.3) that infer this value from the data directly make the primary assumption that the dataset has a single granularity to recover in the first place. Natural language data does not necessarily adhere to this assumption, and for a clustering benchmark corpus, the granularity is fully decided by the human annotator together with the chosen clustering criterion. Nearly every method surveyed in §2.4 avoids this issue by taking the ground-truth k as given, which is not available in deployment. The question is whether an LLM, given a certain clustering criterion, is able to align with an annotator's decision on granularity, given the same criterion.

1.4.3 Research Question 3

To what extent can an LLM, acting as both partitioner and judge, equipped with prior knowledge of the clustering criterion, and inserted into the clustering pipeline, align unsupervised clustering with the partitions humans would draw?

The methods reviewed in §2.4 show that inserting an LLM at one or more stages of the clustering pipeline improves agreement with benchmark labels. However, as noted above, these algorithms still have two main limitations. First, none of the methods attempts to include the clustering criterion (intent, domain, etc.) itself into the clustering pipeline. Second, every method relies on the same baseline geometric clustering of the corpus. This thesis attempts to draw on language reasoning not only to improve the baseline clustering, but also to judge which clustering to use in the first place.

1.4.4 Research Question 4

Can a single, general clustering algorithm, equipped with a stated criterion and a label-free LLM judge, match or exceed published LLM-in-the-loop methods across the published benchmark suite when the encoder and language model are held equal?

For each benchmark discussed in this work, a single algorithm scores highest on each evaluation metric. These algorithms are considered the state of the art but, importantly, can only be compared

against on a same-document, same-encoder, and same-LLM basis. As such, we use a different configuration of the final algorithm for each comparison.

Chapter 2

Related Work

This chapter covers the current state of the art underpinning the research questions discussed in this thesis as of September 2026. The survey of LLM-assisted clustering methods (§2.4–§2.6) was compiled through an iterative keyword search of arXiv and the ACL Anthology, forward and backward citation chasing from ClusterLLM [Zhang et al., 2023b], and verification of reported results against the original publications. Some of the discussed literature exists only as preprints, and accordingly, claims resting on these are indicated by the citation form. Further, we carefully considered only state-of-the-art results on a given benchmark from published research.

2.1 A Brief History of Text Representation

2.1.1 Sparse Features and Static Word Vectors

The earliest proposed method for computational text representation [Salton et al., 1975] was a "bag-of-words" approach in which words in a sentence or paragraph are represented as sparse points in a vocabulary-indexed vector space, where the dimensionality of the representation space equals a chosen vocabulary length. Pointwise weighting schemes such as Term Frequency–Inverse Document Frequency (TF-IDF) [Salton and Buckley, 1988] later refined this by weighting sparse points by frequency in the document or sentence. Although these methods made retrieval and clustering computationally feasible, they quantified text similarity only through exact overlap of used words. In these representation spaces, semantically similar sentences phrased differently still appear completely dissimilar which then makes semantic clustering infeasible.

The first attempt to address this semantic mismatch was Latent Semantic Analysis (LSA) [Deerwester et al., 1990], which applies matrix factorisation (truncated singular value decomposition) to the word–document count matrix to capture co-occurrence patterns that link words used in similar contexts. By drastically reducing the available dimensions, the algorithm forces words that frequently share the same surrounding vocabulary into grouped slots. Probabilistic successors such as latent Dirichlet allocation later extended this idea [Blei et al., 2003].

A major leap in capturing semantic meaning came later with learned, dense word vectors. Word2vec [Mikolov et al., 2013a,b] revealed that a shallow model trained to predict which words appear near which others in a sentence yields vectors whose underlying geometry in representation

space encodes semantic regularity. This methodology was later expanded by algorithms such as GloVe [Pennington et al., 2014], which incorporates global co-occurrence statistics, and fastText [Bojanowski et al., 2017], which extends to sub-word units to include rare and misspelt words. For document clustering purposes, the vectors to cluster are taken as the average of all the word vectors in the document.

These approaches came with two main limitations, however. First, averaging word vectors in a document loses all information about word order and document composition. Second, and most importantly, the mapping from word or subword to its corresponding vector in the representation space is static. This prevents the vector from adapting to the word's surrounding context. These limitations partly motivated the next generation of text-representation algorithms that are, at the time of writing, still considered state-of-the-art methods: Transformer-Based Sentence Encoders.

2.1.2 Transformer-Based Sentence Encoders

Introduced by Google AI in 2018, the BERT encoder [Devlin et al., 2019] uses bi-directional self-attention to produce a token representation that is conditioned on all other tokens in the sentence (forward and backwards) to fully incorporate the context in which the token sits into its vector representation. Although a major advance, BERT suffered from highly anisotropic token vector representations [Ethayarajh, 2019]. The embedded vectors only occupy a very narrow cone in representation space, making them difficult to discriminate and thus cluster by cosine distance. Some early workarounds included post-hoc corrections to the representation space itself [Li et al., 2020, Su et al., 2021].

However, a more substantial remedy to the issue of anisotropy came in training explicitly for sentence-level similarity in which individual tokens are replaced with entire token sequences so that the final representation places semantically similar sentences close together. These sentence-BERT [Reimers and Gurevych, 2019] encoders are trained by fine-tuning BERT-level encoders through sentence-pair supervision (labelled example pairs of semantically similar sentences). Subsequently, researchers expanded the fine-tuning data through a process known as contrastive learning, in which semantically similar sentence pairs (question-answer pairs, perturbed versions of sentences, paraphrasing, etc.) are pulled together, and semantically unrelated pairs are "pushed apart" in representation space. SimCSE [Gao et al., 2021] showed that a remarkably simple version of this methodology already yields strong sentence embeddings. Other sentence embedding models based on similar contrastive fine-tuning approaches, such as E5 [Wang et al., 2022] and GTE [Li et al., 2023], also showed promising early performance.

In this work, the term "encoder" refers to these types of sentence embedders. These encoders typically map to a vector space of anywhere between 384 and 1,024 dimensions and, at the time of writing, form the representational base of what most modern text-clustering pipelines are built on, with a few exceptions for encoder-free methods which are discussed in §2.4.4.

2.1.3 Instruction-Tuned Encoders

Another relatively recent improvement to sentence encoders which is particularly relevant to semantic clustering is instruction-tuned encoders. The encoders discussed in §2.1.2 all have a fixed, general-purpose notion of semantic similarity. This general notion is not task-specific and

is somehow learned from the general aggregate of its training data. An instruction-tuned encoder, instead, accepts a natural-language description of the task together with its usual input and conditions its geometry on this criterion without further fine-tuning.

Instruction-tuned encoders such as INSTRUCTOR [Su et al., 2023] allow the user to prepend an instruction such as "Represent the sentence for topic clustering: ". During its initial multi-task training phase, the network learned to associate diverse natural language instructions with their optimal target vector spaces. Because of this extensive prior training, a single frozen model (weights fixed after training) produces embeddings dynamically adapted to the stated criterion.

Combined with a clustering algorithm, these instruction-tuned encoders enable domain-specific clustering of a corpus. For example, given a corpus of customer feedback reports for a mobile app, an instruction-tuned embedder could be given a criterion for which aspect of customer feedback the user wants the corpus clustered. Instructed to represent the feedback reports for clustering by topic rather than sentiment, the encoder should place two reports about the billing system close together even if one praises the interface and the other complains about crashes. The instruction-tuned embedder can thus shift which similarities the geometry emphasises and thus allow for easier criterion-conditioned clustering down the line.

Instruction tuning is the advance most relevant to this thesis, but the broader field of sentence encoders is much larger and moves quickly. The field includes a wide array of alternative models and architectures, including LLM-based embedders [BehnamGhader et al., 2024, Muennighoff et al., 2025, Lee et al., 2025a] and distilled encoders [Zhang et al., 2024a].

2.1.4 The MTEB Benchmark

The Massive Text Embedding Benchmark (MTEB) consolidates evaluations of many different sentence encoders across various task types, including clustering. For clustering evaluation, candidate encoders are evaluated by running the same simple clustering algorithm (mini-batch k-means; §2.2.1) on fixed corpora and measuring agreement with reference human-generated clusterings [Muennighoff et al., 2023], recently extended multilingually [Enevoldsen et al., 2025].

2.2 Classical Text Clustering

Classical (non-LLM) text clustering techniques rely on a well designed encoder to map a corpus to an n-dimensional embedding space, in addition to classical geometry-based clustering algorithms. These can vary in complexity and fall into four broad families: centroid-based, hierarchical, density-based and graph-based methods.

2.2.1 Clustering Algorithms

- *Centroid methods* assign each datapoint to a cluster defined by a central point, or centroid, in the embedding space. The most widely used example is k-means [MacQueen, 1967, Lloyd, 1982], which locally minimises within-cluster variance by iteratively assigning points to their nearest centroid and updating each centroid to the mean of its assigned data points. As results are initialisation-dependent, k-means is usually run several times, and its results

averaged. Another method of mitigating the dependence on random initialisation is the k-means++ algorithm [Arthur and Vassilvitskii, 2007]. It starts by choosing a single datapoint randomly and iteratively adds new centroids by selecting from the remaining data points with a probability proportional to their squared distance from the nearest existing centroid, until all k initial centres are chosen. Careful seeding [Arthur and Vassilvitskii, 2007] and mini-batch updates [Sculley, 2010] make k-means the default at scale. As it minimises within cluster variance, k-means algorithms have an inherent bias towards spherical clusterings. One alternative algorithm that mitigates this bias is Gaussian Mixture Models (GMMs). These models allow for ellipsoidal (Gaussian) cluster shapes and enable soft assignment by representing the underlying data points as weighted sums of multivariate Gaussian distributions. GMMs are fitted through Expectation-Maximisation [Dempster et al., 1977].

- *Hierarchy methods* build a tree of merges starting from the bottom where each data point is considered its own cluster and clusters are iteratively merged based on a chosen linkage rule. Ward's linkage, which merges the pair of clusters that least increases within-cluster variance, is the standard choice for embeddings [Ward, 1963]. The tree is then cut at a chosen level to yield a desired number of clusters.

- *Density-based methods* define clusters as high-density regions where points are only assigned to their nearest cluster if they lie within a certain density threshold and unassigned points in low-density regions are simply treated as noise or outliers. These methods allow for arbitrarily shaped clusters. The most widely used examples are DBSCAN [Ester et al., 1996] and its hierarchical successor HDBSCAN [Campello et al., 2013].

- *Graph methods* represent the data as connected nodes in which each point is connected to its nearest neighbours (usually defined as points within some cosine similarity threshold or k-nearest neighbours) as a graph network with edge weights. This graph is then partitioned into well connected communities (clusters) where inter-community edges are dense and intra-community edges are mostly sparse. This partition is usually achieved spectrally whereby the weakest edges between regions are severed [Ng et al., 2002, von Luxburg, 2007]. Another common approach is through bottom-up community detection, where communities are built agglomeratively by starting at the level of individual nodes and progressively merging them into larger structures based on some condition. The most common approach here is Leiden clustering [Traag et al., 2019], where a partition is chosen in which modularity is maximised. Modularity is the metric that defines how much more edge weight falls inside communities than would be expected if the same nodes were connected at random. Leiden clustering builds upon the previously developed Louvain clustering by mathematically guaranteeing that all resulting communities remain internally connected, thereby preventing arbitrarily disconnected sparse communities, which Louvain is known to produce [Blondel et al., 2008].

2.2.2 Geometric Assumptions of Clustering Algorithms on Text Embeddings

Each family of clustering algorithms carries with it some inductive bias through assumptions of the underlying data distribution. The k-means objective assumes compact, roughly spherical,

comparably sized clusters. Ward's linkage in hierarchical clustering makes similar variance-based assumptions, where clusters are merged to minimise the increase in within-cluster variance which implicitly favours compact and similarly sized clusters. Gaussian mixtures somewhat relax the cluster shape to ellipsoids, whereas density-based methods assume clusters separated by low-density regions. These assumptions can be made for many different data types, and they thus prove very useful by themselves in many relatively low-dimensional applications such as image segmentation, bioinformatics, and customer profiling.

However, when it comes to clustering text representations in relatively high-dimensional feature space, some of these methods break down. In spaces of hundreds of dimensions, distances become less discriminative. The contrast between a point's nearest and furthest neighbours shrinks (Beyer et al., 1999, Aggarwal et al., 2001), and some points may therefore recur among many other points' nearest neighbours (*hubness*) which can distort the underlying neighbourhood structure [Radovanović et al., 2010].

While many modern embedder frameworks, such as sentence-BERT models, partly resolve the issue of extreme anisotropy of embedded text data through contrastive training, this data still often exhibits complex topologies in embedding space rather than compact, easily separable regions. Consequently, compactness-seeking objectives will often incorrectly fragment or merge them. Early comparative studies on documents found different algorithm families trading places in clustering quality depending on the specific characteristics of the corpus itself [Steinbach et al., 2000]. There is therefore a practical "no free lunch" theorem for text clustering whereby no single one of these classical geometric objectives is appropriate and works well across all types of corpora.

A final major assumption of these algorithms is that the correct cluster structure is inherently encoded in the data's geometry. For the semantic clustering of text, this assumption fails. Returning to the previous example, a corpus of customer feedback can be validly organised by multiple criteria, such as intent, domain, sentiment, or phrasing. A single embedding space captures all these aspects at varying strengths, but a purely geometric algorithm will simply recover whichever structure happens to dominate the space. In this regime, classical clustering algorithms have no input on which aspect of the corpus the clustering is based. Consequently, evaluating a purely geometric algorithm against a single human-annotated benchmark is fundamentally flawed, because the benchmark reflects only one of several valid human criteria. As von Luxburg et al. [2012] argue, clustering objectives cannot be meaningfully evaluated apart from their intended use.

To change the geometry of the encoded text data towards a specific clustering criterion, one option is to fine-tune the embedder itself, such as through the use of instruction-tuned models. This exact strategy of fine-tuning an embedder to increase downstream classical clustering performance is what underpins an active field of research. Progress across this field is primarily tracked by the MTEB benchmark discussed in §2.1.4, as a standard leader board for evaluation of how well different encoders capture a given intended semantic clustering criterion. Another strategy to overcome this anisotropy is to enhance the clustering algorithm itself by using the language understanding of LLMs in the clustering loop (§2.4).

2.2.3 Estimating the Number of Clusters

The clustering algorithms in the previous section can be divided into two main categories: oracle-k and non-oracle-k methods. Oracle-k methods, such as k-means, GMMs, hierarchical methods, and spectral clustering, require prior knowledge of the number of clusters in the distribution (k). Methods such as DBSCAN/HDBSCAN and Leiden/Louvain clustering can automatically infer this value by applying spatial density thresholds or by maximising the modularity score, respectively.

In addition to these methods, many algorithms have been proposed to infer the intrinsic granularity of datasets. A common approach to k-inference is a bottom-up "elbow" approach where as k increases you stop at the point (the "elbow") where adding more clusters stops significantly reducing cluster spread and take this as your granularity [Thorndike, 1953]. Another approach is simply to choose the granularity that maximises the silhouette score (§2.3.1) [Rousseeuw, 1987]; related criteria include the gap statistic [Tibshirani et al., 2001] and X-means [Pelleg and Moore, 2000]. Bayesian nonparametric mixture models treat the number of components as unknown and infer it through Bayesian inference with a nonparametric prior over mixture components [Antoniak, 1974, Rasmussen, 2000]. Finally, stability-based methods infer the k whose clusterings are most reproducible across resamples of the data [Ben-Hur et al., 2002, von Luxburg, 2010].

These granularity estimation methods all rely on the core assumption that one true intrinsic granularity exists for any given dataset. When it comes to text, this assumption is somewhat relaxed. Text corpora can often be clustered at multiple granularities at once, depending on the clustering criterion. For example, a corpus can have a relatively high granularity when clustered for domain, but only have a small amount of clusters that correspond to intent. For benchmark corpora with labelled cluster assignments, the target value of k is determined by the annotation decisions of the researchers who made the benchmark rather than by an intrinsic property of the embedding space, so algorithms designed to infer k are effectively estimating that choice rather than recovering a uniquely correct number of clusters. Consequently, many modern text clustering algorithms require prior knowledge of k (oracle-k), although exceptions exist. This tension is exactly what research question 2 investigates.

2.3 Evaluation of Clusterings

Metrics for evaluating the quality of the clustering of a given dataset can be divided into two main categories: label-free and label-based, where label-free metrics allow us to evaluate purely on geometric grounds, while label-based metrics can only be used in addition to a ground-truth clustering that is usually generated by researchers, which in the case of this thesis comes in the form of benchmark datasets (§2.5).

2.3.1 Label-Free Evaluation Metrics

There are three main label-free evaluation metrics, with comparative studies finding no one index superior across diverse data conditions [Arbelaitz et al., 2013]:

- *The silhouette score* is an effective metric to score how well separated and cohesive the computed clusters are. It compares each point's mean intra-cluster distance to its mean

nearest-cluster distance averaged over all points in the dataset. This yields a score near +1 when clusters are compact and well separated, near 0 when they overlap, and negative when points are misassigned [Rousseeuw, 1987].

- *The Caliński–Harabasz (CH) index* computes the ratio of between-cluster dispersion to within-cluster dispersion, normalised by degrees of freedom for N data points and k clusters. This metric is high and unbounded positive when clusters are dense and well-isolated, and approaches zero when clusters are diffuse and overlapping [Caliński and Harabasz, 1974].

- *The Davies–Bouldin index* evaluates each cluster exclusively against its most overlapping neighbour. It calculates the maximum ratio of combined within-cluster scatter to centroid separation for every cluster and then averages these worst-case scores. Scores approach 0 for compact and widely separated clusters and increase as clusters become diffuse or overlapping [Davies and Bouldin, 1979]. It is therefore a measure of worst-case structural overlap.

2.3.2 Label-Based Evaluation Metrics

The Adjusted Rand Index (ARI) takes all possible pairs of documents in a corpus (or a random subset for computational reasons) and generates the rate at which the clustering and the ground-truth labels agree on each pair (same group in both, or different in both). This is then corrected for the agreement expected by chance so that it takes values in $[-1, 1]$, with an expected value of 0 if clusters were assigned randomly [Hubert and Arabie, 1985].

Mutual Information (MI) $I(U;V)$ measures the mutual dependence between two random variables U and V. Calculating it tells us how much knowing the outcome of one reduces uncertainty about the other. In the context of clustering, it measures the reduction in uncertainty about the ground truth given the predicted clusters (or vice versa). *Normalised Mutual Information (NMI)* normalises the MI by the average of the entropies of the ground-truth label $H(U)$ and predicted class $H(V)$. This bounds the score between 0 and 1, normalising for the inherent uncertainty in both partitions:

$$\mathrm{NMI}(U,V) = \frac{I(U;V)}{\frac{1}{2}\left(H(U) + H(V)\right)} \tag{2.1}$$

When comparing clusterings of different sizes, even completely random assignments will yield a score greater than zero by pure chance. Because this effect increases as the number of clusters grows, NMI artificially increases for more fine-grained clusterings. *Adjusted Mutual Information (AMI)* offsets NMI's bias towards fine grained clustering by subtracting the expected mutual information $\mathbb{E}[I]$ if you were to randomly assign all data points at the same granularity of your chosen clustering [Vinh et al., 2010]:

$$\mathrm{AMI}(U,V) = \frac{I(U;V) - \mathbb{E}[I]}{\frac{1}{2}\left(H(U) + H(V)\right) - \mathbb{E}[I]} \tag{2.2}$$

AMI is therefore the better choice when comparing clustering at different granularities. A related entropy-based alternative is the V-measure [Rosenberg and Hirschberg, 2007].

Accuracy (ACC) finds the best one-to-one matching between predicted clusters and true classes, then calculates the rate of correct assignments:

$$\text{ACC} = \max_{\sigma} \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}[\sigma(c_i) = t_i] \tag{2.3}$$

where N is the total number of data points, c_i is the predicted cluster assignment for the i-th point, t_i is its true class label, σ is a permutation mapping cluster labels to true class labels. This optimal mapping σ is in practice calculated with the compute-efficient Hungarian algorithm [Kuhn, 1955]. One caveat here is that, since matchings are one-to-one, accuracy heavily penalises a clustering that splits one ground-truth class into two clusters, even if the split is semantically reasonable.

2.3.3 Discrepancies Between Label-Free and Label-Based Evaluation on Text

On text embeddings, label-free and label-based evaluation metrics are known to often disagree. A recent comparison of text embeddings found that configurations achieving high label-based metrics could also receive low silhouette and Caliński–Harabasz index scores which indicates good agreement with the ground-truth labels despite relatively low cluster compactness and separation in the embedding space [Petukhova et al., 2025]. Similarly, a statistical analysis of label-free evaluations of datasets in learned embedding spaces concluded that standard label-free metrics are unreliable in the text-clustering regime [Wang and Ye, 2025]. Similar evidence appears in single-cell genomics, where silhouette-based evaluation of learned embeddings is often systematically misleading [Rautenstrauch and Ohler, 2025].

The main consensus is that label-free metrics are not guaranteed to align with human judgement because semantically consistent clusterings often form complex, non-compact geometries in embedding space. This thesis investigates how far the semantic reasoning of LLMs can be used to produce label-free evaluations that agree with human judgement in natural language processing (NLP) clustering.

2.4 LLM-in-the-Loop Text Clustering

Since 2023, a growing field of research known as LLM-in-the-loop clustering has emerged. Proposed methods insert LLMs into dedicated clustering algorithms, using their natural language reasoning to address the gap between semantic clustering and geometric representation discussed in the previous section. Because LLM inference is more computationally expensive than standard distance calculations but cheaper than human annotation, these systems are often designed to optimise where they apply LLMs.

Viswanathan et al. [2024] categorise LLM-in-the-loop clustering algorithms into three stages: before clustering (improving representations), during clustering (constraining assignments), and after clustering (correcting results) and current research largely implements one or more of these strategies. One additional approach discussed in this thesis are fully LLM-native methods that do not include any classical clustering or encoding methods and instead rely solely on the LLM.

The methods discussed below were chosen on three main criteria from the literature. First, the methods must have results published in peer-reviewed papers. Second, the language model must be used inside the clustering pipeline. Finally, the paper must report results on the benchmarks of §2.5, so that a like-for-like comparison is possible. Methods that see any amount of ground-truth labels, such as semi-supervised algorithms, are also excluded.

Adjacent lines of work fall outside these criteria and are not compared: semi-supervised generalised category discovery [Vaze et al., 2022, An et al., 2024, Liang et al., 2024], classification-style LLM clustering with seed labels [Huang and He, 2025], LLM-based taxonomy generation and corpus-mining systems [Wang et al., 2023, Wan et al., 2024, Tamkin et al., 2024], human-alignment studies of LLM clustering [Miller and Alexander, 2025, Hong et al., 2025], and agentic or reinforcement-based clustering [Xu et al., 2025, Löwe and Silcock, 2026, Qing et al., 2026]. Figure 1 shows an overview of the 11 LLM-in-the-loop algorithms compared in this thesis and Table 5 (Appendix A) provides a detailed breakdown of each method's headline configuration, encoder backbone, and language model.

2.4.1 Improving the Representation Before Clustering

Pre-clustering methods implement LLMs to augment document representations before applying any kind of clustering algorithm in order to add more semantic information to the document representations to increase the semantic signal to noise ratio.

- Few-Shot Clustering [Viswanathan et al., 2024] augments document representations with semantically dense information by concatenating the original vectors with encoded LLM-generated keyphrases. To achieve this, it prompts an LLM to generate a given amount of keyphrases for each document. These keyphrases are then encoded using the same encoder as the original text, and the resulting keyphrase vectors are concatenated to the

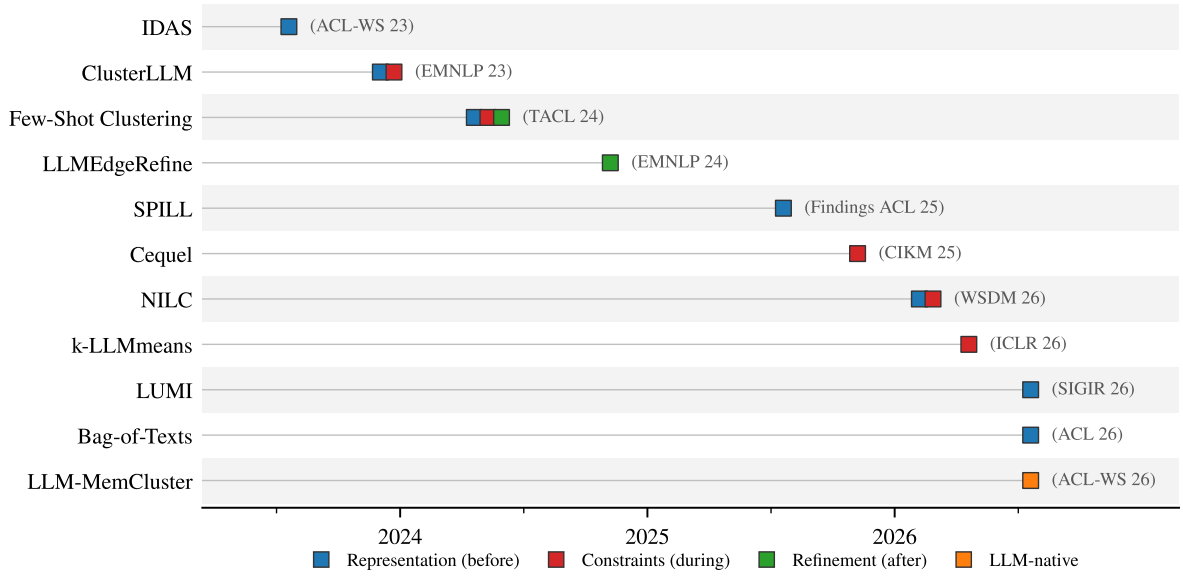


Figure 1: The 11 LLM-in-the-loop algorithms compared in this work by publication date and intervention type.

corresponding original representation vectors, doubling the dimensionality of the embedding space.

- The IDAS algorithm [De Raedt et al., 2023] replaces the original document representation entirely with the vector of an LLM-generated summary. Expanding on the previous idea, it uses an LLM to generate a complete contrastive summary of the document and then represents the document solely by the vector representation of that summary.

- Similarly, the LUMI algorithm [Lin et al., 2026b] enhances representations by averaging the original text vectors with embedded LLM-generated pseudo-labels. It achieves this by using multiple prompts and/or models to generate pseudo-labels for each document, which are then embedded and averaged with the original text vectors.

- The SPILL clustering algorithm [Lin et al., 2025] pulls documents of the same intent closer together by representing each document as an average of its own embedding and LLM-verified same-intent neighbours. Designed specifically for clustering by intent rather than by any general notion of category or meaning, SPILL first generates a candidate pool for a given target document containing its nearest neighbours in embedding space. For each neighbour in the pool, an LLM is prompted to judge whether that candidate expresses the same general intent as the target. The encoded vectors of the accepted candidates are averaged together with the target's own embedding, so that each document is represented by a small, LLM-verified group of same-intent documents rather than by its own text alone.

- The Bag-of-Texts approach [Lin et al., 2026a] attempts to completely eliminate dependence on the original geometry by replacing the notion of distance in embedding space with an LLM-judged similarity to a set of representative anchors. This is done by first picking 1,024 representative documents (anchors) as the medoids of a hierarchical clustering at $k = 1{,}024$. These documents are then treated the same way bag-of-words algorithms treat a vocabulary, where each document represents a one-hot-encoded vector in a 1,024-dimensional vector space. For each remaining document, an LLM is then shown the document together with a shortlist of the anchors most similar to it and is asked: which of these, if any, express the same thing? The ticks on the checklist become the document's new representation in 1,024-dimensional space. For example, a complaint about app crashes might tick a given set of anchors whereas another similar app crash complaint that is phrased differently will usually also tick the same set of anchors. As such, the two documents receive nearly identical representations even though they might have very different wording. With this mechanism, distance in this new space describes how much an LLM judged the two documents to belong together. Regular clustering then recovers those groups that have been judged to be similar.

- ClusterLLM [Zhang et al., 2023b] fine-tunes the sentence encoder itself instead of augmenting the data representations directly, using LLM judgments on difficult similarity triplets. Specifically, an LLM is used to generate answers to triplet questions of the form "which of B or C is more similar to A?". For example, given the anchor A = "the app keeps crashing" and the candidates B = "app freezes right after start-up" and C = "my delivery arrived two days late", the LLM answers B. The encoder is then fine-tuned so that its distances

agree with the answer. As such, A is "pulled" towards B and away from C in vector space. These triplets are sampled close to the boundaries between clusters, where the current representation of the corpus is most uncertain about cluster assignment. As such, only the difficult examples are adjusted. This procedure is also repeated over several rounds with the two different embedders (INSTRUCTOR and E5), so there are three main variants of the algorithm: ClusterLLM-I (INSTRUCTOR, one round), ClusterLLM-I-iter (INSTRUCTOR, iterated), and ClusterLLM-E-iter (E5, iterated).

2.4.2 Guiding Assignment During Clustering

The second approach is to integrate the LLM directly into the clustering objective of classical clustering algorithms to change them into constrained optimisation problems [Wagstaff et al., 2001]. The constraints come in the form of *must-link* constraints (these two documents belong in the same cluster) and *cannot-link* constraints (they do not) generated by the LLM, which the algorithm is rewarded for satisfying.

- Few-Shot Clustering uses an LLM to generate must-link and cannot-link constraints that penalise a k-means objective. It first selects document pairs from the corpus by the Explore–Consolidate strategy [Basu et al., 2004]. These pairs are then passed to an LLM through a prompt requesting a must-link or cannot-link label per pair. The generated constraints are passed to pairwise-constrained k-means (PCKMeans), which minimises

$$\mathcal{J} \;=\; \sum_{x_i \in \mathcal{X}} \|x_i - \mu_{l_i}\|^2 \;+\; \sum_{(x_i,x_j)\in\mathcal{M}} w\,\mathbf{1}[l_i \neq l_j] \;+\; \sum_{(x_i,x_j)\in\mathcal{C}} \bar{w}\,\mathbf{1}[l_i = l_j]\,,$$

  where $\mathcal{X}$ is the embedded corpus, l_i is the cluster assigned to document x_i, and μ_{l_i} its centroid. The first term is the ordinary k-means objective (within-cluster variance). The second adds a penalty w for every must-link pair in $\mathcal{M}$ that is split across clusters. The third is a penalty $\bar{w}$ for every cannot-link pair in $\mathcal{C}$ placed in the same cluster. As such, the judgement of the LLM is traded off against the classical embedding distances with the trade-off rate set by the penalty weights w and $\bar{w}$ [Viswanathan et al., 2024].

- The Cequel algorithm [Wang et al., 2025] explicitly trades off clustering quality with LLM cost by strategically targeting the most ambiguous relations for constraint generation. It works similarly to Few-Shot clustering but optimises for LLM-call cost by starting with a fixed maximum number of queries and adjusted spending strategies. Cequel begins by generating a similarity graph over the corpus where each document vector is a node and every edge weighted between nodes is their embedding cosine similarity. Two spending strategies then exist. EdgeLLM starts by scoring each edge by its spanning edge centrality (the measure that is high for edges bridging otherwise weakly connected regions and low for edges inside dense neighbourhoods). It then greedily chooses a given amount of pairs with the highest centrality as these represent the most ambiguous pairs of documents. TriangleLLM selects triangles of three documents in the same greedy way and gets the relation between every two of them in a single LLM call, so one query yields three pairwise

constraints rather than one. These labelled must-link or cannot-link pairs are then used in a pairwise constrained k-means algorithm as before.

- k-LLMmeans [Díaz-Rodríguez, 2026] modifies the standard k-means algorithm by replacing geometrically calculated centroids with embedded LLM-generated cluster summaries. It achieves this by inserting an LLM into the recursive loop. Instead of recursively recalculating the centroids as the mean of cluster assigned points, k-LLMmeans samples a few documents from the cluster and prompts an LLM to write a short summary of the cluster's documents. This summary is then embedded and taken as the updated centroid. With this adjustment, cluster centres are chosen semantically rather than geometrically, remain human-readable, and are less sensitive to outlying members.

- NILC [Wang et al., 2026] extends k-LLMmeans by guiding assignment with both semantic and geometric centroids simultaneously, and by having the LLM rewrite ambiguous documents. First, LLM descriptions are taken alongside regular Euclidean distance centroids as dual centroids so that point assignment in the next round is guided by both. Second, it rewrites a small set of ambiguous documents. After each clustering round, every document's distances to the k centroids are converted through a softmax into an assignment distribution, and the documents with the highest entropies are selected as ambiguous samples. The LLM is then shown each ambiguous document along with its centroid summary and is prompted to rewrite it more explicitly in that context.

2.4.3 Refining Clusters After Clustering

Post-clustering refinements use LLMs to reassign ambiguous points that lie far from their cluster centroids in embedding space after the main clustering has been performed.

- In addition to pre-clustering improvements, Few-Shot Clustering [Viswanathan et al., 2024] also reclassifies points after clustering. This is done by identifying the n points with the smallest margin between their distances to the nearest and second-nearest centroids. For each of these points, an LLM is then prompted to decide which of the two clusters it belongs to by showing it representative documents (points close to the centroid) from each. Points are finally reassigned based on the LLM's decision.

- LLMEdgeRefine [Feng et al., 2024] targets the same low-confidence points in a two-stage process starting from a classical k-means clustering of the embedded documents. In the first stage, the points furthest away from their centroid are identified and their corresponding text documents are combined into one larger document. This concatenation of documents is then embedded as one "super point" and it is used to replace the original points. All the data is then reclustered agglomeratively. This stage simultaneously stops outliers from dragging their cluster's centroid towards the edge and lets a group of stray points migrate together. The LLM is then used in the second stage where over a few iterations, the points now furthest from each refined centroid are individually reassessed as before.

Post-clustering refinement methods are useful because can easily be combined with every other family in this section. However, one unexplored risk does exist when implementing these types of

methods. Iterated LLM-guided correction optimises agreement with the LLM's own notion of the taxonomy, which may drift from the analyst's (in this work the researchers that generated a given benchmark dataset). Section 2.6.2 discusses an analogous, better-documented bias in LLM evaluation. This question is also empirically investigated in Chapter 3.

2.4.4 LLM-Native Clustering

LLM-Native clustering completely removes the geometric aspect and has the LLM itself perform the entire clustering. The main issue with a fully LLM-native approach is that each LLM call is stateless where each call begins with no recollection of the previous one so a model prompted to group a corpus batch by batch (which is necessary for large corpora due to a limited context window) cannot keep its groups consistent. LLM-MemCluster [Zhu et al., 2026] solves this by supplying the LLM explicitly with a dynamic memory (the list of clusters discovered so far with their labels) at each prompt. The LLM can then assign documents to existing clusters, open new clusters for genuinely distinct topics, and merge or rename existing ones and the updated dynamic memory is then carried into the next LLM call.

Two main problems arise with this approach, however. First, each document needs to be passed through an LLM, these methods get significantly costly for large corpora, which is something encoder-based pipelines avoid by using classical clustering as a base. Second, with no encoder in the loop, representational quality and LLM reasoning quality can not be disentangled from each other, and thus no encoder-matched comparison can be made to other clustering techniques.

2.5 Benchmarks for Text Clustering

2.5.1 The Standard Evaluation Suite

Because unlabelled corpora by themselves do not allow for quantitative evaluation outside of label-free evaluation metrics, which have been shown to break down in the regime of text clustering (§2.3.3), a line of benchmarks has been proposed as a standard evaluation suite in the literature. These clustering benchmarks mostly repurpose classification datasets, where the method sees only the input and uses the class labels only afterwards as cluster labels. A fairly stable and widely used suite has emerged of mostly intent, domain and topic benchmarks. Every clustering algorithm proposed in the field's literature is evaluated on a specific subset of these benchmarks. This thesis compares against 19 of these benchmarks, namely every corpus in the suite for which a peer-reviewed clustering result has been published.

Six of them are intent benchmarks: Banking77 [Casanueva et al., 2020], CLINC(I) [Larson et al., 2019], MTOP(I) [Li et al., 2021], MASSIVE(I) [FitzGerald et al., 2023], SNIPS [Coucke et al., 2018] and M-CID [Arora et al., 2020] with the last covering COVID-19 queries. Three further benchmarks reuse the CLINC, MTOP and MASSIVE corpora but label them by domain instead of intent, giving CLINC(D), MTOP(D) and MASSIVE(D). The documents are identical for MTOP and MASSIVE, while CLINC(D) drops the 1,000 out-of-scope utterances that carry no domain label. Because only the labels differ on the shared documents, these three pairs of benchmarks show that a corpus has no single correct clustering independent of the specific clustering criterion, as discussed in §2.2.2.

Six are topic benchmarks: BBC News [Greene and Cunningham, 2006b], Tweet89 [Yin and Wang, 2016] and StackOverflow [Xu et al., 2017], together with ArXiv-S2S, Reddit-S2S and StackEx-S2S, three corpora re-split for clustering by Zhang et al. [2023b]. The remaining four ask for organisations that are not topical at all: DBPedia [Zhang et al., 2015] and FewNerd [Ding et al., 2021] group by entity type, FewRel [Han et al., 2018] by the relation between two entities, and FewEvent [Deng et al., 2020] by event type. Together, the suite covers six different criteria, which is what makes it a fair test of a method that is supposed to work for any stated criterion.

Figure 2 shows a visualisation of these benchmarks by how many clusters they represent and how many documents are present. Cluster counts range from 5 to 151. Table 6 (Appendix A) lists the cluster and document counts for the benchmarks discussed above.

2.5.2 Benchmark Coverage and Comparability Across Methods

An extensive survey of published literature reveals that, although a thorough suite of shared benchmarks exists, coverage is fragmented. Each algorithm discussed in §2.4 is only evaluated on a subset of these benchmarks, and the subsets overlap surprisingly little. Across the 14 method variants and 19 benchmarks considered in this work, only 33% have been reported. Algorithm coverage is also relatively uneven across the benchmark suite. Banking77 and CLINC(I) evaluate 12 of the 14 variants, while benchmarks such as BBC News, DBPedia, M-CID, SNIPS, StackOverflow, and Tweet89 evaluate only two or fewer. However, the evaluation metrics are more consistent than the coverage. Every method reports ACC and NMI, and therefore this work focuses on those two. Figures 13 and 14 in Appendix A.2 show every published ACC and NMI value by method variant and benchmark.

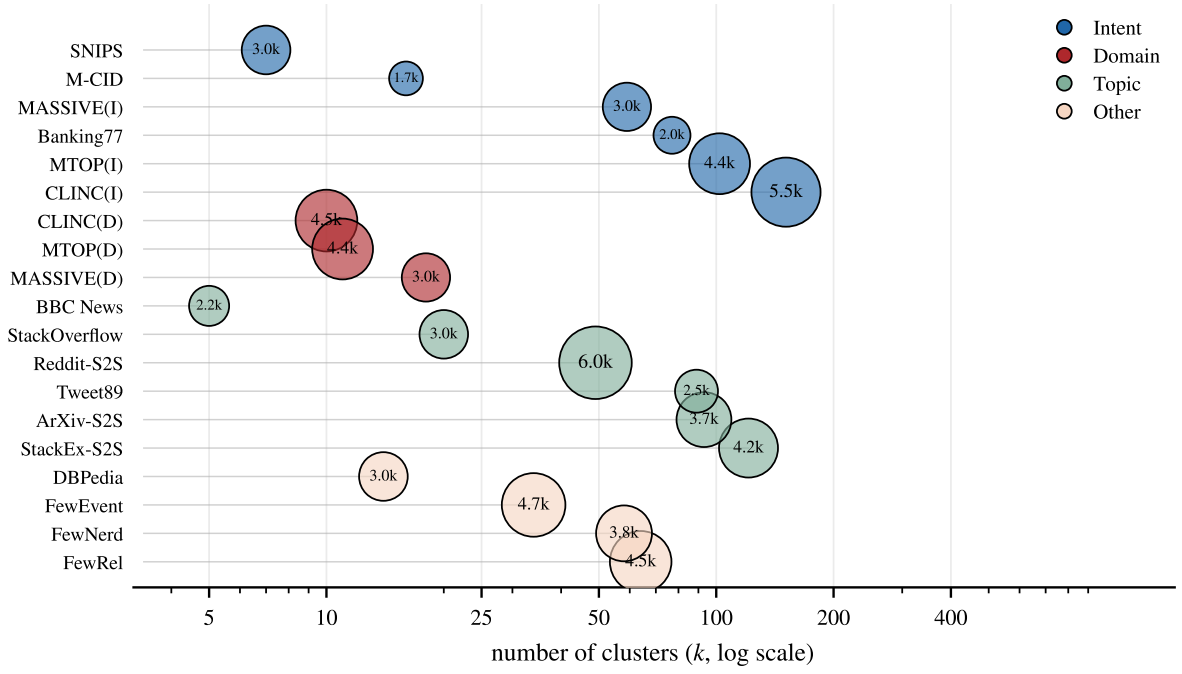


Figure 2: Overview of the benchmark clustering corpora covered in this work by criterion family, granularity and number of documents.

Another important caveat is that the underlying encoders and LLMs vary significantly across reported algorithms. Results are reported on DistilBERT, INSTRUCTOR, E5, proprietary OpenAI embeddings, or no encoder at all. Subsample sizes, seed counts, and reporting precision also vary across papers. Granularity can also be handled slightly differently. Most methods rely on a known k, while others infer granularity from the data itself, which is a significantly harder condition to evaluate against.

A final inconsistency across the literature is benchmark naming and exact which exact documents each method is evaluated against. For example, ClusterLLM re-split the arXiv, Reddit and StackExchange corpora, yielding benchmark granularities (93 arXiv subcategories, 121 StackExchange communities) that differ from the MTEB versions of the same-named benchmarks [Zhang et al., 2023b]. The benchmark "StackOverflow" may also denote either the 20-class short-title set [Xu et al., 2017] or a StackExchange community task across different research. As such, two seemingly matched evaluations may not actually be comparable at all.

This work argues that a fair comparison of methods requires evaluation on the exact same setup (documents, encoders and LLMs). A comparison based only on headline results is therefore not rigorous. To the author's knowledge, no published work in this literature has applied such a matched protocol systematically as of September 2026. This work therefore develops a rigorous testing procedure that accounts for these variables.

2.5.3 The State of the Art

For each of the 19 benchmarks and separately for each metric, the state of the art (SOTA) results discussed in this work are chosen as the highest value published by any of the methods in §2.4, using each method's best-performing variant on the standard evaluation suite defined in §2.5.1. Figure 3 shows the resulting SOTA values for ACC and NMI. For the reasons set out in §2.5.2, these values are not strictly comparable to one another and are therefore treated as reference points instead of a ranking.

2.6 LLMs as Evaluators

2.6.1 The LLM-as-a-Judge Paradigm

In addition to their use inside pipelines for algorithms across multiple domains, LLMs have been shown to be useful as evaluators of such pipelines and thus show promise as replacements for expensive human evaluation. Zheng et al. [2023] showed close correspondence between GPT-4 judgements and human preferences on open-ended generation at rates comparable to human–human agreement. This result established the "LLM-as-a-judge" paradigm. LLMs have also been shown to match and even exceed crowd workers on classification in certain tasks [Gilardi et al., 2023].

This growing field has rarely been applied to clustering, although there are clear reasons to do so. Human judgements of topic quality have, for example, been shown to diverge from the statistical measures used to evaluate topic models [Chang et al., 2009]. ProxAnn [Hoyle et al., 2025] replaces human annotators with LLMs that execute use-oriented judging protocols for clustering output and shows that their results correlate well with those of humans on the

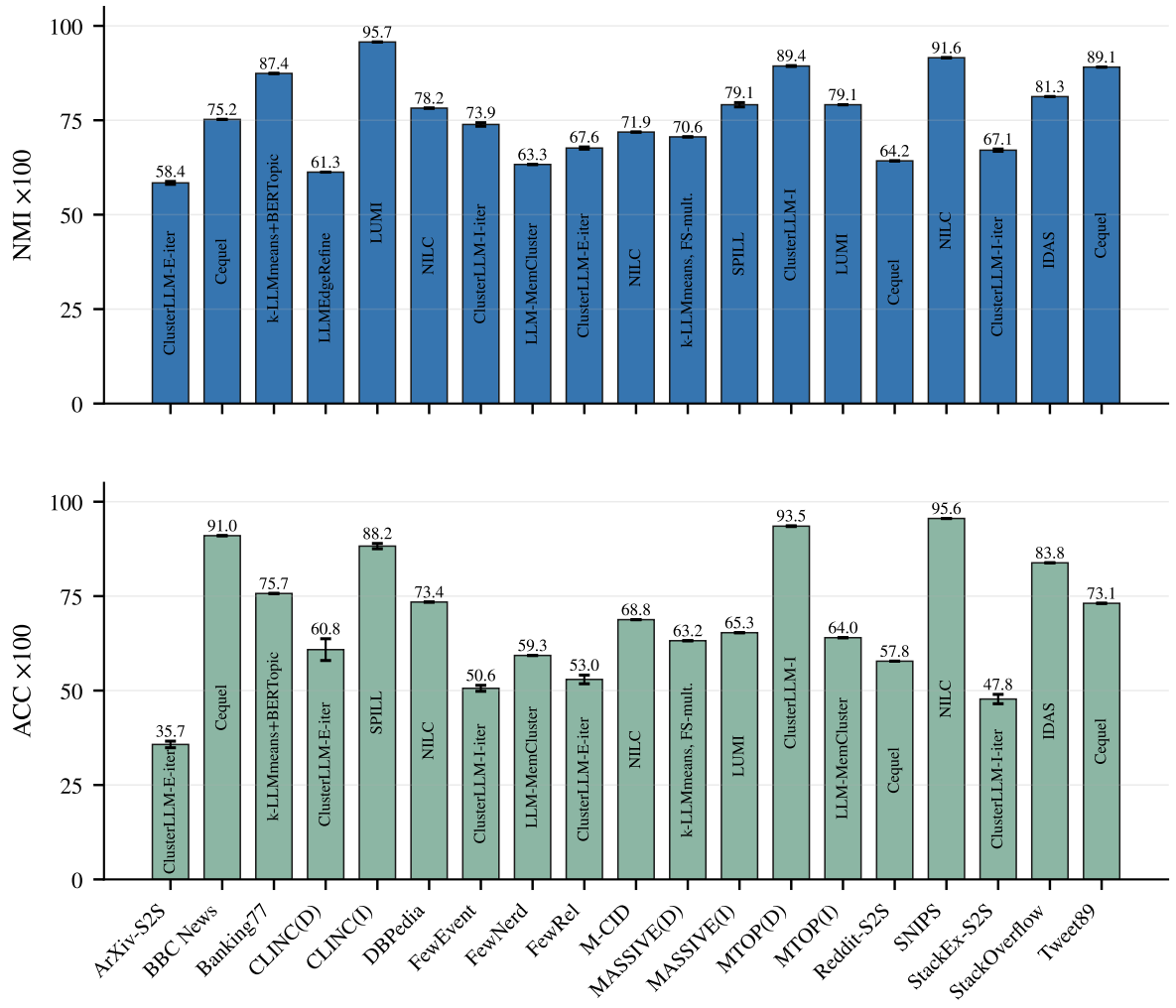


Figure 3: The best published NMI (top) and ACC (bottom) for each of the 19 benchmarks, with the clustering algorithm holding each result named inside the bar. Error bars show the standard deviation reported by the original research paper.

same tasks. However, no large-scale, clustering-specific investigation has been performed on this matter. At the time of writing, this work has not identified any literature that employs an LLM evaluator inside the clustering loop itself as a signal for choosing between candidate clusterings. This is investigated in Chapter 3.

2.6.2 Documented Biases and Failure Modes

Previous research has shown that LLM-as-a-judge methods (outside clustering) have critical biases that must be accounted for. Specifically, judge models have been shown to exhibit self-enhancement bias (preferring their own outputs), position bias (preferring answers by their order of presentation; Wang et al., 2024) and verbosity bias (preferring longer answers) [Zheng et al., 2023]. These biases can be partly mitigated by calibration strategies such as averaging several judgements in which the model reasons before scoring. Rubric-based judges have also been shown to have a higher preference for LLM-generated over human-written text [Liu et al., 2023]; Gu et al. [2024] survey these failure modes systematically.

Whether these biases extend to clustering remains largely uninvestigated. Another uninvestigated failure mode in clustering is the circularity that may arise when the same LLM is used to both cluster and evaluate a clustering. This breaks independence since the pipeline could possibly raise its own judge score by moving closer to whatever the model already believes.

2.7 Research Gap

This work addresses four deficits in current text clustering research.

First, there is a clear lack of label-free clustering-quality validation metrics that is shown to correlate with human judgement. The only widely used label-free metrics in clustering are geometry-based (§2.3.1) and have been demonstrated ineffective in the NLP regime due to the lack of compact, separable clusters (§2.3.3). It remains untested whether LLM-human agreement persists when geometric compactness and separation are controlled. This discrepancy prevents evaluation or tuning of state-of-the-art clustering models in production environments. Research question 1 addresses this challenge.

Second, granularity is typically treated as a fixed input. As such, many state-of-the-art methods require the ground-truth k. Classical methods for estimating k therefore also start from this assumption, which contradicts multi-granular text corpora (e.g. CLINC utterances form 10 domain clusters or 151 intent clusters). Granularity in a natural language context is therefore a function of the criterion used for clustering. The current literature offers little research on whether an LLM, given a clustering criterion, can recover the annotator-selected granularity. Determining whether k can be inferred establishes whether an LLM-powered clustering pipeline will always require it as a manual input. Research question 2 explores this problem.

Third, no current state-of-the-art LLM-in-the-loop algorithm allows the clustering criterion to be fully exploited. Instruction-tuned encoders [Su et al., 2023] condition the representation on the criterion but leave the clustering objective and LLM refinement unconditioned. An LLM is also mostly inserted into the pipeline for pre-clustering representation augmentation and post-clustering refinement. Methods that employ LLMs directly within the clustering process do so by enhancing a single given clustering algorithm such as k-means. No current method uses the reasoning abilities of LLMs to make active decisions on which enhanced version of different types of clustering algorithms is optimal. Research question 3 addresses this by using the LLM as a criterion-aware partitioner and judge.

The final gap in current research does not concern the clustering methods themselves, but rather the lack of fair comparison between them. Current state-of-the-art results lack direct comparability because benchmark coverage is completely fragmented (see Figures 13 and 14 in Appendix A.2). Existing methods with state-of-the-art results also rely on different encoders and LLMs. The values in §2.5.3 therefore serve as reference points rather than a standardised ranking. To resolve this, in research question 4, as much effort as possible is put into making a

fair matched same-benchmark, same-LLM, and same-encoder comparison to other algorithms in the current literature.

Chapter 3

Experiments

All experiments in this chapter are implemented in Python, using scikit-learn [Pedregosa et al., 2011], SciPy [Virtanen et al., 2020], NumPy [Harris et al., 2020], Sentence-Transformers [Reimers and Gurevych, 2019] and Matplotlib [Hunter, 2007].

3.1 Research Question 1

Given only the criterion underlying a human-generated clustering of a corpus, do the scores of an LLM acting as a judge correlate with human judgement? Is any such correlation more than a restatement of what label-free geometric metrics already measure?

Human judgement of a candidate clustering of a corpus in this methodology is approached as the agreement (calculated using Adjusted Mutual Information) with the human-generated benchmark clustering of that corpus itself. The benchmark's labels record how human annotators (using the criterion) actually clustered the corpus. As such, we can say that a candidate clustering is judged well by human standards if it agrees with those labels. The question then becomes whether the LLM's scores correlate with each candidate's AMI with those human-generated benchmark labels. Geometric quality is a confound that needs to be taken into account. A candidate with compact, well-separated clusters in embedding space could score high for both the judge and the benchmark labels for the same latent geometric reason, which would inflate this correlation.

These questions are answered in four stages. First, an LLM-as-a-judge algorithm is proposed, which takes a clustering criterion and a candidate clustering of a corpus as input and outputs a score for that clustering. Second, an extensive test bed of candidate clusterings is generated for each of the 19 benchmark corpora. Third, we evaluate these same clusterings using purely geometric metrics. Finally, we measure how strongly the judge's scores correlate with each clustering's agreement against the benchmark's labels and repeat that method with the geometric metrics controlled for, so that any correlation that survives this control cannot be explained by geometry alone. The whole analysis is then repeated with five different judge LLMs to test whether the answer depends on the model.

3.1.1 The Codebook Judge

Design

The codebook judge is designed to take as input both a candidate clustered corpus D where each document is hard-assigned by $y : D \to \{1, \ldots, k\}$ to one of k clusters, and a one-sentence clustering criterion c. These clustering criteria are unique to the input corpus and in this work, each benchmark has been given a pre-determined criterion sentence. For example, any candidate clustering y of the Banking77 corpus is input into the codebook judge algorithm alongside the criterion sentence: "which banking customer-service intent the query expresses". A full breakdown of the exact criterion sentences for each of the 19 benchmarks can be found in Table 7 (Appendix B).

Each criterion sentence was written by the author based on each corresponding benchmark's own documentation description of the labelled documents. Concretely, the annotation target described in each benchmark's dataset paper was rephrased into a criterion sentence of the form "which *category type* the *document type* expresses or belongs to". Thus the sentence names only the organising principle and the kind of document and deliberately says nothing about the number or the names of the categories. All 19 criterion sentences in Table 7 follow the same grammatical template, so that any of them can be inserted verbatim into the fixed prompt templates used throughout the pipeline. The codebook judge is then designed to take these two inputs and output an AMI-based score, typically between 0 and 1, for the candidate clustering.

The judge algorithm itself follows a simple three stage process. First, 300 random (unlabelled) documents S are sampled from the input corpus D. These documents are then combined with the clustering criterion c into a single prompt using the template in Figure 15 in Appendix C. For a given corpus and criterion, this prompt is then fed into a chosen LLM with reasoning enabled, producing the codebook B.

Once this codebook is generated, 1,000 documents S' (disjoint from the 300 sampled before) are again sampled uniformly at random from the corpus, and each document is assigned to one of the codebook categories using the prompt template in Figure 16 in Appendix C, which has been populated with the codebook categories/definitions of B generated previously and the clustering criterion c itself. As such, one LLM call (reasoning disabled) is made per sampled document, which returns a single category id, with 0 reserved for "fits none" documents that the LLM has decided do not fit any of the category members. This stage then results in a reference clustering R of $(1{,}000 - n_{\text{fits-none}})$ documents.

Two more hard-coded diagnostics are also applied to this reference clustering to maintain its quality. First, no single category is allowed to contain more than 50% of the documents, and second, no more than 20% of the documents are allowed to be categorised as "fits none". If either of these criteria is not met, the output of the algorithm will include a degeneracy warning to the user.

Finally, to compare this reference cluster assignment R and the candidate clustering y, all documents in the candidate that are not included in the reference are discarded, leaving the shared document set D^*. This allows us to make comparisons between both without requiring any benchmark labels, as the reference labelling takes the place of a ground truth. The Adjusted Mutual Information (§2.3.2) between the reference and candidate cluster assignments over D^* is

then calculated as the judge's output score since it allows for different numbers of clusters between clusterings. This final step requires no further LLM calls, and as such, for a given corpus, the reference labelling R is calculated once. Using this reference, many different candidate clusterings can be evaluated at no additional LLM cost.

Any LLM can be used for this algorithm. In this methodology the full protocol is run with five different judge LLMs, all at temperature zero: `gemini-2.5-flash` [Gemini Team, Google, 2025], `glm-5` [Zhipu AI, 2026], `gpt-4o` and `gpt-4o-mini` [OpenAI, 2024b], and `gpt-3.5-turbo` [OpenAI, 2023]. Assignment is one call per document for `gemini-2.5-flash` and runs in batches of 20 documents per call, with a batched variant of the assignment prompt, for the other four judges.[1] This algorithm design is summarised in Algorithm 1.

Algorithm 1 The codebook judge

Require: corpus D; criterion sentence c; candidate clustering $y : D \to \{1, \ldots, k\}$
Ensure: judge score for y

procedure BUILDREFERENCE(D, c)
  $S \leftarrow$ 300 documents sampled uniformly from D
  $B \leftarrow$ LLM(codebook-discovery prompt(S, c)) ▷ reasoning on (Figure 15)
  $S' \leftarrow$ 1,000 documents sampled uniformly from $D \setminus S$
  for each document $d \in S'$ **do**
    $r(d) \leftarrow$ LLM(assignment prompt(d, B, c)) ▷ reasoning off (Figure 16)
  end for
  $R \leftarrow \{(d, r(d)) : d \in S',\ r(d) > 0\}$ ▷ the reference labelling
  return R
end procedure

procedure JUDGE(y, R)
  $D^* \leftarrow$ documents appearing in both R and y
  return AMI$\big(R|_{D^*},\ y|_{D^*}\big)$
end procedure

3.1.2 Corpora

The same exact 19 corpora discussed in §2.5.1 are used to test all four research questions, which keeps the methodology consistent while ensuring diversity across intent, domain, topic, entity, relation, and event criteria. The human-annotation labels included in each benchmark corpus in this section are only used to compare to human-reference metrics and are never "seen" by the codebook judge algorithm. All 19 corpora are embedded once with a single frozen encoder (`instructor-large`) with the generic instruction *"Represent the sentence(s) for clustering: "*, and these same encodings are shared by every candidate clustering and every geometric metric below.

All embedding vectors in this work are additionally ℓ_2-normalised to unit length, and whenever a distance in representation space is mentioned, it refers to the cosine distance between these normalised vectors. On unit-length vectors, Euclidean distance is a monotone function of cosine

[1] Two model-specific deviations apply. For `glm-5`, reasoning is disabled on the codebook call, as its reasoning traces regularly overran the reply limit. For `gpt-3.5-turbo`, whose context window is smaller, discovery documents are clipped to 150 characters on long-document corpora, and a codebook reply that repeatedly truncated falls back to accumulating the codebook over 20-document chunks.

distance, so the Euclidean objectives of the clustering algorithms optimise the same orderings as this cosine distance. The one exception is the 50-dimensional PCA projections used by some candidates in §3.3.1, which are not unit-length and operate in ordinary Euclidean space.

3.1.3 Candidate Clusterings

To evaluate the correlation between the judge's scores and each candidate's agreement with the benchmark's human clustering, a diverse test set of different clusterings of all 19 benchmarks was produced. These allow for a base of candidate clusterings of different qualities, granularities and corpora for which the benchmark labelling is known and which we can then evaluate using the judge algorithm and the benchmark labels. For each corpus, we generated a set of five algorithms, one from each family of geometric bias reviewed in §2.2.1 (k-means, a Gaussian mixture model, spectral clustering, Ward's agglomerative clustering and Leiden community detection), crossed with five granularities ($k_{\text{bench}} \times \frac{1}{4}, \frac{1}{2}, 1, 2, 4$) and three random seeds, all on the identical frozen embeddings. This yielded 1,222 candidate clusterings across the 19 corpora (after removing the runs that received a degeneracy warning from the codebook judge algorithm).

3.1.4 Statistical Protocol

The primary metric used to quantify the correlation between the judge's scores and human evaluation is the Spearman rank correlation coefficient, ρ, between the judge's scores and each candidate's AMI with respect to the benchmark labels. This value of ρ is calculated by first converting both quantities for all n candidates to ranks (from low to high) such that, for a given benchmark, each candidate clustering i is assigned an AMI with respect to the benchmark and a judge score (its AMI with respect to the LLM's reference clustering). The coefficient is then calculated as the Pearson correlation between these two rank sequences. In the absence of tied ranks, this is equivalent to

$$\rho = 1 - \frac{6 \sum_{i=1}^{n} \delta_i^2}{n(n^2 - 1)},$$

where δ_i is the difference between candidate i's two ranks. A ρ of $+1$ means the judge orders the candidates by AMI exactly as their benchmark agreement does. A ρ of 0 means there is no monotonic relationship, and a negative ρ means the judge tends to prefer worse candidates, so an inverse relationship exists. In addition to this coefficient, which is calculated for each benchmark, the permutation p-value is also calculated by randomly shuffling the judge scores 10,000 times and recalculating ρ each time. The p-value is then the fraction of shuffles that produce a correlation at least as extreme as the observed one, representing the probability, under the null hypothesis of no association, of obtaining such a correlation by chance. If, for a given benchmark, the p-value associated with its ρ value is smaller than the chosen significance level, $\alpha = 0.05$, the correlation is considered statistically significant. These per-benchmark tests are reported uncorrected. Under a Bonferroni correction across the 19 benchmarks ($\alpha = 0.05/19$), every judge remains significant on at least 18 of the 19 corpora, so the reported counts are not an artefact of multiple testing.

For the geometry-controlled analysis, first the silhouette score and the Caliński–Harabasz index are calculated for each candidate clustering of a given benchmark. The Davies–Bouldin index of §2.3.1 is not used as a third control, since it is built from the same within-cluster

and between-cluster dispersion quantities as the other two and therefore adds no independent geometric axis. For a given geometry control, say the silhouette score, the analysis has three steps. First, the judge scores, AMI versus benchmark scores, and the geometry control are converted to ranks, like before. Second, a linear regression model is fitted by least squares to predict each candidate's judge rank from its control rank. For each candidate, the residual on the final model is kept:

$$e_i^{\text{judge}} \;=\; \mathrm{rk}(J_i) - \big(a\,\mathrm{rk}(C_i) + b\big),$$

where $\mathrm{rk}(J_i)$ and $\mathrm{rk}(C_i)$ are candidate i's judge and control ranks, and a, b are the fitted line's coefficients. This process is repeated for AMI versus benchmark, and the residual e_i^{AMI} is computed in the same way. Now that we have two residuals per candidate, we repeat the same process from above, where a Spearman rank correlation coefficient and an associated p-value are calculated between both residuals. If the judge's scores were simply a restatement of the geometric metric, the fitted line would predict them almost perfectly; the residuals would be close to random noise, and therefore their correlation would be close to zero. Any correlation that remains actually shows the agreement between the judge and human ranking that the controlled geometric metric is unable to account for.

The same method is additionally applied with both geometric metrics controlled at once. In this joint version, the regression predicts each rank from the ranks of the silhouette score and the Caliński–Harabasz index together. A correlation that survives this joint control therefore cannot be explained by either geometric metric alone or by both together.

These analyses are done on a per-benchmark basis, and at no point are all the candidates pooled. This was a deliberate choice to mitigate the risk of confounding due to Simpson's paradox. Simpson's paradox is a statistical phenomenon where a clear trend observed within isolated groups disappears or even reverses when the data is pooled. In this context, the choice of benchmark itself could act as a confounding variable. These benchmarks have different attributes, such as clustering difficulty and the number of documents. If all data were pooled, some candidates evaluated on easy benchmarks could, for instance, falsely appear to outperform a better candidate who was simply evaluated on a more rigorous benchmark, and this could then completely change a trend. By restricting to a per-benchmark basis, we ensure that these confounding variables do not influence the outcome. The single headline result was therefore not chosen to be the ρ over all candidates pooled, but instead the median over all benchmark-specific correlation coefficients. Since the candidates within a benchmark share embeddings, algorithms and seeds, they are not fully independent, and the permutation p-values should therefore be read as evidence that the judge's ranking matches the benchmark ranking beyond chance, not as tests on independent observations.

Finally, because an LLM call is inherently random, every judge run was repeated ten times independently, with each repetition drawing a fresh discovery sample, codebook, and reference labelling. Each benchmark-specific coefficient entering the median above is the mean of that benchmark's ten per-run coefficients. The run-to-run standard deviation of each reported statistic, recomputed within each repetition, is reported alongside it, and a benchmark's correlation is considered significant when it reaches $p < 0.05$ in a majority of the ten repetitions.

3.2 Research Question 2

To what extent can LLMs, given a clustering criterion, infer the granularity of purpose-built human taxonomies from unlabelled data?

As discussed in Section 2.2.3, the granularity of a corpus clustering is ambiguous in itself and depends on the criterion by which the clustering is performed. This experiment asks whether the natural language reasoning of LLMs, given that criterion, can be exploited to resolve the ambiguity. Eight LLM-based granularity estimators are compared: two are adaptations of published methods, one adapts a classical diversity estimator to LLM judgements, and five are of this work's own design. Each method runs on the full benchmark suite with multiple independent repetitions, and the inferred granularities $\hat{k}$ are compared to the benchmark granularities k to test whether the criterion alone is enough for an LLM to infer the granularity a human chose.

3.2.1 Experimental Design

Eight estimation methods are run on each of the 19 corpora in the benchmark suite, with ten independent repetitions per pair for a total of 1,520 runs. For all methods, the same frozen encoder (`instructor-large` with the generic instruction *"Represent the sentence(s) for clustering: "*) and LLM (`gemini-2.5-flash` at temperature zero) are used. The exact same benchmark-specific criterion sentences (Table 7 in Appendix B) are applied as before. Like before, LLM calls that generate a single quality-critical output (a codebook, a final deduplication, a numeric estimate) run with the model's reasoning enabled and simple yes/no comparison calls run with reasoning disabled. To test whether the findings depend on the LLM, the full eight-method protocol is additionally rerun with `gpt-4o-mini` and `gpt-3.5-turbo`.

Granularity Inference Methods

1. **Codebook:** The Codebook method replicates the first stage of the codebook judge of research question 1 exactly, where an LLM is prompted to generate a codebook of categories based on a 300-document sample and the criterion (Figure 15 in Appendix C). The inferred $\hat{k}$ is then simply taken as the number of categories.

2. **Direct:** The Direct method is a simplification of the Codebook method in which an LLM is shown 300 documents and the criterion but is simply prompted to guess the number of categories directly. The full prompt is shown in Figure 17 in Appendix C.

3. **Probe:** The Probe method is an extension of the Codebook method where the LLM is shown the same 300 documents and criterion, but is asked to generate three different codebooks: one with "coarse granularity", one with "fine granularity" and one with "natural granularity" (Figure 18 in Appendix C). The output number of clusters $\hat{k}$ is then chosen to be the number of categories of the one with natural granularity. This forces the LLM to reason about different possible notions of granularity for the same criterion instead of generating a codebook in one shot.

4. **Pairwise:** The Pairwise estimator is an adaptation from the ClusterLLM [Zhang et al., 2023b] algorithm. First, a grid of candidate granularities is constructed as 25 logarithmically spaced points on $[2, 200]$ rounded to integers for a total of 23 unique $\hat{k}$ values:

$$\mathcal{K} = \{2, 3, 4, 5, 6, 8, 9, 11, 14, 17, 20, 24, 29, 36, 43, 52, 63, 77, 93, 112, 136, 165, 200\}.$$

A Ward hierarchy is then built over a 2,000-document subsample of the corpus embeddings. At each level $k \in \mathcal{K}$, the two clusters that merge at that level are identified. From each cluster pair, three document pairs are then sampled that straddle the two clusters. Each document pair is then fed into a prompt template that is used to ask an LLM whether it belongs to the same category as per the clustering criterion as a yes/no question. This exact "yes/no" prompt can be found in Figure 19 in Appendix C. The same process is then repeated at the two adjacent, finer merges, from $k+2$ to $k+1$ clusters and from $k+3$ to $k+2$ clusters, so that every grid level $k \in \mathcal{K}$ aggregates nine yes/no votes drawn from three neighbouring merges of the hierarchy.

From the pooled nine votes at each grid level, the share of "no" answers is calculated. This fraction represents the proportion of merges at that granularity the LLM considers "unjustified". This share is expected to be high at low k, where merges join unrelated categories, and low at high k, where merges join fragments of the same category.

Finally, a logistic curve of this fraction against $\log k$ is fitted and the final $\hat{k}$ is chosen to be the point where it crosses 50% (which is then rounded and clipped to the nearest k in the grid range). This $\hat{k}$ can therefore be thought of as the point between where the LLM believes the clustering is too granular and where it thinks it is not granular enough, which is by construction the point of optimal granularity.

5. **Sequential:** The Sequential estimator is adapted from the TopicGPT algorithm [Pham et al., 2024]. In this method, 600 random samples from the corpus are streamed into the LLM in batches of 20. At each new batch, the LLM (reasoning disabled) gets as input the new batch, the criterion and the running category list and may add only genuinely new categories required by documents that fit none of the existing ones. After three consecutive batches add nothing to the running list of categories, the process is stopped. Finally, the LLM (reasoning enabled) is prompted to merge duplicate and near-duplicate categories to make sure that the same category found under different possible names in different batches is never counted more than once, which would artificially inflate the number of categories. The final $\hat{k}$ is then chosen as the length of the final list. Both the batchwise prompt and the deduplication prompt can be found in Figure 20 in Appendix C.

6. **Simpson:** The Simpson estimator is an adaptation of a metric from the field of ecology, originally introduced to quantify the concentration of a population across species [Simpson, 1949]. For a label distribution $p_1, \ldots, p_k$, the Simpson index $\lambda = \sum_{i=1}^{k} p_i^2$ is the probability that two randomly drawn documents share a category (where originally it was the probability that two randomly drawn individuals belong to the same species), and for k equally sized categories $p_i = 1/k$ gives $\lambda = 1/k$. To calculate $\hat{k}$ this way, first 120 random sample document pairs are taken from the corpus. For each pair, an LLM is then prompted (see

Figure 19 in Appendix C) to vote on whether they belong to the same or a different cluster, given the criterion. The resulting estimate $\widehat{P}(\text{same})$ is taken as the fraction of "same" answers, and this is then used to yield $\hat{k} = 1/\widehat{P}(\text{same})$. One main bias in this method is that in our case, the equal-cluster-size assumption is clearly wrong. Therefore, the estimator is systematically biased downward, and we include it as a relatively low-cost baseline.

7. **Judge sweep:** The Judge sweep method is a hybrid between classical clustering and an LLM-based method, in which we run a simple k-means clustering algorithm at every value in $\mathcal{K}$, and each partition is scored by the judge of research question 1. The $\hat{k}$ is simply chosen as the one that corresponds to the highest judge score.

8. **Merge convergence:** This is again a hybrid method that combines k-means with LLM reasoning. First, we run an over-segmented k-means model with $k_0 = 200$. We then iteratively merge these micro-clusters until we reach an optimal k. We do this by first summarising each micro-cluster by its three most central documents in embedding space. Then, the following is repeated for multiple rounds. First, we calculate the distance between all centroids of the micro-clusters. Second, we identify the eight closest cluster pairs by centroid distance. Then, we prompt an LLM with the three-document summary of each micro-cluster to determine whether to merge them. If the LLM reasons they should be merged, we merge them. This repeated algorithm stops once a single round results in no more merges. The final amount of clusters is then taken as $\hat{k}$.

3.2.2 Design Rationale

Because the literature offers no established methodologies for k-inference with LLMs, this work tested a variety of methods in an attempt to cover a diverse set of possible approaches. The eight estimator methods were chosen to cover a wide range of distinct possible routes by which an LLM could be used to infer k: enumerating categories and counting them (Codebook, Sequential); naming a number outright (Direct, Probe); aggregating many local same/different judgements into a total estimate (Pairwise, Simpson); and combining the model's judgement with the embedding geometry so that the estimate corresponds to an actual clustering (Judge sweep, Merge convergence). In this sense, many bases are covered, but no fully exhaustive search of all possible LLM methods is performed.

3.2.3 Statistical Protocol

The inferred values $\hat{k}$ are compared to the benchmark values k using summary statistics, a calibration analysis, and a paired test against an LLM-free baseline, as follows. To reiterate: the actual benchmark k values are never seen by the k-inference methods and are used only to statistically evaluate the results.

Per corpus, each method runs at ten different random seeds, and each repetition reruns every stochastic element of the method. These stochastic elements include the document samples shown to the model, the subsample for the Ward hierarchy, any random document pairs, and the k-means initialisations. For the Codebook and Judge sweep methods, the ten repetitions are the ten independent judge runs of the research question 1 judge (independent random discovery

sample, codebook, and reference labelling). All per-corpus results are the median estimate over the ten repetitions, and we keep the individual estimates for the stability analysis.

Six main summary statistics are reported to answer this research question. The first statistic is the median signed error between $\hat{k}$ and k. The benchmark granularities over all 19 chosen benchmarks span more than one order of magnitude. As such, the individual error between a single inferred $\hat{k}$ and benchmark k was chosen as the log-ratio error:

$$\varepsilon = \log_2\left(\hat{k}/k\right),$$

where the sign of the error represents whether the inference overshot the benchmark ($\varepsilon > 0$) or undershot it ($\varepsilon < 0$). Here, the magnitude of ε counts factor-of-two steps. A correctly inferred $\hat{k} = k$ will correspond to $\varepsilon = 0$, one that is double or half the benchmark k has $|\varepsilon| = 1$, and one that is four times too large or too small has $|\varepsilon| = 2$ and so on.

For each corpus, we compute one error from the median estimate over the ten repetitions, giving 19 errors per method. The median signed error for a method is the median of these 19 per-corpus errors. Second, the median absolute error for a given method is the median of the absolute values $|\varepsilon|$ of these 19 per-corpus errors. Third, the fraction of corpora on which the median estimate for a method lands within a factor of 1.5 of the benchmark k is calculated as the fraction of the 19 per-corpus errors for which $|\varepsilon| \leq \log_2 1.5$. The fourth statistic is a stricter version of this count, the fraction of corpora on which the median estimate lands within ten percent of the benchmark k, that is $|\hat{k} - k| \leq 0.1\,k$.

The fifth metric is Lin's concordance correlation coefficient (CCC) between $\log_2 \hat{k}$ and $\log_2 k$ across the 19 corpora, where $\hat{k}$ is each corpus's median estimate over the ten repetitions [Lin, 1989]. The CCC measures how well the pairs of inferred and benchmark granularities agree. In other words, it measures how closely the 19 points $(\log_2 k, \log_2 \hat{k})$ fall on the identity line $\hat{k} = k$. A CCC of 1 means all the inferred $\hat{k}$ are in perfect agreement with benchmark k, while a CCC near 0 means no agreement at all. Between these extremes, the CCC goes down for random scatter and systematic deviation from the identity line $\hat{k} = k$. More exactly, the CCC metric is calculated as follows:

$$\mathrm{CCC} \;=\; \frac{2\,s_{xy}}{s_x^2 + s_y^2 + (\bar{x} - \bar{y})^2},$$

where, for each of the 19 corpora i, $x_i = \log_2 k_i$ is its benchmark granularity and $y_i = \log_2 \hat{k}_i$ its inferred granularity (the median estimate over the ten repetitions). The means $\bar{x}$ and $\bar{y}$, the variances s_x^2 and s_y^2, and the covariance s_{xy} are all calculated across these 19 corpus-level pairs.

The final metric is calculated as the spread over ten seeds. This spread is the range of $\hat{k}$ across the ten repetitions divided by its median, per corpus. A spread near one thus means the estimate typically swings by its own size under resampling.

To analyse the calibration of the inferred granularities against the benchmark values, we regressed $\log_2 \hat{k}$ on $\log_2 k$ across the 19 corpora for each method. The slope of this fit measures the degree to which changes of k across corpora are tracked by the corresponding inferred $\hat{k}$. A perfectly calibrated estimator would simply have a slope of 1, and an estimator that returns the same number regardless of the corpus would have slope 0. Uncertainty on the fitted line was calculated using a bootstrap over the corpora (2,000 resamples of the 19 benchmarks with

replacement), from which a 95% confidence band is built. In each bootstrap resample, the regression is refitted on the redrawn set of these corpora, and the confidence band shows the central 95% of the resulting fitted lines at every point.

We additionally compare all methods against a non-LLM baseline k-inference method. The baseline is chosen to be the silhouette-argmax method (§2.2.3), in which we test the same series of candidate k values $\mathcal{K}$ as before and take the one with the highest silhouette score (§2.3.1) on k-means for that corpus as the inferred k. The baseline is run under the same protocol as the other methods. Each LLM method is then compared to this baseline with a two-sided Wilcoxon signed-rank test over the 19 per-corpus absolute errors $|\varepsilon|$, paired by corpus.

3.3 Research Question 3

To what extent can an LLM, acting as both partitioner and judge, equipped with prior knowledge of the clustering criterion, and inserted into the clustering pipeline, align unsupervised clustering with the partitions humans would draw?

To answer this research question, this work introduces CLEAR (Clustering with LLM Evaluation And Repair), a new LLM text clustering algorithm built for exactly this purpose. CLEAR is a criterion-conditioned clustering pipeline that introduces an LLM at several stages. On the one hand, in line with research question 1, an LLM serves as an active judge to select from a list of candidate base (classical) clustering algorithms (e.g. k-means, GMM, etc.). On the other hand, the LLM acts as a partitioner to enhance the chosen base clustering algorithm through techniques seen before, such as pre-clustering document rewrites, generating category examples and post-clustering boundary repair. CLEAR is an oracle-k algorithm, meaning it requires an input for granularity. This aligns with other state-of-the-art algorithms in the literature. The algorithm was run on all 19 benchmarks over five random seeds and analysed under the statistical protocol below.

3.3.1 The CLEAR Algorithm

CLEAR takes three inputs. The first input is the unlabelled corpus D. The second is the criterion sentence c which again is the exact same one-sentence statement used throughout this thesis (Table 7 in Appendix B). The third is the input granularity k. This granularity k was chosen to be user-controlled rather than inferred following the main conclusion of research question 2. The algorithm then simply outputs a partition of the corpus D into k hard-assigned clusters.

The clustering pipeline, visualised in Figure 4, has three main stages in which LLM calls are made. First, a suite of candidate base clusterings is generated. These include relatively simple cluster assignments, including classical non-LLM methods, as well as slightly LLM-enhanced versions of base algorithms. Second, an LLM judge similar to the codebook judge discussed in §3.1.1 scores each method in the suite on how well it clusters the input corpus D. The base algorithm with the highest judge score is then chosen. The final stage then uses the LLM to refine the partitions as a post-clustering (boundary repair) step. CLEAR is also designed such that the chosen base encoder and LLM are user-controlled inputs.

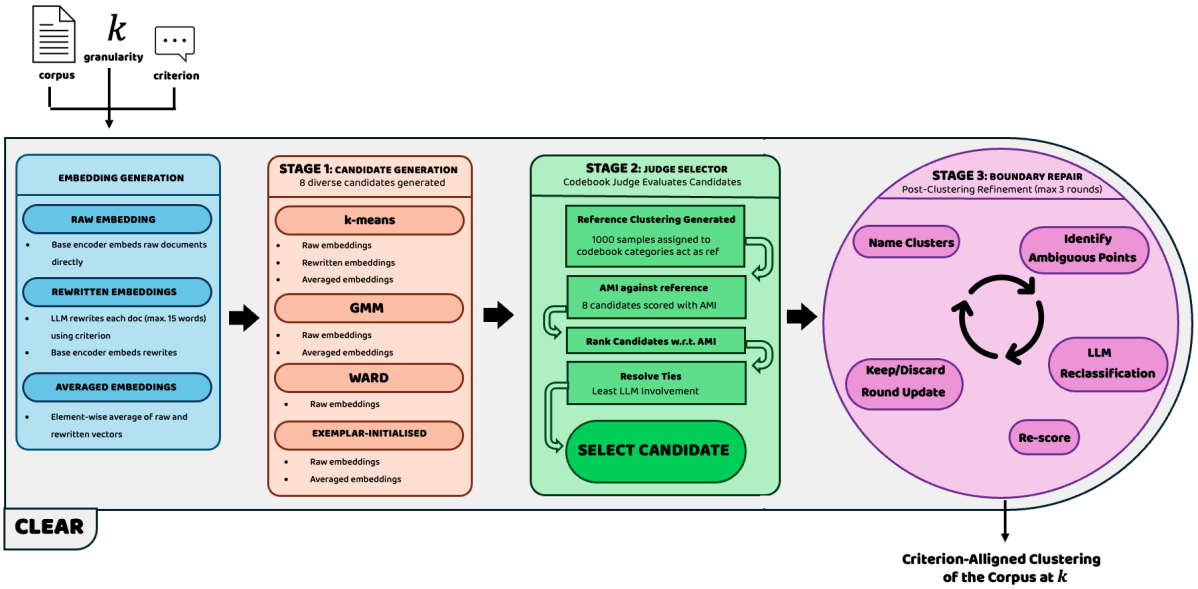


Figure 4: Overview of the three-stage CLEAR pipeline. The inputs in the top-left are the original raw corpus, the choice of granularity, and the clustering criterion. Output on the bottom-right is the criterion-aligned clustering of the corpus at k

Candidate Generation

The candidate suite consists of eight different candidate base-clustering algorithms at the user-specified k. The suite was chosen to have maximal diversity across techniques in three different directions: algorithm family, embedding space, and initialisation. The eight candidates are:

- **k-means (raw embeddings)**: k-means applied directly to the raw corpus embeddings X_r produced by the chosen input encoder.
- **k-means (rewritten embeddings)**: For each document in the corpus, an LLM is first prompted to rewrite it in a phrase of at most 15 words that captures only the criterion instead of lexical noise such as tone, detail or phrasing. These rewritten documents are then encoded as X_w, and k-means is run on these embeddings. The exact prompt can be found in Figure 21 in Appendix C.
- **k-means (averaged embeddings)**: This method combines the raw and rewritten embeddings, where each document's raw and rewritten embedding vectors are averaged element-wise and ℓ_2-normalised to produce the averaged embeddings X_c, on which k-means is then run.
- **exemplar-initialised (raw embeddings)**: Simple k-means on X_r, but the initialisation centroids are generated by an LLM instead of being randomly chosen. The k initial centroids are generated in three steps. First, using the exact same prompt as the codebook judge from research question 1 (Figure 15 in Appendix C), an LLM is used to generate a codebook at the LLM's chosen granularity. Second, for every category in this codebook, the LLM is prompted to provide five short example documents that would clearly belong to it (Figure 22 in Appendix C). These five short examples are encoded, and their embeddings are averaged and ℓ_2-normalised to generate the initialisations. The LLM codebook generally includes

either more or fewer categories than the user-specified k. When the codebook k is higher than the user-specified k, a separate k-means algorithm is run on the initialisations to reduce the number of initialisations to the user-specified k. If the codebook k is lower than the user-specified k, all initial centroids are kept. Each missing centroid is then generated one by one by sampling from the documents with a probability proportional to their squared distance from the nearest centroid chosen so far. This filling procedure therefore produces initialisation centroids in the regions the codebook does not cover. This filling method is very similar to the k-means++ algorithm discussed in §2.2.1.

- **exemplar-initialised (averaged embeddings)**: This method repeats the exact same methodology as exemplar-initialised (raw), but instead starting from the averaged embeddings X_c.
- **GMM (raw embeddings)**: A Gaussian Mixture Model with diagonal covariance is run using the user-specified k on 50-dimensional principal component analysis (PCA) projections of the X_r embeddings, and each document is then hard-assigned to the component with the highest posterior probability.
- **GMM (averaged embeddings)**: The same exact procedure as GMM (raw), but instead using the X_c embeddings.
- **Ward (raw embeddings)**: Agglomerative clustering with Ward's linkage cut at k on the 50-dimensional PCA projection of X_r.

The Judge

For a given corpus, we now have eight ways to divide the corpus into k clusters. At this point, we score each candidate clustering using the same codebook judge described in §3.1.1 and rank them from lowest to highest score with one small refinement. Judge scores inherently carry some amount of random noise. As such, two candidates with judge scores within 0.01 are treated as ties. We resolve these ties by picking the candidate with the least LLM involvement as the winner. The reason we do this is to mitigate any bias the LLM judge may hold towards LLM-generated answers. From least to most LLM involvement, ties are resolved in the order of k-means (raw), GMM (raw), exemplar-initialised (raw), GMM (averaged), exemplar-initialised (averaged), k-means (averaged), k-means (rewritten), and Ward (raw).

Boundary Repair

The chosen candidate clustering of the embedded documents is then refined using an LLM. First, each cluster is named by prompting an LLM once per cluster to choose an appropriate cluster title based on the criterion and the eight documents with embeddings closest to the centroid in cosine distance. The exact prompt can be found in Figure 23 in Appendix C. We generate these cluster names once and cache them.

The cluster refinement then runs for a total of three rounds. In each round, we first select the 200 least confident documents (those points with the smallest difference in distance between their own cluster centroid and the next closest one), excluding documents already selected in previous

rounds. These points represent the documents where geometry is most uncertain about their cluster assignment. Thus, these points represent the documents where geometry is most likely to disagree with the partition a human following the criterion would draw, and they are therefore the ones targeted for reassignment by the LLM. The LLM reclassifies each selected document in batches of 20 using the cached category names. If the LLM does not find a cluster for a document, nothing changes. The exact prompt for this step can be found in Figure 24 in Appendix C.

At the end of each round, the judge rescores the full clustering. If the judge score improved, the round's changes are kept, and the next round starts from these changes. If it did not, the changes are discarded, and the next round starts from the previous clustering. As such, for each run of the CLEAR algorithm, an LLM individually assigns the 600 most "problematic" documents to their cluster.

This algorithm design is summarised in Algorithm 2. Here D is the unlabelled corpus, c the criterion sentence, k the input granularity and f the frozen encoder. B is the LLM-generated codebook, X_r, X_w and X_c are the raw, rewritten and averaged embeddings, and P holds one prototype vector per codebook category. R is the judge's reference labelling, $\mathcal{Y}$ the candidate suite, $J(y)$ the judge score of a candidate y and y^* the best clustering found so far.

Algorithm 2 The CLEAR algorithm

Require: corpus D; criterion sentence c; granularity k; frozen encoder f
Ensure: partition of D into k clusters

$B \leftarrow$ LLM-generated codebook from a 300-document sample of D and c
$X_r, X_w, X_c \leftarrow$ the raw, rewritten and averaged embeddings of D under f
$P \leftarrow$ one embedded prototype per category of B
$R \leftarrow \textsc{BuildReference}(D, c)$ ▷ the judge's reference labelling, Algorithm 1

$\mathcal{Y} \leftarrow$ the eight candidate clusterings of §3.3.1 at granularity k
$y^* \leftarrow$ the candidate with the highest judge score $J(y) = \textsc{Judge}(y, R)$, with ties within $\varepsilon = 0.01$ resolved towards the least LLM involvement

name each cluster of y^* once, from its eight most central documents
for round $= 1, 2, 3$ **do**
    $U \leftarrow$ the 200 not-yet-selected documents closest to a cluster boundary
    $y' \leftarrow y^*$ with every $d \in U$ reassigned by the LLM against the cluster names
    if $J(y') > J(y^*)$ **then**
        $y^* \leftarrow y'$ ▷ keep the round, else discard it
    end if
end for
return y^*

3.3.2 Baselines and Analyses

For all 19 corpora, we run the CLEAR algorithm with five different random seeds that affect the 1,000-document sample behind the judge's reference labelling and the initialisations of all eight candidates. We build the codebook and LLM-generated texts once per corpus, and keep the model temperature at zero throughout. The actual LLM and embedder used are `glm-5`

and `gemini-embedding-001` [Lee et al., 2025b] respectively. For each run on each corpus, we report NMI and ACC against the benchmark clustering. To further investigate how CLEAR aligned unsupervised clustering with the partitions humans would draw, we conducted a few more specialised analyses.

First, we compare against a geometry-only baseline. Plain k-means is run at the same user-specified k on the same raw document embeddings that CLEAR receives using the same `gemini-embedding-001` encoder, and its NMI and ACC against the benchmark clustering are compared to the CLEAR results over the same five random seeds. As such, the geometry-only baseline at each seed is in fact identical to CLEAR's own k-means (raw) candidate at that seed. The per-benchmark mean deltas between the two are tested with a two-sided Wilcoxon signed-rank test, mirroring the statistical protocol of research question 2.

Second, we analyse the judge's performance within the clustering pipeline compared with other baseline methods. To assess how well the judge performed, for every run of CLEAR (19 benchmarks over five random seed runs, for a total of 95 runs), we score all eight candidate clusterings using NMI and ACC against their corresponding benchmark. Now that we know each candidate's true NMI and ACC, we can compare different candidate-picking methods. The methods include random picking, picking the candidate that maximises silhouette score, the judge's actual pick, and picking by maximum ACC and NMI against the benchmark (an upper bound no selector could exceed).

We then summarise each candidate selection strategy by the mean NMI and ACC of the candidate it selects, against the benchmark, averaged over all 95 runs. This lets us compare the judge to other baseline selection methods and, most importantly, to the oracle, showing how close it comes to picking the absolute best candidate. We also record the exact candidates that got chosen and further analyse these data to investigate which parts of the suite the judge chooses most and how its selections vary across corpora.

In the final analysis, we investigate the improvement the post-clustering refinement step makes on the overall algorithm. For every run over all corpora, we evaluate the judge-picked clustering against the benchmark and compare these scores with the full CLEAR algorithm's scores to assess what the final repair rounds contributed.

3.4 Research Question 4

Can a single, general clustering algorithm, equipped with a stated criterion and a label-free LLM judge, match or exceed published LLM-in-the-loop methods across the published benchmark suite when the encoder and language model are held equal?

In this research question, CLEAR's performance is compared to the SOTA results mentioned in Section 2.5.3 to evaluate where this work's methods lie in the current literature. To ensure a fair comparison, the evaluation set contains LLM-in-the-loop clustering algorithms only. Methods that rely on training or fine-tuning the encoder itself, such as those tracked by the MTEB benchmark (§2.1.4), are out of the scope of this thesis. Because no part of the CLEAR algorithm implements fine-tuning, these methods are omitted from the analysis.

3.4.1 Matched and Unmatched Configurations

We compare CLEAR directly to SOTA results in two settings: matched and unmatched. In the unmatched configuration, we run CLEAR using the same frozen encoder and LLM as in research question 3 (`gemini-embedding-001` and `glm-5`) and compare its results on all 19 corpora at face value against published results. This comparison is confounded, as a win against a SOTA method on a benchmark might come from the algorithm or simply from a stronger encoder or LLM than the published method had.

To address this, we also run a matched study in which we match the CLEAR algorithm's encoder and LLM to those of the SOTA algorithm we compare against (see Table 8 in Appendix B). Where the NMI and ACC records for a benchmark are held by different methods, the NMI holder's configuration is matched. The algorithm itself is frozen across both settings. One further design choice applies to the judge stage. The matched LLM produces all generation artifacts (the rewrites, exemplars and codebook), while the judge itself always runs on `gemini-2.5-flash` which is the one validated in research question 1. This keeps the measuring instrument fixed across the 19 matched runs instead of changing it with every record holder's model, at the cost of one component that is not matched. Six matched components are substitutes where the original model can no longer be run or has been superseded, as marked in Table 8; for example, the SPILL algorithm's encoder is no longer publicly served and is proxied by `instructor-large`. We also run a simple ablation study in which CLEAR is matched by the LLM but not by the encoder, and conversely, where the encoder is matched and the LLM is left unmatched.

3.5 Implementation

All LLM calls in this work are routed through a single shared client. A failed call (a rate limit, a timeout or a gateway error) is retried up to three times with exponential backoff, in which the pause before each retry doubles in length. As such, a long unattended run does not stop when a single call fails. The model temperature is kept at zero throughout, and since LLM replies still vary between runs in practice, every reported statistic is an aggregate over the ten replicates or five seeds of its corresponding protocol.

The context window of the chosen LLM limits the assignment stage. A fine-grained codebook can exceed 100 categories with definitions, and the per-document assignment of §3.1.1 re-transmits this codebook with each of the 1,000 assignment calls. The four batched judges and the CLEAR runs therefore classify documents in batches of 20 per call, so the codebook is sent once per 20 documents rather than once per document. Each call is sized against the model's context window. When the full request does not fit, the per-document clip is tightened first, from 1,000 down to 150 characters, the category definitions are dropped second so that only the category names are sent, and the batch size finally becomes whatever fits. This allows the same protocol to run unchanged on an 8k-token context window (`gemma-2-9b-it`) and on a 128k one (`gpt-4o`).

Every prompt asks the model to reply with JSON only, but replies often deviate from this format in practice. Some arrive wrapped in extra characters that mark the reply as a code block, some contain trailing commas that break strict parsing, some return a bare list of numbers instead of the requested objects, and some are cut off before the closing bracket. The parsing layer accepts

each of these shapes and repairs what it can. A codebook reply is parsed in three attempts. It is first read as strict JSON, then read again after repairing trailing commas, and if both attempts fail, every well-formed category entry is recovered individually, so that one malformed definition does not cost the whole codebook. When a batched classification reply cannot be parsed, the same batch is sent once more, and if the second reply also fails, each of the batch's documents is classified with its own individual call instead. A malformed reply therefore costs at most a few extra calls for that one batch, and the run itself always continues.

Finally, every paid artifact (the codebooks, rewrites, exemplars, reference labellings and embeddings) is cached on disk, given a key that matches the corpus and, where relevant, the configuration and seed. Reruns therefore cost nothing further, and the resulting per-run costs are quantified in §5.2.

Chapter 4

Results and Analysis

All NMI and ACC values in this chapter and the remainder of the thesis are reported as percentages, that is, the metrics of §2.3.2 multiplied by 100.

4.1 Research Question 1

Table 1 summarises the main judge validity results for the five judge LLMs. Each judge's median correlation with the benchmark ranking is reported here, along with the number of benchmarks on which the correlation is significant, and the median partial correlations under the single and joint geometry controls, each with its run-to-run standard deviation. Figure 5 visualises the same comparison. Here, the raw and the jointly geometry-controlled medians are displayed for each judge as a pair of bars, with all 19 per-benchmark correlations, each a mean over ten runs, overlaid as dots. Finally, Figure 25 shows the full per-benchmark results behind every row, with three panels per judge, one for the non-controlled setting, one for the dual-controlled (silhouette and CH) setting, and one for the gap between the two.

Throughout this analysis, a correlation is defined as negligible below 0.10, weak from 0.10 to 0.38, moderate from 0.38 to 0.68, strong from 0.68 to 0.89, and very strong above 0.89.

4.1.1 Overall Validity Across Judge LLMs

All five judges correlate strongly or very strongly with the benchmark ranking, and all remain strongly correlated when the silhouette score and the CH index are controlled for. The five

Judge LLM	Median ρ	Significant	Median ρ | Caliński–Harabasz	Median ρ | silhouette	Median ρ | both
`gemini-2.5-flash`	$+0.88 \pm 0.02$	19/19	$+0.85 \pm 0.02$	$+0.76 \pm 0.04$	$+0.82 \pm 0.02$
`glm-5`	$+0.93 \pm 0.01$	19/19	$+0.90 \pm 0.01$	$+0.86 \pm 0.01$	$+0.87 \pm 0.01$
`gpt-4o`	$+0.90 \pm 0.02$	19/19	$+0.86 \pm 0.02$	$+0.84 \pm 0.02$	$+0.84 \pm 0.02$
`gpt-4o-mini`	$+0.90 \pm 0.01$	18/19	$+0.81 \pm 0.03$	$+0.81 \pm 0.03$	$+0.80 \pm 0.03$
`gpt-3.5-turbo`	$+0.86 \pm 0.04$	18/19	$+0.80 \pm 0.02$	$+0.72 \pm 0.04$	$+0.76 \pm 0.02$

Table 1: The codebook judge results across judge LLMs on the generic-INSTRUCTOR testbed, with all statistics as defined in the statistical protocol. The last three columns show the median partial correlations controlling for the named geometric metric singly and jointly. The significant column shows the number of benchmarks whose correlation is significant in a majority of the ten runs.

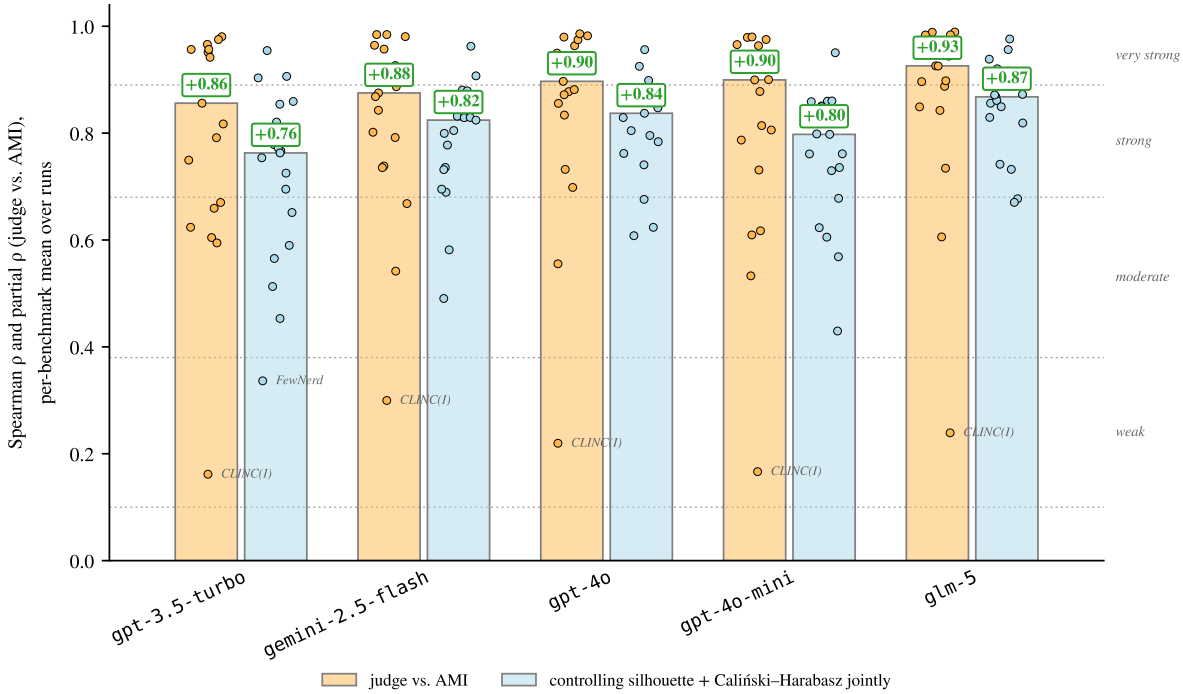


Figure 5: The judge–AMI correlation across the five judge LLMs on the generic-INSTRUCTOR test bed of clusterings. For each judge, the orange bar shows the raw correlation and the light blue bar the partial correlation controlling for the silhouette score and the Caliński–Harabasz index jointly. Each dot is one benchmark's mean correlation over the ten runs, and each bar marks the median of its 19 dots. Dots falling in the weak band are labelled with their benchmark.

medians in Table 1 lie between +0.86 (`gpt-3.5-turbo`) and +0.93 (`glm-5`), and the run-to-run standard deviations are between ±0.01 and ±0.04. Three of the five judges are significant on all 19 benchmarks (`gemini-2.5-flash`, `gpt-4o`, and `glm-5`), while the other two are significant on 18 of the 19.

The joint geometry control answers the second half of the research question. After controlling for all that the silhouette score and the CH index can explain together (§3.1.4), the five medians drop only by 0.06 to 0.10, to between +0.76 and +0.87, so every judge's median remains above the +0.68 strong cutoff.

Figure 5 also visualises the agreement between the judge algorithm's five LLM variants. Their 19 per-benchmark correlations largely overlap, with CLINC(I) as the only benchmark below +0.50 for any judge, and controlling for geometry lowers the correlation by a similarly small amount for every judge. This is clearly visible as the uniform gap between each bar pair.

4.1.2 Per-Benchmark Patterns

Looking at the orange judge-AMI correlation (uncontrolled) data in Figure 25 in Appendix D, it is apparent that the general ordering of correlations of benchmarks is largely the same across the LLMs used. The highest rankings are consistently shared by the topic corpora and the coarser-grained corpora. All six topic corpora (Table 6 in Appendix A) score +0.62 or higher from every judge, and all three domain corpora (MTOP(D), CLINC(D), and MASSIVE(D)) score +0.86 or higher. The middle rankings are occupied by the corpora labelled by entity type, relation, and event type, which score between +0.60 and +0.95.

The intent corpora sit at both extremes. SNIPS, M-CID, and MTOP(I) score +0.94 or higher from every judge and sit at or near the top of every judge's ranking, while MASSIVE(I), Banking77, and CLINC(I) consistently rank at the bottom, with CLINC(I) between +0.16 and +0.30. CLINC(I) is also the only benchmark for which any judge fails to show statistical significance.

Closer inspection of the domain/intent pair benchmarks (benchmarks that have the same corpus clustered by both domain and intent at different granularities) shows another disparity. MASSIVE appears twice with the same 2,974 documents, yet every judge scores its domain labelling at +0.86 to +0.93 against only +0.59 to +0.74 for its intent labelling. CLINC shows an even greater disparity (+0.89 to +0.94 against +0.16 to +0.30).

The light blue panels show that controlling for both geometric metrics together does not break the correlation on any benchmark for any judge. Almost all controlled correlations remain in the strong or very strong region, with between one and six benchmarks per judge falling to the moderate region and only a single value below it.

The red gap panels plot the difference between each benchmark's raw and geometry-controlled correlation. A positive gap means geometry explains part of the judge's agreement on that benchmark, so removing it lowers the correlation. In this case, the geometry artificially inflated the correlation of the judge with benchmark rankings. On the other hand, a negative gap means the control raises the correlation and the judge and the benchmark ranking agree better once the geometric part of both is removed. As such, for negative gaps the geometry can be thought of as obscuring the correlation of the judge with benchmark rankings.

The distribution of these gaps over the benchmarks is largely the same for all five LLMs with most gaps within about ±0.15 and the extremes on the same corpora in every row. The largest positive gaps are for Tweet89, between +0.18 and +0.36 for all five judges, and MTOP(D), between +0.22 and +0.43 for four of the five with `glm-5` the exception at +0.07. The largest negative gaps are shown on the weakest intent corpora. For example, CLINC(I) lies between −0.39 and −0.44 for every judge and Banking77 between −0.20 and −0.29, so on these two corpora the geometric metrics do not explain the judge's agreement, and the correlation is higher after controlling for the silhouette score and the Caliński–Harabasz index.

4.2 Research Question 2

Table 2 shows the summary statistics explained in research question 2's statistical protocol (§3.2.3) for all eight discussed k-inference methods plus the non-LLM silhouette-argmax baseline. The first two columns count benchmarks where the median inferred k (over ten runs) falls within a factor of 1.5 and within ten percent of the benchmark k, respectively. The last three columns are the median absolute log-ratio error, CCC and the spread over the ten runs as discussed in §3.2.3. Higher is better for the two counts and CCC, while lower is better for the other two. Figure 6 shows the corresponding calibration analysis where each estimator's inferred $\hat{k}$ is plotted against the benchmark k over the 19 corpora together with the fitted line.

Method	Within-1.5× ↑	Within ±10% ↑	Med. $\lvert\varepsilon\rvert$ ↓	CCC ↑	Spread ↓
Codebook	7/19	2/19	0.81	0.54	1.09
Direct	**9/19**	2/19	**0.61**	0.57	1.09
Probe	6/19	2/19	1.61	0.05	1.04
Pairwise	8/19	3/19	1.10	**0.62**	0.57
Sequential	5/19	2/19	0.84	0.48	1.14
Simpson	5/19	**4/19**	0.83	0.42	1.03
Judge sweep	6/19	1/19	0.94	0.54	1.09
Merge convergence	1/19	0/19	1.73	0.13	0.17
Silhouette-argmax (no LLM)	9/19	4/19	1.15	0.19	1.06

Table 2: Granularity estimation on the 19-benchmark suite with `gemini-2.5-flash` as the LLM and generic-instruction `instructor-large` embeddings, with all statistics as defined in the statistical protocol.

4.2.1 Estimation Accuracy

Direct is the most accurate estimator. Its estimates lie within 1.5× of the benchmark k on 9 of 19 corpora with a median absolute error of 0.61, which corresponds to a typical miss of about a factor of 1.5 ($2^{0.61} \approx 1.5$). The Pairwise-inferred granularities have the highest concordance (CCC 0.62), and six of the eight methods reach CCC between 0.42 and 0.62. Probe is the clear outlier with a median absolute error 1.61 and CCC of 0.05. Most importantly, no estimator (including the non-LLM baseline) places more than 4 of 19 inferred corpora k within ±10% of the benchmark k.

None of the eight LLM-based methods outperform the baseline method in terms of the number of corpora within 1.5× or ±10% of the benchmark. Under the paired Wilcoxon test of §3.2.3, no

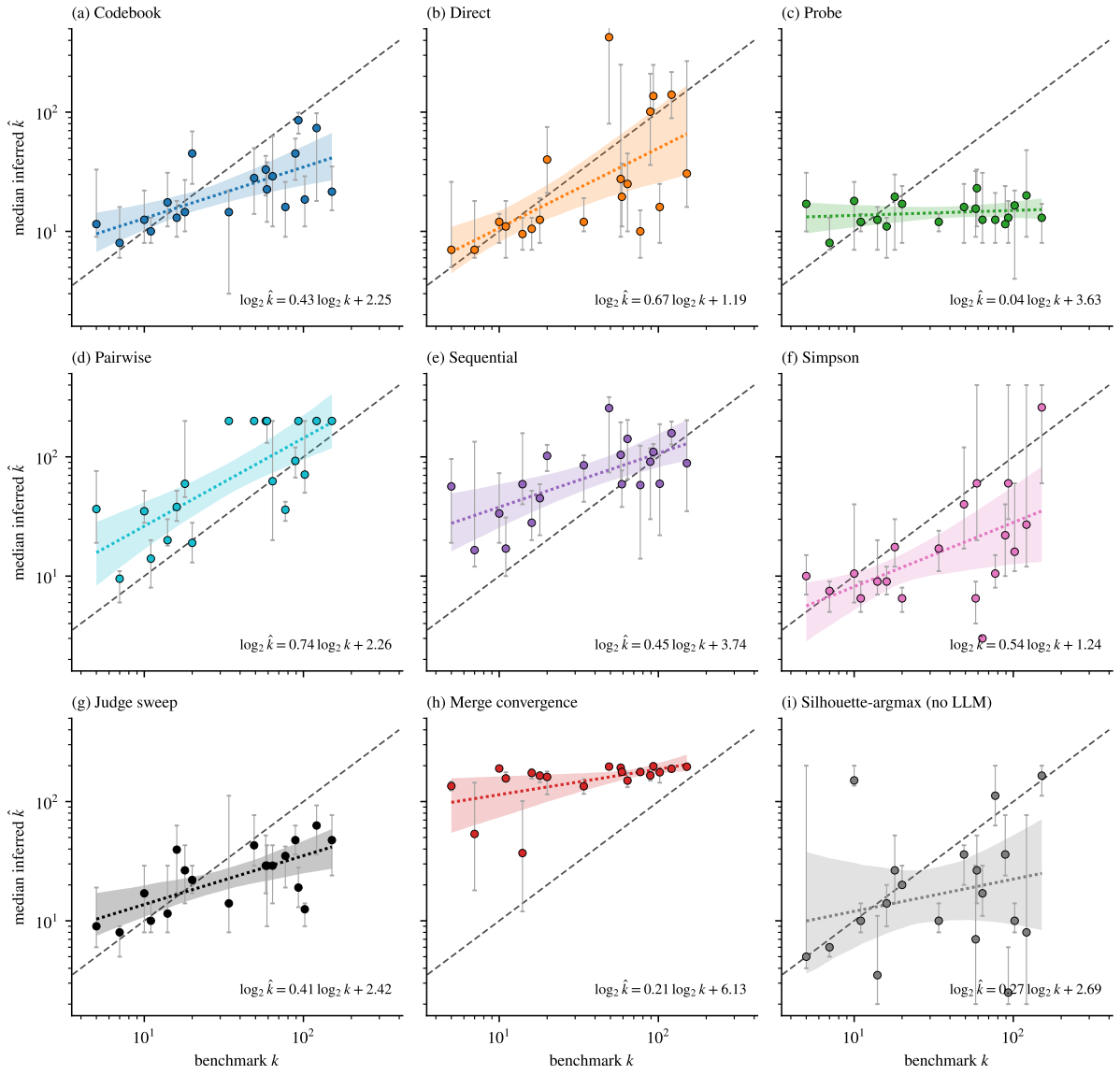


Figure 6: Calibration of the inferred granularity $\hat{k}$ against the benchmark k for the eight estimation methods, panels (a) to (h), and the LLM-free silhouette-argmax baseline, panel (i). Both axes are logarithmic, and each dot is the median $\hat{k}$ of one corpus over the 10 repetitions. Grey whiskers go over the minimum and maximum of all repetitions. The dashed grey line marks perfect calibration ($\hat{k} = k$). The coloured dotted lines show the least-squares fit of $\log_2 \hat{k}$ on $\log_2 k$, and the shaded band gives its 95% confidence interval from 2,000 bootstrap resamples.

method's per-corpus errors differ significantly from the baseline's in either direction (W between 44 and 94, all $p \geq 0.12$, Table 9 in Appendix D). The baseline silhouette-argmax method matches the best LLM method (Direct) on the within-1.5× rate of 9/19, but with a median absolute error of 1.15, almost double that of Direct (0.61). Its CCC of 0.19 shows that its estimates barely co-vary with the benchmark granularity. As such, its hit rate comes from landing in the middle of the granularity range rather than from tracking k. Conversely, the LLM methods' estimates move with the benchmark k even where their absolute accuracy is similar. One striking exception to this is the Probe method which, similarly to the baseline, has a hit rate similar to other LLM methods (within-1.5× = 6/19) by simply predicting a relatively constant k.

The Pairwise, Sequential, and Merge convergence methods systematically overshoot the inferred granularities, while the Codebook, Direct, Probe, Judge sweep and Simpson methods all undershoot, Probe most strongly. Six methods have spreads near 1 (1.03 to 1.14) so repeated runs disagree by about as much as the estimate itself. Pairwise is significantly steadier (spread = 0.57) and Merge convergence is by far the most stable (spread = 0.17) while also being the least accurate (median absolute error 1.73 and within-±10% rate of 0/19).

4.2.2 Calibration

Looking at Figure 6, the first apparent point is that every fitted slope lies below 1. The grey dashed line represents perfect granularity inference, $\hat{k} = k$. Every estimation method therefore pulls its estimates towards the middle of the k range. On corpora with a low k, the inferred $\hat{k}$ is usually too high, and on corpora with a high k, the inferred $\hat{k}$ is usually too low. The methods differ mainly in how strongly the estimates are pulled towards this middle and in how much the estimates scatter around the fitted line (as is apparent from the different widths of the shaded confidence bands).

The replications with `gpt-4o-mini` and `gpt-3.5-turbo` (Tables 10 and 11 in Appendix D) reproduce the headline result on both beds: at most 3 of 19 corpora land within ±10% of the benchmark k, and no method significantly beats the silhouette baseline. The steepest calibration slope falls from 0.74 to 0.61 and 0.39 across the three models.

4.3 Research Question 3

All results in this section use `gemini-embedding-001` embeddings and `glm-5` as the LLM inside the CLEAR algorithm. Figure 7 reports the clustering quality (ACC and NMI versus benchmark clustering) per benchmark. Figure 8 shows the improvement over a simple k-means baseline on the same embeddings. We ran CLEAR over all 19 benchmarks with five random seeds, for a total of 95 runs. Table 3 shows which candidates the judge selected over the 95 runs. Figure 9 isolates the final repair step, measuring what it adds to the judge-selected clustering.

4.3.1 Clustering Quality

Over the 19 benchmarks, CLEAR reaches a mean NMI of 79.0 and a mean ACC of 68.6. Ten benchmarks lie at 80 NMI or higher, topped by SNIPS (96.0 NMI, 98.2 ACC), MTOP(D) and BBC News, and the lowest NMI values belong to the three mention corpora FewEvent, FewRel

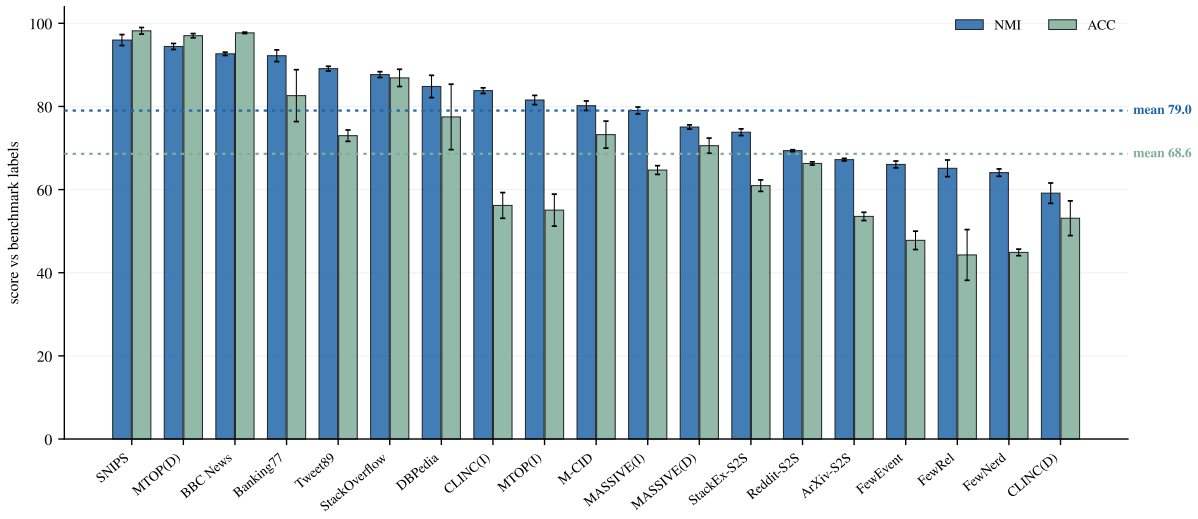


Figure 7: Per-benchmark clustering quality of CLEAR. Bars show the NMI (blue) and ACC (green) means over the five runs. Whiskers show one standard deviation, and the dotted lines show the two means over the 19 benchmarks. Exact values can be found in Table 12 in Appendix D.

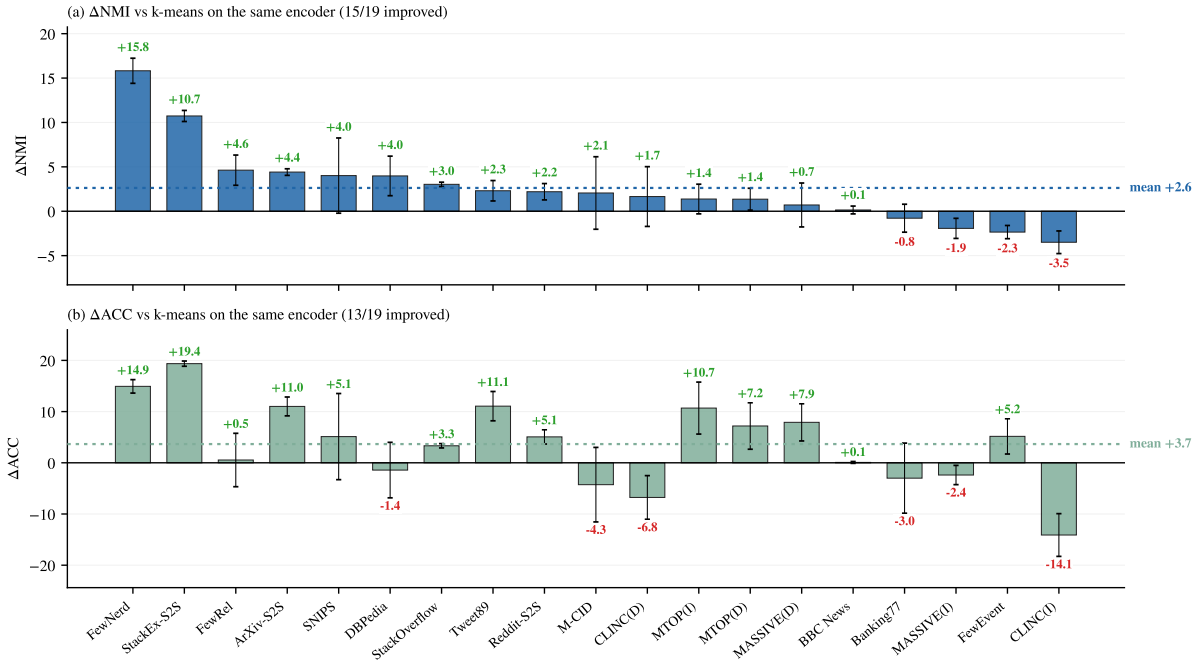


Figure 8: Improvement of CLEAR over k-means on the same encoder, seed-paired per run, for (a) NMI and (b) ACC (`gemini-embedding-001` embeddings, `glm-5` as the LLM). Bars are mean deltas over the five runs. Whiskers show one standard deviation ovver the 5 runs, and the dotted lines show the mean over the 19 benchmarks.

Candidate	Clustering family	Selections	Share
k-means (raw)	k-means	15	16%
k-means (averaged)	k-means	3	3%
k-means (rewritten)	k-means	1	1%
exemplar-initialised (raw)	k-means	37	39%
exemplar-initialised (averaged)	k-means	30	32%
GMM (raw)	GMM	1	1%
GMM (averaged)	GMM	4	4%
Ward (raw)	agglomerative	4	4%

Table 3: How often the judge selected each of the eight candidates across the 95 runs (19 benchmarks, five seeds) of the `gemini-embedding-001`, `glm-5` configuration.

and FewNerd (64 to 66) and to CLINC(D) (59.1). The runs are relatively stable over the five seeds with most standard deviations below 3 points. NMI values are generally higher than ACC. Since ACC matches clusters to benchmark classes one-to-one, a clustering that splits a benchmark class loses ACC even when NMI shows the split preserves most of the label information.

4.3.2 Improvement over the Geometric Baseline

CLEAR improves on k-means on 15 of 19 benchmarks in NMI (mean +2.6) and 13 of 19 in ACC (mean +3.7). A Wilcoxon signed-rank test on the 19 per-benchmark mean deltas confirms the NMI uplift is statistically significant ($W = 34$, $p = 0.012$), while the ACC uplift falls just short of significance ($W = 47$, $p = 0.055$). The largest gains (FewNerd at +15.8 NMI and +14.9 ACC and StackEx-S2S at +10.7 NMI and +19.4 ACC) are consistent across seeds and metrics, both with whiskers of about two points or less. The losses are small on NMI, at worst −3.5 on CLINC(I), but CLINC(I) also has the one large ACC loss at −14.1. The weakest benchmark of the uplift comparison is the finest intent corpus.

4.3.3 Selected Candidates

Table 3 shows that the judge's selections concentrate on the two exemplar-initialised candidates, which together win 67 of the 95 runs (71%). These are the two candidates whose starting centres come from the LLM-generated codebook. Plain k-means on the raw embeddings is the only other frequent pick at 15 runs (16%). Ward, averaged and rewritten k-means win eight runs combined and the two Gaussian mixtures five, so no candidate remained unpicked. Overall, k-means is by far the most selected family of candidates, with approximately 90% of total picks.

4.3.4 The Contribution of the Repair Step

Figure 9 shows, per run, the final score minus the score of the judge-selected candidate before repair. Over the 95 runs, repair adds +0.66 NMI and +0.78 ACC on average. It improves NMI in 60 runs, worsens it in 9, and leaves 26 untouched. ACC improvement is similar, with improvement on 58 runs and worsening on 11 runs. The largest per-benchmark gains are StackOverflow (+3.03 NMI, +3.32 ACC), DBPedia (+1.87 NMI) and StackEx-S2S (+2.79 ACC). MTOP(D) is the only corpus where repair consistently hurts (−1.87 NMI, −1.20 ACC).

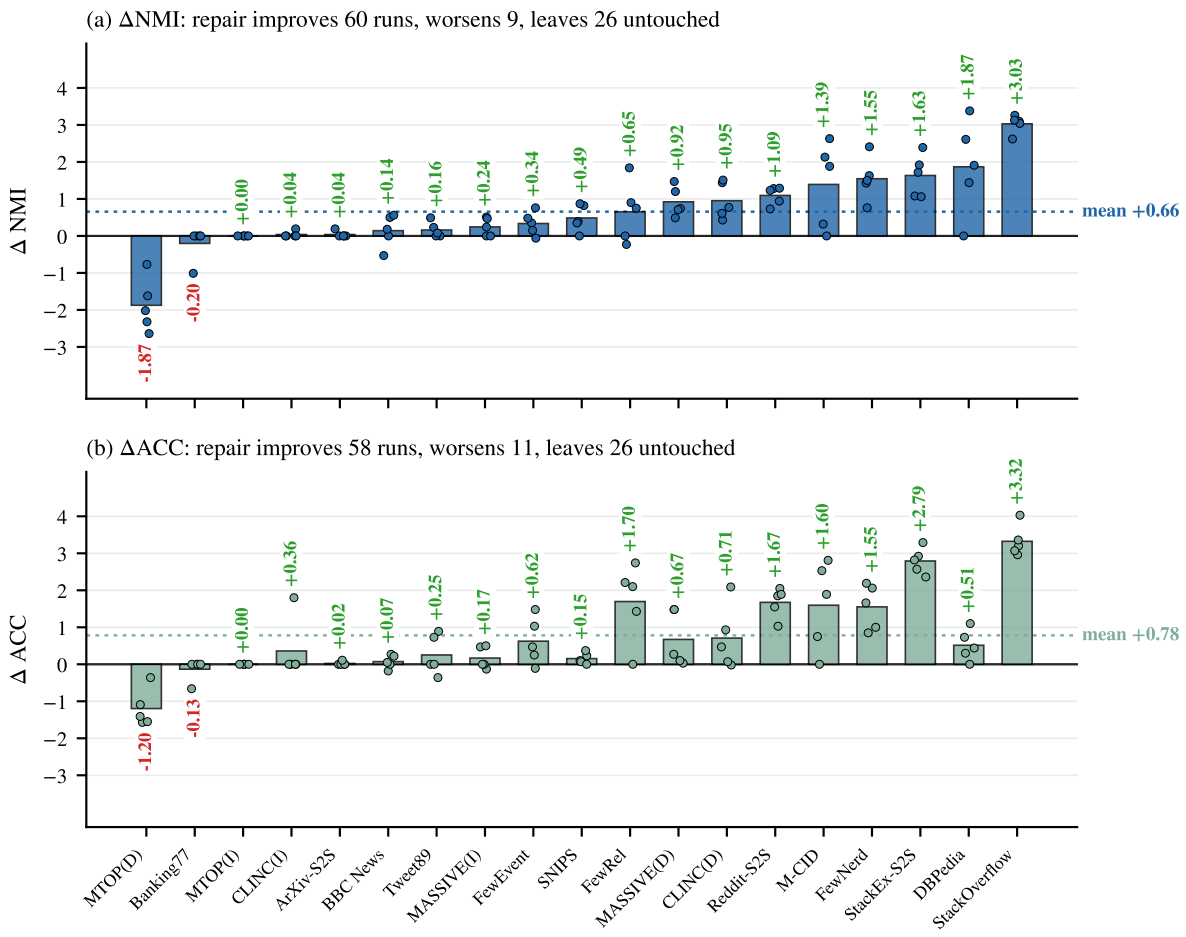


Figure 9: The contribution of the repair step over the 95 runs, for (a) NMI and (b) ACC. Each bar is a benchmark's mean difference between the final clustering and the judge-selected candidate scored before repair. The dots are the five individual runs.

4.4 Research Question 4

Figure 10 compares the CLEAR algorithm to the best published result on every benchmark in the unmatched and matched configurations of §3.4.1. In the unmatched configuration, CLEAR runs in the exact setup of research question 3 and is compared against each benchmark's best published result, regardless of the encoder and language model used to produce that result. In the matched configuration, CLEAR instead uses, for each benchmark, the encoder and LLM of the method that produced the SOTA NMI result, so that the only difference between the two sides is the actual algorithm, allowing for a fairer comparison.

Figure 11 extends this comparison into an additional LLM-matched configuration where only the same LLM is used as the SOTA method but the encoder is `gemini-embedding-001`, and an encoder-matched configuration where the opposite happens. Results are summarised as mean differences from the best published results across the 19 benchmarks for each configuration.

Finally, Figure 12 isolates CLEAR's judge mechanism. It compares the quality of the judge's pick against picking at random, always keeping the k-means candidate, picking by silhouette score, and an oracle that always picks the best candidate. Table 8 in Appendix B lists the encoder and LLM used by each SOTA method and these are exactly the configurations CLEAR used in

the matched comparison. The footnotes explain where the original model is no longer available and a substitute is used.

4.4.1 Against the Published State of the Art

In the unmatched configuration (panels (a) and (b) of Figure 10), CLEAR is ahead of the best published result on 14 of 19 benchmarks in NMI and 11 of 19 in ACC, with a mean difference of +3.0 NMI and +0.1 ACC. The largest margins are BBC News at +17.4 NMI, ArXiv-S2S at +17.8 ACC and StackEx-S2S at +13.2 ACC, where each result is against a different published method. The losses are largest on the fine-grained intent and mention corpora, particularly CLINC(I) at −11.9 NMI and −32.1 ACC, followed by FewNerd at −14.4 ACC.

In the matched configuration (panels (c) and (d)) the wins drop to 8 of 19 in NMI and 9 of 19 in ACC and the mean differences are negative at −1.2 NMI and −4.1 ACC. BBC News does stay at +14.8 NMI and StackEx-S2S at +9.5 ACC, so on these corpora the algorithm itself carries the advantage. The losses on the fine-grained corpora widen instead, with CLINC(I) at −12.3 NMI and −26.5 ACC, FewNerd at −20.0 ACC and MTOP(I) at −15.0 ACC.

An important caveat is that some matched configurations required substitutions because of the unavailability of encoders or LLMs (mentioned as footnotes in Table 8) and therefore, possibly more capable configurations were used than the exact matched configuration on some benchmarks. On the 11 benchmarks whose matched configuration required no substitute, CLEAR wins 5 of 11 in NMI and 4 of 11 in ACC. This is a rate comparable to the suite-wide 8 of 19 and 9 of 19.

4.4.2 Configuration Ablation Study

Figure 11 shows a sharper picture of how configuration influences the mean difference in NMI and ACC with SOTA results. On NMI, the mean difference to the SOTA results falls from +3.0 with both ingredients unmatched to +1.8 when only the LLM is matched, +0.8 when only the encoder is matched, and −1.2 when both matched. On NMI, CLEAR therefore stays ahead of the SOTA on average under either single swap, and only drops below them when both encoder and LLM are matched together. On ACC the same ordering holds at a lower level, from +0.1 for both unmatched through −0.6 for only LLM matched and −2.3 for only encoder matched to −4.1 for both matched. From panels (a) and (b), it is evident that only matching the encoder on average reduces performance more than only matching the LLM.

4.4.3 The Judge as Selector

Figure 12 shows that the mean NMI of the selected candidate rises monotonically from a random pick (75.6), to always taking the k-means candidate (76.4) and the silhouette-argmax choice (77.6), to the judge (78.4), with the oracle ceiling at 79.7. The same monotonic increase is found for ACC. The judge selects candidates that are on average 0.8 NMI better than the silhouette-argmax choice. On ACC the judge's advantage over silhouette goes down to 0.2. Against a random pick, the judge's increase is 2.8 NMI and 3.3 ACC.

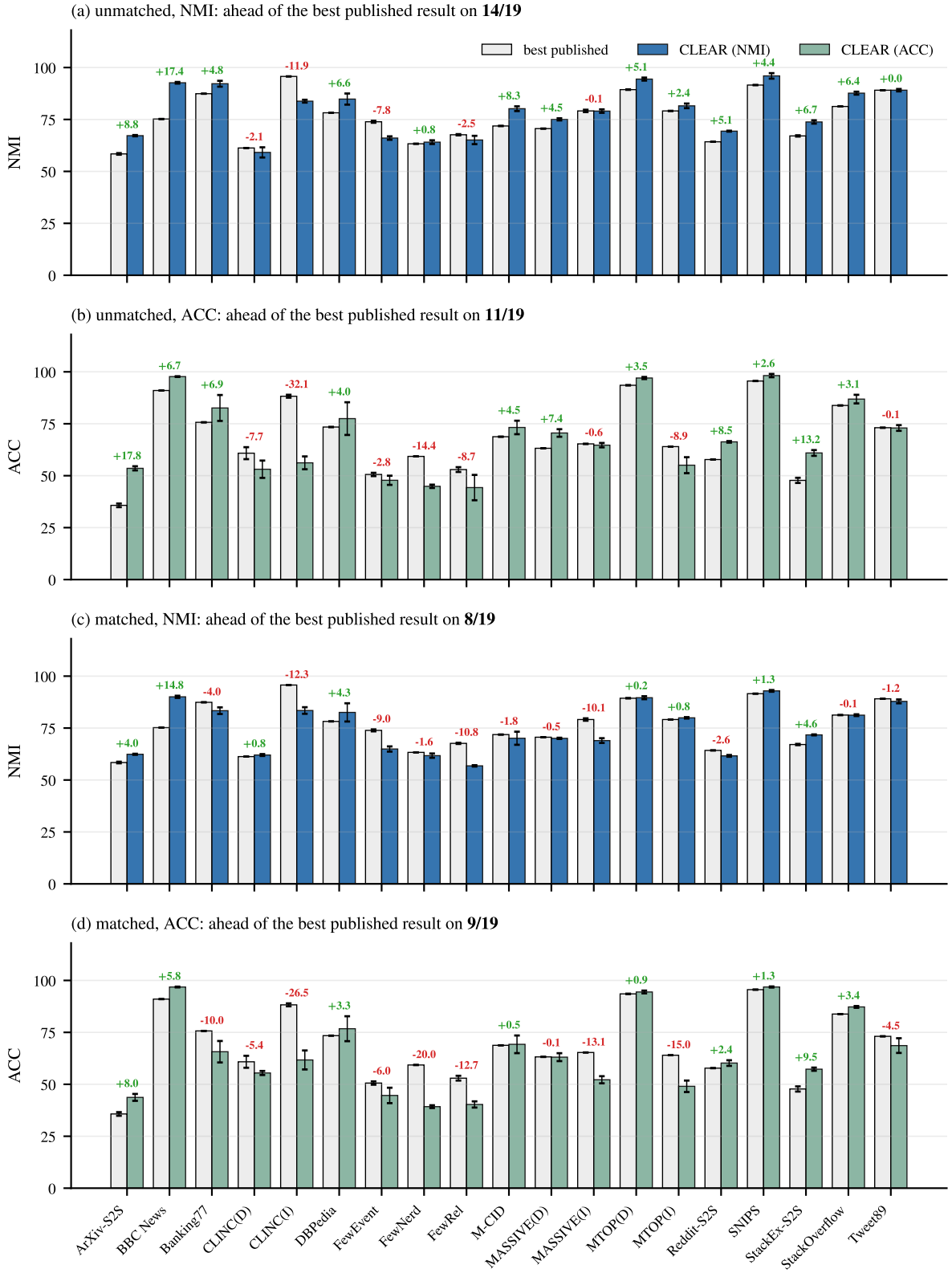


Figure 10: The CLEAR algorithm's results on the benchmark suite against the best published result per benchmark, in the unmatched configuration (`gemini-embedding-001`, `glm-5`) in panels (a) and (b) and in the matched configuration in panels (c) and (d). The signed value above each pair gives the difference, which is positive (green) where CLEAR is ahead and negative (red) where it is behind.

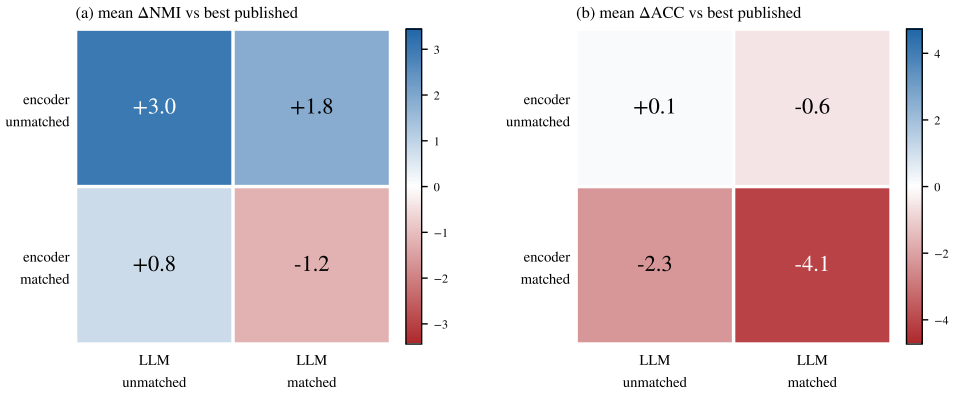


Figure 11: The matched/unmatched ablations as mean difference to the best published result over the 19 benchmarks, for (a) NMI and (b) ACC.

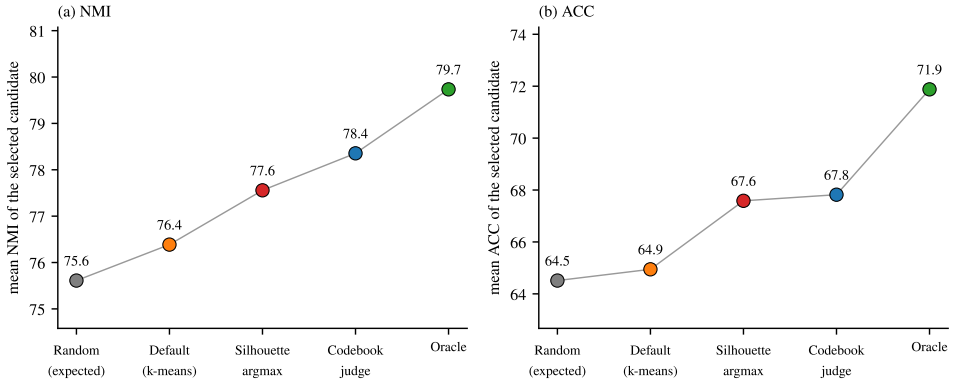


Figure 12: Comparison of selection strategies over the 95 candidate pools (19 benchmarks, five seeds), for (a) NMI and (b) ACC. Each dot is the mean score of the candidate that the strategy selects, as discussed in §4.4.3

Chapter 5

Discussion

This chapter uses the results of the previous chapter to answer the overall question of this work, whether and in what capacity the natural language reasoning of an LLM, given a one-sentence clustering criterion, can be used to evaluate, size and steer the clustering of an unlabelled corpus. The four research questions are answered in order, followed by the anomalies in the results, the algorithms' place in the literature and the limitations that motivate the future work.

5.1 Interpretation of Findings

5.1.1 Research Question 1

The main result of research question 1 is that the label-free scores produced by the codebook judge introduced in Section 3.1.1 correlate with agreement against the benchmark labels significantly on at least 18 of the 19 corpora for every one of the five judge LLMs, with median correlations in the strong to very strong range, and that this correlation largely survives when the silhouette score and Caliński–Harabasz index are controlled for. Three main aspects of the judge's algorithmic design likely explain why this is the case.

First, the judge never scores a clustering directly. The obvious alternative would have been to show the LLM a candidate clustering (for example, a sample of documents per cluster) and ask it to rate the clustering's quality on a numeric scale. That is an open-ended rating task, and §2.6.2 documents that LLM judges on such tasks show position bias (rating what is shown first more favourably), verbosity bias (favouring clusters with longer or more numerous documents) and a preference for LLM-generated over human-written text.

Instead, the judge builds a reference labelling R and calculates the AMI between R and the candidate. This turns the rating task into two tasks: listing the categories present in a sample of documents, and classifying a single document into a fixed list. None of these tasks prompts the LLM to compare candidates, so position bias has nothing to act on. Neither task asks it to rate quality either, so verbosity bias has nothing to reward. The final comparison to the reference labelling is a deterministic, chance-corrected metric that no model biases can enter at that stage.

Second, the judge is designed specifically to measure semantic agreement between cluster members rather than geometry, since AMI depends only on the contingency between two labellings and never sees an embedding coordinate or distance. This is plausibly why controlling jointly for the silhouette score and the Caliński–Harabasz index only lowers the five medians in Table 1 slightly (for example, from +0.93 to +0.87 for `glm-5` and from +0.86 to +0.76 for `gpt-3.5-turbo`) and why every judge remains in the strong range.

Third, the criterion is inserted at both the codebook generation stage and the reference-building stage, so the judge's reference is strongly conditioned on it. The domain/intent pairs show this most clearly: MASSIVE(D) and MASSIVE(I) are benchmark clusterings of the exact same documents but using different criteria, and every judge still correlates moderately to strongly with both, scoring the domain labellings between +0.86 and +0.93 and the intent labellings between +0.59 and +0.74. This suggests the judge is responding to the criterion sentence rather than a fixed notion of similarity.

The medians only span +0.86 (`gpt-3.5-turbo`) to +0.93 (`glm-5`) despite a three-year gap in release date (2023 against 2026). As such, above a basic capability floor, LLM capacity does not seem to limit the judge's correlation with the benchmark labelling, which practically means the judge can be run with a cheap model.

The judge does have one systematic weakness. CLINC(I) is the only corpus that falls in the weak band for every judge, with correlations between +0.16 and +0.30. It is also the only corpus on which significance is ever lost (for two of the five judges). Part of the mechanism lies in the judge's first stage. The codebook is discovered from a sample of 300 documents. At $k = 151$ that

is roughly two documents per intent, so many intents enter the sample once or not at all. The fitted line in Figure 6(a) implies that a corpus of this granularity generally receives a codebook of about 40 categories, roughly four times coarser than the benchmark labelling. Coarseness alone does not explain the collapse, however. The same line implies a similarly compressed codebook for MTOP(I) at $k = 102$, yet that corpus never approaches the weak band.

Under the joint control, CLINC(I) rises to between +0.57 and +0.69 for every judge. Once the geometric signal is held fixed, the judge's ranking still tracks AMI on this corpus. Therefore, the low raw correlation appears to be largely the geometric metrics obscuring the judge's agreement on this corpus, which is the negative gap effect of Figure 25, where CLINC(I) shows the largest negative gap for every judge. The compressed codebook is at most a contributing factor. CLINC(I) does reappear as the weakest corpus in every later experiment, although there the codebook's sampling ceiling matters directly, since granularity estimation and candidate ranking both depend on the resolution of the judge's reference.

The answer to both halves of the research question is therefore yes: given only the criterion, the judge's scores correlate strongly with human judgement, and that correlation is more than a restatement of what the label-free geometric metrics already measure.

5.1.2 Research Question 2

Research question 2 returned a negative result. No estimator places more than 4 of 19 corpora within ±10% of the benchmark k, and every fitted slope in Figure 6 lies below one. Looking at the calibration plots, all estimators regress towards the middle of the granularity range, and this happens for a different reason at each end of the scale. At high k, the systematic weakness discussed above (§5.1.1) applies. An estimator that sees 300 documents simply cannot enumerate 151 categories. This appears to be a sampling ceiling rather than a reasoning ceiling. In the low k regime, overshoot plausibly occurs because the prompt lacks more in-depth granularity constraints than the criterion. When instructed to list all evident categories, the LLM naturally captures the visible sub-topics within the broad benchmark classes, since nothing specifies a stopping criterion.

Direct is the most accurate method. It only has to state a number rather than write the categories down, so it can estimate more categories than its 300-document sample explicitly shows. Because of this, it is also the only method that ever guesses above 200. Its fitted slope of 0.67 is therefore also the steepest of the sample-based methods.

The remaining methods trade accuracy against stability in different ways. Pairwise reaches the highest concordance and a lower spread by leaning on the Ward hierarchy as a scaffold. Merge convergence is by far the most stable while also being the least accurate, so a stable estimator can be stably wrong. Simpson undershoots exactly as predicted in §3.2.1, and Probe collapses to a near-constant estimate of around 13.

The Judge sweep (slope 0.41) appears to be estimating the codebook's granularity rather than the benchmark's, since the AMI against the reference labelling is maximised when the candidate's granularity matches that of the reference. The judge therefore carries a prior on granularity, and the right response is to use it only where that prior affects every option equally, which is exactly what CLEAR does by comparing all candidates at the same k. Finally, the silhouette-argmax

baseline matches Direct on the within-1.5× count while barely co-varying with the benchmark granularity, so matching it is no evidence of tracking k.

The LLM estimators do carry a real signal, with most reaching a moderate CCC, but with the exception of Pairwise, their run-to-run spread is simply too high. An estimate that can double or halve under a fresh sample is not a usable k. This negative result is also not specific to the LLM. The regression to the middle and instability is unchanged under both replications.

The answer to research question 2 is therefore that, given only the criterion, LLMs track the direction of granularity but cannot infer its value. As discussed above, each method has its own failure mode, but the most apparent evidence of their overall unreliability is that no estimator places more than 4 of 19 corpora within ±10% of the benchmark k, and all but one vary by roughly their own size under a new sample. Of course, this answer is only as extensive as the eight methods that were tested. The space of possible prompts and procedures is unbounded, so potentially a better estimator than the ones covered here could always exist.

5.1.3 Research Question 3

Over the 19 benchmarks and five runs per benchmark, CLEAR reaches a mean NMI of 79.0 and a mean ACC of 68.6. The per-benchmark extrema run from SNIPS at the top down to CLINC(D) in NMI and FewRel in ACC at the bottom (§4.3.1). Compared against seed-paired k-means on the same embeddings, CLEAR is ahead on 15 of 19 benchmarks in NMI (mean +2.6, significant at $p = 0.012$) and 13 of 19 in ACC (mean +3.7, $p = 0.055$). The analyses of §4.3.4 and §4.4.3 show how this gain can be decomposed. Always picking the k-means candidate gives a mean NMI of 76.4 (ACC 64.9). Selecting the candidate with the judge instead raises this to 78.4 (ACC 67.8), and the repair step adds a further +0.66 (ACC +0.78), arriving at CLEAR's final 79.0 (ACC 68.6). As such, roughly three quarters of the gain comes from selecting a better candidate and one quarter from repairing it.

In the candidate selection, the judge picks one of the two exemplar-initialised candidates in 71% of runs. While random initialisation places centres in dense embedding regions, codebook initialisation starts at criterion-aligned category prototypes. This should help the final partition land closer to a local optimum aligned with the one-sentence criterion, which is exactly what we want.

A natural concern with this selection pattern is circularity, since in the unmatched configuration the same LLM produces the exemplar artifacts and judges the candidates (§2.6.2). The matched configuration offers a partial control, as its judge is `gemini-2.5-flash` rather than `glm-5`, and the cross-family judge still selects the exemplar-initialised candidates most often (48% of the 95 matched runs against 71% under the same-family judge). The preference therefore does not appear to be purely self-recognition, although a systematic test of same-model preference in judge-based selection remains future work.

The two largest gains over simple k-means in Figure 8 are for benchmarks in which we expect this criterion-aligned start to matter most. First, on FewNerd (+15.8 NMI, panel (a)), the criterion is the type of a named entity (a person, place, organisation and so on) singled out at the end of each text, but a sentence embedding mostly encodes the sentence's overall topic, so k-means from random starts recovers topic structure rather than entity types. Thus, starting

from the embedded prototypes of LLM-generated entities encourages the initialisation to reflect the intended criterion and thus pull the clustering towards it. Second, StackEx-S2S (+19.4 ACC, panel (b)) contains 121 classes. In this fine-grained regime, random initialisation often leaves benchmark classes without centres, and these classes are then often merged, whereas the codebook seeds a centre for every category the LLM found, based on the criterion.

The rewritten candidate wins only 1 of 95 runs (Table 3). This can plausibly be attributed to the fact that compressing each document to a 15-word phrase discards too much information the encoder needs. The averaged variants, which mix the rewrite embeddings into the raw ones instead of replacing them altogether, are chosen on 37 total runs (30 exemplar-initialised, 4 GMM and 3 k-means in Table 3). As such, the rewritten version is a useful supplement to the original rather than a full replacement itself. This is mostly in line with other SOTA LLM-in-the-loop algorithms in the literature such as LUMI, which averages pseudo-labels into the original vectors, and Few-Shot Clustering which concatenates keyphrases (§2.4.1).

The judge's advantage as a selector is larger on NMI than on ACC (outperforming silhouette selection by 0.8 and 0.2 in panels (a) and (b) of Figure 12). The judge scores using AMI, which, like NMI, does not penalise splitting a benchmark class, whereas ACC's one-to-one matching does. Therefore, the judge aligns with NMI by construction, and only indirectly with ACC. The repair gain is small relative to the candidate selection gain by design. Only 600 documents per run are reclassified and each round is kept only if the judge score rises, so the 26 untouched runs in Figure 9 are those where the judge rejected the first round outright. Where repair actually does act, it helps in 60 runs and hurts in 9.

It appears to help most where the text explicitly states the category, but the embedding does not separate it. On StackOverflow (+3.03 NMI repair gain in Figure 9), which is clustered by programming topic, titles about different technologies share most of their words (e.g. "how to parse a JSON string in Java" sits next to the same question about Python). Many titles therefore lie between centroids, even though each explicitly names its topic. The LLM reads the documents and can fix exactly those that the geometry placed incorrectly.

On MTOP(D), which clusters by assistant command domain and is the one corpus where repair consistently hurts (−1.87 NMI in the same figure), the opposite happens. The pre-repair clustering is already close to the benchmark (96.3 NMI before repair), so the uncertain documents are often not mistakes, but can genuinely fit two assistant domains at once. For example, an utterance like "remind me to call mom" belongs to the reminder domain under the benchmark's annotation rules, but interpreting it as a calling request is just as defensible. The LLM thus sometimes picks the wrong domain. Because each round is accepted by the judge rather than by the benchmark labels, these changes are kept, causing agreement with the benchmark to fall.

The main failure case of the CLEAR algorithm is the same as in research question 1. On CLINC(I), the judge cannot reliably order the eight candidates (§5.1.1) since the codebook covers only a fraction of the 151 intents. Consequently, the selected candidate is no better than plain k-means and is clearly worse on ACC, accounting for the single large ACC loss in panel (b) of Figure 8 (−14.1). Repair does not improve the performance, so it changes the score by almost nothing (+0.04 NMI in Figure 9). However, this weakness stays limited to a few corpora, and CLEAR still reaches a high mean NMI and ACC over the full suite (§4.3.1).

The answer to research question 3 is therefore that an LLM acting as a criterion-equipped partitioner and judge does align unsupervised clustering more closely with the partitions humans draw, across nearly every corpus and criterion type in the suite. Most of this alignment comes from selecting a criterion-aligned clustering rather than repairing one. This alignment fails mainly where the granularity of the human taxonomy exceeds what the judge's sampled reference can resolve.

5.1.4 Research Question 4

To remind the reader, in the unmatched configuration, CLEAR is compared to SOTA results on each benchmark under the default configuration of research question 3 (`gemini-embedding-001` as the encoder and `glm-5` as the LLM). Conversely, in the matched configuration, it adopts each benchmark's NMI record holder's encoder and LLM, so only the algorithm itself differs. In the unmatched configuration, CLEAR beats the best published result on most benchmarks on both NMI and ACC (Figure 10). Matching the encoder and LLM to each record holder lowers the win counts, but a substantial share of the benchmarks are still won.

This is a strong result since it means this work's general, criterion-driven algorithm, frozen across all 19 corpora, holds its own against a large portion of the 19 separately optimised state-of-the-art records (8 of 19 in NMI and 9 of 19 in ACC). The results of this analysis suggest that the advantage on those benchmarks is unlikely to come from stronger configurations and most likely lies in the algorithm itself. This default configuration is, however, almost certainly stronger than any configuration behind the published records, as both `gemini-embedding-001` and `glm-5` postdate every encoder and LLM used by the record holders (Table 8).

Figure 11 suggests that both ingredients matter, but in different amounts. Swapping the encoder costs roughly twice as much mean performance as swapping the LLM on NMI, and more than three times as much on ACC. This makes sense when analysing CLEAR's design. The encoder alone likely does most of the work in shaping the distribution of text in the embedding space. An LLM can enhance this positioning (via the averaged or rewritten embeddings), but the encoder largely dictates the primary structure. Everything else the LLM contributes (the codebook, exemplar starts, the judge's selection, and repair rounds) builds on top of this base representation to guide or refine the clustering.

As such, a stronger encoder should improve the whole clustering pipeline, and `gemini-embedding-001` (Jul 2025) is stronger than many SOTA encoders in the literature, such as the `instructor-large` (Dec 2022) encoder used in 13 out of 19 benchmarks. As expected, CLINC(I) is again the weakest benchmark in the matched configuration, for the same reason as in the previous research questions. The judge's sampled reference cannot resolve its 151-intent granularity (§5.1.1).

The answer to research question 4 is therefore a qualified yes. With the encoder and language model held equal, the single, general CLEAR algorithm matches or exceeds the published LLM-in-the-loop record on a fair share of the suite, falls behind mainly where the record holder is a specialist for that corpus's criterion, and gains a distinct overall advantage as soon as its ingredients are allowed to be modern ones.

5.2 Token Cost

Both the codebook judge and the CLEAR algorithm have a specific token budget. Let N be the corpus size, $\bar{\ell}$ the average document length in tokens, k the target granularity and m the LLM-generated codebook size (number of categories in the codebook). Of these variables, only m is unknown before running, but it can be budgeted in advance from k, since the codebook calibration of Figure 6(a) predicts $m \approx 2^{2.25}\, k^{0.43} \approx 4.8\, k^{0.43}$. Table 4 fully itemises the expected input tokens per stage. We assume the fixed input token usage of the prompt templates is negligible. Output tokens are assumed negligible everywhere except in the rewrite stage. Here, the returned phrases are expected to add roughly another $20N$ tokens.

Stage	LLM calls	Input tokens	Output tokens
Codebook discovery	1	$300\,\bar{\ell}$	—
Reference assignment	50	$1{,}000\,\bar{\ell} + 1{,}250\,m$	—
Judge total	51	$1{,}300\,\bar{\ell} + 1{,}250\,m$	—
Document rewrite	$N/25$	$N\,\bar{\ell}$	$\approx 20\,N$
Category exemplars	m	$\approx 100\,m$	—
Cluster naming	k	$8\,k\,\bar{\ell}$	—
Boundary repair	30	$600\,\bar{\ell} + 150\,k$	—
CLEAR total (with judge)	$N/25 + k + m + 81$	$(N + 8k + 1{,}900)\,\bar{\ell} + 1{,}350\,m + 150\,k$	$\approx 20\,N$

Table 4: Expected LLM token cost by stage, as a function of corpus size N, average document length $\bar{\ell}$ in tokens, target granularity k and codebook size m. Call counts are exact per protocol, and we assume the number of repair rounds is at the three-round maximum.

Summed over the full 19-corpus suite, one full CLEAR pass therefore costs about 6.1 million tokens, roughly 4.7 million of input and 1.4 million of output, of which the 19 judge references account for about 1.4 million input tokens (where N sums to 69,002 documents, $\bar{\ell}$ averages roughly 40 tokens, k ranges from 5 to 151). Using `glm-5` as the configuration LLM, with an input token cost of \$1.00 per million and an output token cost of \$3.20 per million, that pass comes to

roughly $9. That is equivalent to an average dollar cost of roughly $0.0133 per 100 documents. Encoding cost is not included since its total cost is negligible.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

This thesis asked whether the natural language reasoning of an LLM, given access to a one-sentence clustering criterion, can be used to evaluate, infer the granularity of, and generate the clustering of an unlabelled corpus. Each of the four research questions answered one part of that question, and together they draw a consistent picture of where criterion-conditioned LLM reasoning helps, where it does not, and why.

For evaluation, the answer is yes. The codebook judge of §3.1.1 converts a one-sentence criterion and the corpus into a reference labelling of that corpus and scores any candidate clustering against it without ever seeing a benchmark label. Across the full suite of corpora and criterion types, its scores correlate significantly with agreement against the benchmark labelling on nearly every corpus for every one of the five LLMs tested, with median correlations in the strong to very strong range. The correlation mainly survives joint control for the silhouette score and the Caliński–Harabasz index. The judge is therefore not a restatement of geometry. It does fail in one predictable regime: corpora whose granularity exceeds what its sampled discovery stage can resolve (e.g. CLINC(I)).

For granularity estimation, the answer is a clear no. None of the eight granularity estimators, whether they rely on enumeration, direct estimation, vote aggregation or hybrid geometric designs, recovers the benchmark granularity with useful accuracy.

Every calibration slope lies below one, and most estimates vary by roughly their own size under different random seeds. Replicating the entire experiment with two further LLMs reproduced the same negative result, so the failure seems to stem not from the capacity of any single model but from the task itself. The practical consequence is therefore that granularity is an analyst's choice and must remain a user-controlled input if the CLEAR algorithm is to be used in production.

For actual clustering, the answer is also yes but within some defined limits. CLEAR uses the fact that the validated judge algorithm from research question 1 correlates significantly with human evaluation to perform clustering itself. It does this by turning the judge into an active component that selects among eight candidate clusterings and performs an additional bounded repair step. This lets the clustering algorithm be tailored to the corpus by choosing the base clustering algorithm that produces the highest human-aligned score. It beats seed-paired k-means

on the same embeddings on most benchmarks on both metrics. As expected, most of that gain comes from the candidate selection process and the remainder from repairing it, where an LLM conditioned on the criterion is used in both.

Against the published state of the art, CLEAR wins most benchmarks in its own modern configuration, and still holds a substantial share of the records when compared on an LLM- and encoder-equal basis. The decomposition of LLM and encoder contribution attributes roughly twice as much of the effect of configuration to the encoder as to the LLM. As such, the results indicate that the choice of encoder, as well as the LLM, substantially affects CLEAR's performance.

One important caveat applies to every result in this thesis. Whenever results show the judge's agreement with human judgement, an estimator's recovery of granularity, or a clustering's quality against human labels, the quantities computed measure agreement with one specific benchmark labelling, produced by one or more particular annotators under a given annotation scheme. As such, results that show agreement with human clustering can only be said to show alignment with the annotators who built these benchmarks. The generalisation to other human clusterings is therefore strictly an assumption rather than a finding.

6.2 Contributions

The contributions in this work can be summarised in three main points:

1. A new label-free, LLM-powered clustering evaluation method. In the first large-scale validation of its kind, its scores are shown to correlate strongly with benchmark labels across corpora, criterion types and judge LLMs, beyond what geometric metrics already measure.

2. A systematic study showing that the granularity of a human taxonomy cannot be reliably inferred from the criterion and the unlabelled corpus alone, at least not by any of the estimation methods tested here, across three different LLMs.

3. CLEAR, a general clustering algorithm that takes as input only the corpus, the criterion and a target k. It beats the published state of the art on most benchmarks in its own configuration, and still wins a substantial share when matched to each record holder's own encoder and LLM configuration. As of the time of writing, this work's proposed clustering algorithm is therefore fully competitive with state-of-the-art LLM-in-the-loop text clustering algorithms.

6.3 Future Work

As identified in §5.1.1, the granularity ceiling is the biggest weakness of both the codebook judge and the CLEAR algorithm. Most weaknesses that recur across the four research questions trace back to the judge's discovery-stage reading of a 300-document sample, which prevents it from resolving taxonomies with 100 or more categories, such as CLINC(I). This 300-document choice was made to trade off context-window fit against sampling breadth. A promising next step could be a hierarchical discovery stage. The LLM would first build a coarse codebook from the sample

and would then be prompted to split each category into subcategories. This would multiply the granularity the judge can resolve without enlarging the actual sample.

In the configuration ablation of research question 4, we found that the encoder contributed roughly twice as much to the overall performance of the CLEAR algorithm as the LLM. Enhancing the encoder is therefore a natural target for future research. CLEAR currently treats the encoder as frozen. Fine-tuning the encoder on judge-derived signals, for example, criterion-conditioned triplets in the style of ClusterLLM but labelled by the codebook judge rather than by benchmark labels, would allow the representation to adapt to the criterion and might therefore substantially boost performance. A CLEAR variant fine-tuned this way could then also be compared directly against other state-of-the-art encoder fine-tuning methods. This was deliberately left out of the scope in this work.

Finally, the caveat of §6.1 could be investigated empirically. All results here measure agreement with one benchmark labelling per corpus and not human labelling in general. A large-scale human study in which fresh annotators cluster corpora under stated criteria would test whether the judge tracks human judgement in general rather than specific benchmarks. This would generalise the result beyond the annotators who built the benchmarks.

Data and Code Availability

All relevant code and data behind this thesis are available at this repository.

Bibliography

Charu C. Aggarwal, Alexander Hinneburg, and Daniel A. Keim. On the surprising behavior of distance metrics in high dimensional space. In *Database Theory — ICDT 2001*, volume 1973 of *Lecture Notes in Computer Science*, pages 420–434. Springer, 2001.

Wenbin An, Wenkai Shi, Feng Tian, Haonan Lin, QianYing Wang, Yaqiang Wu, Mingxiang Cai, Luyan Wang, Yan Chen, Haiping Zhu, and Ping Chen. Generalized category discovery with large language models in the loop. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 8653–8665. Association for Computational Linguistics, 2024.

Charles E. Antoniak. Mixtures of Dirichlet processes with applications to Bayesian nonparametric problems. *The Annals of Statistics*, 2(6):1152–1174, 1974.

Olatz Arbelaitz, Ibai Gurrutxaga, Javier Muguerza, Jesús M. Pérez, and Iñigo Perona. An extensive comparative study of cluster validity indices. *Pattern Recognition*, 46(1):243–256, 2013.

Abhinav Arora, Akshat Shrivastava, Mrinal Mohit, Lorena Sainz-Maza Lecanda, and Ahmed Aly. Cross-lingual transfer learning for intent detection of COVID-19 utterances. In *Proceedings of the 1st Workshop on NLP for COVID-19*, 2020.

David Arthur and Sergei Vassilvitskii. k-means++: The advantages of careful seeding. In *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 1027–1035. SIAM, 2007.

Sugato Basu, Arindam Banerjee, and Raymond J. Mooney. Active semi-supervision for pairwise constrained clustering. In *Proceedings of the 2004 SIAM International Conference on Data Mining*, pages 333–344. SIAM, 2004.

Parishad BehnamGhader, Vaibhav Adlakha, Marius Mosbach, Dzmitry Bahdanau, Nicolas Chapados, and Siva Reddy. LLM2Vec: Large language models are secretly powerful text encoders. In *Proceedings of the First Conference on Language Modeling (COLM)*, 2024.

Asa Ben-Hur, André Elisseeff, and Isabelle Guyon. A stability based method for discovering structure in clustered data. *Pacific Symposium on Biocomputing*, 7:6–17, 2002.

Kevin Beyer, Jonathan Goldstein, Raghu Ramakrishnan, and Uri Shaft. When is "nearest neighbor" meaningful? In *Database Theory — ICDT '99*, volume 1540 of *Lecture Notes in Computer Science*, pages 217–235. Springer, 1999.

David M. Blei, Andrew Y. Ng, and Michael I. Jordan. Latent Dirichlet allocation. *Journal of Machine Learning Research*, 3:993–1022, 2003.

Vincent D. Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment*, 2008(10):P10008, 2008.

Piotr Bojanowski, Edouard Grave, Armand Joulin, and Tomas Mikolov. Enriching word vectors with subword information. *Transactions of the Association for Computational Linguistics*, 5: 135–146, 2017.

Tadeusz Caliński and Jerzy Harabasz. A dendrite method for cluster analysis. *Communications in Statistics*, 3(1):1–27, 1974.

Ricardo J. G. B. Campello, Davoud Moulavi, and Jörg Sander. Density-based clustering based on hierarchical density estimates. In *Advances in Knowledge Discovery and Data Mining (PAKDD 2013)*, volume 7819 of *Lecture Notes in Computer Science*, pages 160–172. Springer, 2013.

Iñigo Casanueva, Tadas Temčinas, Daniela Gerz, Matthew Henderson, and Ivan Vulić. Efficient intent detection with dual sentence encoders. In *Proceedings of the 2nd Workshop on Natural Language Processing for Conversational AI*, pages 38–45. Association for Computational Linguistics, 2020.

Jonathan Chang, Jordan Boyd-Graber, Sean Gerrish, Chong Wang, and David M. Blei. Reading tea leaves: How humans interpret topic models. In *Advances in Neural Information Processing Systems 22*, pages 288–296, 2009.

CLINC. CLINC-OOS "plus" configuration with `domains.json` intent-to-domain mapping. https://huggingface.co/datasets/clinc/clinc_oos and https://github.com/clinc/oos-eval, 2019. Accessed August 2026.

Alice Coucke, Alaa Saade, Adrien Ball, Théodore Bluche, Alexandre Caulier, David Leroy, Clément Doumouro, Thibault Gisselbrecht, Francesco Caltagirone, Thibaut Lavril, et al. Snips voice platform: An embedded spoken language understanding system for private-by-design voice interfaces, 2018. arXiv:1805.10190.

David L. Davies and Donald W. Bouldin. A cluster separation measure. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-1(2):224–227, 1979.

Maarten De Raedt, Fréderic Godin, Thomas Demeester, and Chris Develder. IDAS: Intent discovery with abstractive summarization. In *Proceedings of the 5th Workshop on NLP for Conversational AI*, pages 71–88. Association for Computational Linguistics, 2023.

DeepPavlov. CLINC150 dataset (hugging face hub). https://huggingface.co/datasets/DeepPavlov/clinc150, 2019. Accessed August 2026.

Scott Deerwester, Susan T. Dumais, George W. Furnas, Thomas K. Landauer, and Richard Harshman. Indexing by latent semantic analysis. *Journal of the American Society for Information Science*, 41(6):391–407, 1990.

Arthur P. Dempster, Nan M. Laird, and Donald B. Rubin. Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society: Series B*, 39(1):1–38, 1977.

Shumin Deng, Ningyu Zhang, Jiaojian Kang, Yichi Zhang, Wei Zhang, and Huajun Chen. Meta-learning with dynamic-memory-based prototypical network for few-shot event detection. In *Proceedings of the 13th International Conference on Web Search and Data Mining*, pages 151–159. ACM, 2020.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT 2019*, pages 4171–4186. Association for Computational Linguistics, 2019.

Jairo Díaz-Rodríguez. Summaries as centroids for interpretable and scalable LLM-based text clustering. In *Proceedings of the Fourteenth International Conference on Learning Representations (ICLR)*, 2026. Also known as k-LLMmeans (arXiv:2502.09667).

Ning Ding, Guangwei Xu, Yulin Chen, Xiaobin Wang, Xu Han, Pengjun Xie, Haitao Zheng, and Zhiyuan Liu. Few-NERD: A few-shot named entity recognition dataset. In *Proceedings of ACL-IJCNLP 2021*, pages 3198–3213. Association for Computational Linguistics, 2021.

Kenneth Enevoldsen, Isaac Chung, Imene Kerboua, Márton Kardos, Ashwin Mathur, David Stap, et al. MMTEB: Massive multilingual text embedding benchmark. In *Proceedings of the Thirteenth International Conference on Learning Representations (ICLR)*, 2025.

Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96)*, pages 226–231. AAAI Press, 1996.

Kawin Ethayarajh. How contextual are contextualized word representations? Comparing the geometry of BERT, ELMo, and GPT-2 embeddings. In *Proceedings of EMNLP-IJCNLP 2019*, pages 55–65. Association for Computational Linguistics, 2019.

Facebook AI. M-CID: multilingual COVID-19 intent dataset (english split). https://fb.me/covid_mcid_dataset, 2020. Accessed August 2026.

Zijin Feng, Luyang Lin, Lingzhi Wang, Hong Cheng, and Kam-Fai Wong. LLMEdgeRefine: Enhancing text clustering with LLM-based boundary point refinement. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 18455–18462. Association for Computational Linguistics, 2024.

Jack FitzGerald, Christopher Hench, Charith Peris, Scott Mackie, Kay Rottmann, Ana Sanchez, et al. MASSIVE: A 1M-example multilingual natural language understanding dataset with 51 typologically-diverse languages. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pages 4277–4302. Association for Computational Linguistics, 2023.

Tianyu Gao, Xingcheng Yao, and Danqi Chen. SimCSE: Simple contrastive learning of sentence embeddings. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 6894–6910. Association for Computational Linguistics, 2021.

Gemini Team, Google. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities, 2025. arXiv:2507.06261.

Gemma Team, Google. Gemma 2: Improving open language models at a practical size, 2024. arXiv:2408.00118.

Fabrizio Gilardi, Meysam Alizadeh, and Maël Kubli. ChatGPT outperforms crowd workers for text-annotation tasks. *Proceedings of the National Academy of Sciences*, 120(30):e2305016120, 2023.

Derek Greene and Pádraig Cunningham. BBC news article dataset (full text). http://mlg.ucd.ie/datasets/bbc.html, 2006a. Accessed August 2026.

Derek Greene and Pádraig Cunningham. Practical solutions to the problem of diagonal dominance in kernel document clustering. In *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pages 377–384. ACM, 2006b.

Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, et al. A survey on LLM-as-a-judge, 2024. arXiv:2411.15594.

Xu Han, Hao Zhu, Pengfei Yu, Ziyun Wang, Yuan Yao, Zhiyuan Liu, and Maosong Sun. FewRel: A large-scale supervised few-shot relation classification dataset with state-of-the-art evaluation. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 4803–4809. Association for Computational Linguistics, 2018.

Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, et al. Array programming with NumPy. *Nature*, 585:357–362, 2020.

Will Held. MTOP dataset, english test split (hugging face hub). https://huggingface.co/datasets/WillHeld/mtop, 2021. Accessed August 2026.

Mengze Hong, Wailing Ng, Chen Jason Zhang, Yuanfeng Song, and Di Jiang. Dial-In LLM: Human-aligned LLM-in-the-loop intent clustering for customer service dialogues. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2025.

Alexander Hoyle, Lorena Calvo-Bartolomé, Jordan Boyd-Graber, and Philip Resnik. ProxAnn: Use-oriented evaluations of topic models and document clustering. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15872–15897. Association for Computational Linguistics, 2025.

Chen Huang and Guoxiu He. Text clustering as classification with LLMs. In *Proceedings of SIGIR-AP 2025*. ACM, 2025.

Lawrence Hubert and Phipps Arabie. Comparing partitions. *Journal of Classification*, 2(1): 193–218, 1985.

John D. Hunter. Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.

Harold W. Kuhn. The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955.

Stefan Larson, Anish Mahendran, Joseph J. Peper, Christopher Clarke, Andrew Lee, Parker Hill, et al. An evaluation dataset for intent classification and out-of-scope prediction. In *Proceedings of EMNLP-IJCNLP 2019*, pages 1311–1316. Association for Computational Linguistics, 2019.

Chankyu Lee, Rajarshi Roy, Mengyao Xu, Jonathan Raiman, Mohammad Shoeybi, Bryan Catanzaro, and Wei Ping. NV-Embed: Improved techniques for training LLMs as generalist embedding models. In *Proceedings of the Thirteenth International Conference on Learning Representations (ICLR)*, 2025a.

Jinhyuk Lee, Feiyang Chen, Sahil Dua, Daniel Cer, Madhuri Shanbhogue, Iftekhar Naim, et al. Gemini embedding: Generalizable embeddings from Gemini, 2025b. arXiv:2503.07891.

Bohan Li, Hao Zhou, Junxian He, Mingxuan Wang, Yiming Yang, and Lei Li. On the sentence embeddings from pre-trained language models. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 9119–9130. Association for Computational Linguistics, 2020.

Haoran Li, Abhinav Arora, Shuohui Chen, Anchit Gupta, Sonal Gupta, and Yashar Mehdad. MTOP: A comprehensive multilingual task-oriented semantic parsing benchmark. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics*, pages 2950–2962. Association for Computational Linguistics, 2021.

Zehan Li, Xin Zhang, Yanzhao Zhang, Dingkun Long, Pengjun Xie, and Meishan Zhang. Towards general text embeddings with multi-stage contrastive learning, 2023. arXiv:2308.03281.

Jinggui Liang, Lizi Liao, Hao Fei, Bobo Li, and Jing Jiang. Actively learn from LLMs with uncertainty propagation for generalized category discovery. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 7845–7858. Association for Computational Linguistics, 2024.

I-Fan Lin, Faegheh Hasibi, and Suzan Verberne. SPILL: Domain-adaptive intent clustering based on selection and pooling with large language models. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 15723–15737. Association for Computational Linguistics, 2025.

I-Fan Lin, Faegheh Hasibi, and Suzan Verberne. LLMs enable bag-of-texts representations for short-text clustering. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6432–6447. Association for Computational Linguistics, 2026a.

I-Fan Lin, Faegheh Hasibi, and Suzan Verberne. LUMI: Unsupervised intent clustering with multiple pseudo-labels. In *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 3958–3963. ACM, 2026b. doi: 10.1145/3805712.3809876.

Lawrence I-Kuei Lin. A concordance correlation coefficient to evaluate reproducibility. *Biometrics*, 45(1):255–268, 1989. doi: 10.2307/2532051.

Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. G-Eval: NLG evaluation using GPT-4 with better human alignment. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 2511–2522. Association for Computational Linguistics, 2023.

Stuart P. Lloyd. Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982.

Sindy Löwe and Emily Silcock. Agentic clustering: Controllable text taxonomies via multi-agent refinement, 2026. arXiv:2606.01255.

James MacQueen. Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, volume 1, pages 281–297. University of California Press, 1967.

Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space, 2013a. arXiv:1301.3781.

Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S. Corrado, and Jeffrey Dean. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems 26*, pages 3111–3119, 2013b.

Justin K. Miller and Tristram J. Alexander. Human-interpretable clustering of short text using large language models. *Royal Society Open Science*, 12(1):241692, 2025.

MTEB. amazon_massive_intent (english split, hugging face hub). https://huggingface.co/datasets/mteb/amazon_massive_intent, 2023a. Accessed August 2026; scenario prefixes used for the domain split.

MTEB. reddit-clustering (hugging face hub). https://huggingface.co/datasets/mteb/reddit-clustering, 2023b. Accessed August 2026; largest split, 6,000-document subsample.

Niklas Muennighoff, Nouamane Tazi, Loïc Magne, and Nils Reimers. MTEB: Massive text embedding benchmark. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, pages 2014–2037. Association for Computational Linguistics, 2023.

Niklas Muennighoff, Hongjin Su, Liang Wang, Nan Yang, Furu Wei, Tao Yu, Amanpreet Singh, and Douwe Kiela. Generative representational instruction tuning. In *Proceedings of the Thirteenth International Conference on Learning Representations (ICLR)*, 2025.

Andrew Y. Ng, Michael I. Jordan, and Yair Weiss. On spectral clustering: Analysis and an algorithm. In *Advances in Neural Information Processing Systems 14*, pages 849–856. MIT Press, 2002.

OpenAI. Introducing ChatGPT and Whisper APIs. https://openai.com/index/introducing-chatgpt-and-whisper-apis/, 2023. gpt-3.5-turbo model release; accessed August 2026.

OpenAI. New embedding models and API updates. https://openai.com/index/new-embedding-models-and-api-updates/, 2024a.

OpenAI. GPT-4o system card, 2024b. arXiv:2410.21276; also covers gpt-4o-mini.

OpenAI. Introducing GPT-4.1 in the API. https://openai.com/index/gpt-4-1/, 2025. gpt-4.1 model family, including gpt-4.1-mini; accessed August 2026.

Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems 35*, 2022.

Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

Dan Pelleg and Andrew W. Moore. X-means: Extending k-means with efficient estimation of the number of clusters. In *Proceedings of the Seventeenth International Conference on Machine Learning*, pages 727–734. Morgan Kaufmann, 2000.

Jeffrey Pennington, Richard Socher, and Christopher D. Manning. GloVe: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, pages 1532–1543. Association for Computational Linguistics, 2014.

Alina Petukhova, João P. Matos-Carvalho, and Nuno Fachada. Text clustering with large language model embeddings. *International Journal of Cognitive Computing in Engineering*, 6:100–108, 2025.

Chau Minh Pham, Alexander Hoyle, Simeng Sun, Philip Resnik, and Mohit Iyyer. TopicGPT: A prompt-based topic modeling framework. In *Proceedings of NAACL-HLT 2024*, pages 2956–2984. Association for Computational Linguistics, 2024.

PolyAI. Banking77 dataset (hugging face hub). https://huggingface.co/datasets/PolyAI/banking77, 2020. Accessed August 2026.

Peijun Qing, Puneet Mathur, Nedim Lipka, Varun Manjunatha, Ryan Rossi, Franck Dernoncourt, Saeed Hassanpour, and Soroush Vosoughi. Cluster-R1: Large reasoning models are instruction-following clustering agents, 2026. arXiv:2603.23518.

Miloš Radovanović, Alexandros Nanopoulos, and Mirjana Ivanović. Hubs in space: Popular nearest neighbors in high-dimensional data. *Journal of Machine Learning Research*, 11:2487–2531, 2010.

Md Rashadul Hasan Rakib. Tweet dataset, `tweet-original-order.txt` (short-text-clustering-enhancement repository). https://github.com/rashadulrakib/short-text-clustering-enhancement, 2020. Accessed August 2026.

Carl Edward Rasmussen. The infinite Gaussian mixture model. In *Advances in Neural Information Processing Systems 12*, pages 554–560. MIT Press, 2000.

Philipp Rautenstrauch and Uwe Ohler. Shortcomings of silhouette in single-cell integration benchmarking. *Nature Biotechnology*, 2025. Advance online publication.

Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of EMNLP-IJCNLP 2019*, pages 3982–3992. Association for Computational Linguistics, 2019.

Andrew Rosenberg and Julia Hirschberg. V-measure: A conditional entropy-based external cluster evaluation measure. In *Proceedings of EMNLP-CoNLL 2007*, pages 410–420. Association for Computational Linguistics, 2007.

Peter J. Rousseeuw. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65, 1987.

Gerard Salton and Christopher Buckley. Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5):513–523, 1988.

Gerard Salton, Anita Wong, and Chung-Shu Yang. A vector space model for automatic indexing. *Communications of the ACM*, 18(11):613–620, 1975.

David Sculley. Web-scale k-means clustering. In *Proceedings of the 19th International Conference on World Wide Web*, pages 1177–1178. ACM, 2010.

Edward H. Simpson. Measurement of diversity. *Nature*, 163:688, 1949.

Snips. SNIPS NLU benchmark, 2017-06 custom intent engines. https://github.com/sonos/nlu-benchmark, 2018. Accessed August 2026; 3,010-document stratified subsample.

Michael Steinbach, George Karypis, and Vipin Kumar. A comparison of document clustering techniques. Technical Report 00-034, University of Minnesota, Department of Computer Science, 2000.

Hongjin Su, Weijia Shi, Jungo Kasai, Yizhong Wang, Yushi Hu, Mari Ostendorf, Wen-tau Yih, Noah A. Smith, Luke Zettlemoyer, and Tao Yu. One embedder, any task: Instruction-finetuned text embeddings. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 1102–1121. Association for Computational Linguistics, 2023.

Jianlin Su, Jiarun Cao, Weijie Liu, and Yangyiwen Ou. Whitening sentence representations for better semantics and faster retrieval, 2021. arXiv:2103.15316.

Alex Tamkin, Miles McCain, Kunal Handa, Esin Durmus, Liane Lovitt, Ankur Rathi, et al. Clio: Privacy-preserving insights into real-world AI use, 2024. arXiv:2412.13678.

Robert L. Thorndike. Who belongs in the family? *Psychometrika*, 18(4):267–276, 1953.

Robert Tibshirani, Guenther Walther, and Trevor Hastie. Estimating the number of clusters in a data set via the gap statistic. *Journal of the Royal Statistical Society: Series B*, 63(2):411–423, 2001.

Vincent A. Traag, Ludo Waltman, and Nees Jan van Eck. From Louvain to Leiden: Guaranteeing well-connected communities. *Scientific Reports*, 9:5233, 2019.

Sagar Vaze, Kai Han, Andrea Vedaldi, and Andrew Zisserman. Generalized category discovery. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 7492–7501. IEEE, 2022.

Nguyen Xuan Vinh, Julien Epps, and James Bailey. Information theoretic measures for clusterings comparison: Variants, properties, normalization and correction for chance. *Journal of Machine Learning Research*, 11:2837–2854, 2010.

Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, et al. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020.

Vijay Viswanathan, Kiril Gashteovski, Carolin Lawrence, Tongshuang Wu, and Graham Neubig. Large language models enable few-shot clustering. *Transactions of the Association for Computational Linguistics*, 12:321–333, 2024.

Ulrike von Luxburg. A tutorial on spectral clustering. *Statistics and Computing*, 17(4):395–416, 2007.

Ulrike von Luxburg. Clustering stability: An overview. *Foundations and Trends in Machine Learning*, 2(3):235–274, 2010.

Ulrike von Luxburg, Robert C. Williamson, and Isabelle Guyon. Clustering: Science or art? In *Proceedings of ICML Workshop on Unsupervised and Transfer Learning*, volume 27 of *JMLR Workshop and Conference Proceedings*, pages 65–79, 2012.

Kiri Wagstaff, Claire Cardie, Seth Rogers, and Stefan Schroedl. Constrained k-means clustering with background knowledge. In *Proceedings of the Eighteenth International Conference on Machine Learning*, pages 577–584. Morgan Kaufmann, 2001.

Mengting Wan, Tara Safavi, Sujay Kumar Jauhar, Yujin Kim, Scott Counts, Jennifer Neville, et al. TnT-LLM: Text mining at scale with large language models. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 5836–5847. ACM, 2024.

Hongtao Wang, Taiyan Zhang, Renchi Yang, and Jianliang Xu. Cequel: Cost-effective querying of large language models for text clustering. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management (CIKM)*. ACM, 2025. doi: 10.1145/3746252.3761074.

Hongtao Wang, Renchi Yang, and Wenqing Lin. NILC: Discovering new intents with LLM-assisted clustering. In *Proceedings of the Nineteenth ACM International Conference on Web Search and Data Mining (WSDM)*, pages 671–680. ACM, 2026.

Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. Text embeddings by weakly-supervised contrastive pre-training, 2022. arXiv:2212.03533.

Peiyi Wang, Lei Li, Liang Chen, Zefan Cai, Dawei Zhu, Binghuai Lin, et al. Large language models are not fair evaluators. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 9440–9450. Association for Computational Linguistics, 2024.

Zeya Wang and Chenglong Ye. Deep clustering evaluation: How to validate internal clustering validation measures. *Journal of the American Statistical Association*, 2025. Advance online publication.

Zihan Wang, Jingbo Shang, and Ruiqi Zhong. Goal-driven explainable clustering via language descriptions. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 2803–2817. Association for Computational Linguistics, 2023.

Jr. Ward, Joe H. Hierarchical grouping to optimize an objective function. *Journal of the American Statistical Association*, 58(301):236–244, 1963.

Jiaming Xu. StackOverflow short-text dataset (title and label files). https://github.com/jacoxu/StackOverflow, 2017. Accessed August 2026; 3,000-document stratified subsample.

Jiaming Xu, Bo Xu, Peng Wang, Suncong Zheng, Guanhua Tian, Jun Zhao, and Bo Xu. Self-taught convolutional neural networks for short text clustering. *Neural Networks*, 88:22–31, 2017.

Yuwei Xu, Yifan Yuan, Vijay Viswanathan, and Graham Neubig. ClusterFusion: Hybrid clustering with embedding guidance and LLM adaptation, 2025. arXiv:2512.04350.

Jianhua Yin and Jianyong Wang. A model-based approach for text clustering with outlier detection. In *2016 IEEE 32nd International Conference on Data Engineering (ICDE)*, pages 625–636. IEEE, 2016.

Dun Zhang, Jiacheng Li, Ziyang Zeng, and Fulong Wang. Jasper and Stella: Distillation of SOTA embedding models, 2024a. arXiv:2412.19048.

Hanlei Zhang, Hua Xu, Xin Wang, Fei Long, and Kai Gao. A clustering framework for unsupervised and semi-supervised new intent discovery. *IEEE Transactions on Knowledge and Data Engineering*, 36(11):5468–5481, 2024b. doi: 10.1109/TKDE.2023.3340732.

Xiang Zhang and Yann LeCun. DBpedia-14 ontology classification dataset. https://s3.amazonaws.com/fast-ai-nlp/dbpedia_csv.tgz, 2015. Accessed August 2026; 3,010-document stratified subsample of the test split.

Xiang Zhang, Junbo Zhao, and Yann LeCun. Character-level convolutional networks for text classification. In *Advances in Neural Information Processing Systems 28*, pages 649–657, 2015.

Yuwei Zhang, Haode Zhang, Li-Ming Zhan, Xiao-Ming Wu, and Albert Lam. New intent discovery with pre-training and contrastive learning. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 256–269. Association for Computational Linguistics, 2022.

Yuwei Zhang, Zihan Wang, and Jingbo Shang. ClusterLLM released benchmark splits (FewRel, Few-NERD, FewEvent, ArXiv, StackExchange). https://github.com/zhang-yuwei/ClusterLLM, 2023a. Accessed August 2026; see Zhang et al. (2023), Appendix E for the re-splitting protocol.

Yuwei Zhang, Zihan Wang, and Jingbo Shang. ClusterLLM: Large language models as a guide for text clustering. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 13903–13920. Association for Computational Linguistics, 2023b.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36, Datasets and Benchmarks Track*, 2023.

Zhipu AI. GLM-5. https://open.bigmodel.cn, 2026. Large language model, accessed via API, August 2026.

Yuanjie Zhu, Liangwei Yang, Ke Xu, Weizhi Zhang, Zihe Song, Jindong Wang, and Philip S. Yu. LLM-MemCluster: Empowering large language models with dynamic memory for text clustering. In *Proceedings of the 4th Workshop on Towards Knowledgeable Foundation Models (KnowFM), ACL 2026*. Association for Computational Linguistics, 2026.

Appendix A

Literature Survey

A.1 Compared Methods and Benchmarks

Algorithm	Stage	Citation	Variant	Encoder	LLM
IDAS	Before	De Raedt et al. [2023]	—	MTP / paraphrase-mpnet	GPT-3 (text-davinci-003)
ClusterLLM	Before	Zhang et al. [2023b]	ClusterLLM-I	instructor-large	gpt-3.5-turbo
ClusterLLM	Before	Zhang et al. [2023b]	ClusterLLM-I-iter	instructor-large	gpt-3.5-turbo
ClusterLLM	Before	Zhang et al. [2023b]	ClusterLLM-E-iter	e5-large	gpt-3.5-turbo
Few-Shot Clust.	Before	Viswanathan et al. [2024]	keyphrase expansion	Instructor / DistilBERT	gpt-3.5-turbo-0301
LLMEdgeRefine	After	Feng et al. [2024]	GPT-3.5	Instructor	gpt-3.5-turbo-0301
SPILL	Before	Lin et al. [2025]	Summarizer (Gemma)	E5 / Instructor (+ LLM-as-embedder)	qwen2.5-7b / llama3.1-8b / gemma-2-9b
Cequel	During	Wang et al. [2025]	TriangleLLM +WCSC	instructor-large	gpt-4o-mini
NILC	During	Wang et al. [2026]	—	USNID encoder	gpt-4o-mini
k-LLMmeans	During	Díaz-Rodríguez [2026]	FS-multiple	text-embedding-3-small	gpt-4o
k-LLMmeans	During	Díaz-Rodríguez [2026]	+BERTopic hybrid	BERTopic + text-embedding-3-small	gpt-4o
LUMI	Before	Lin et al. [2026b]	instructor-large + Gemma	MiniLM-L6 / instructor-large	gemma-2-9b / llama3.1-8b
Bag-of-Texts	Before	Lin et al. [2026a]	E5 backbone	TF-IDF / BERT / MiniLM / E5	gemma-2-9b-it
LLM-MemCluster	LLM-native	Zhu et al. [2026]	—	— (encoder-free)	gpt-4.1-mini

Table 5: The 11 LLM-in-the-loop algorithms compared in this work, with one row per variant. Stage denotes the primary intervention point in the taxonomy of §2.4. The variant, encoder, and LLM columns give each paper's headline configuration.

The exact dataset artefacts used for each corpus, with access dates, are [PolyAI, 2020, Greene and Cunningham, 2006a, DeepPavlov, 2019, CLINC, 2019, Zhang et al., 2023a, Zhang and LeCun,

Benchmark	Year	Dataset paper	Criterion	Documents are	k	N
BBC News	2006	Greene and Cunningham [2006b]	Topic	news articles	5	2,225
DBPedia	2015	Zhang et al. [2015]	Entity type	encyclopedia abstracts	14	3,010
Tweet89	2016	Yin and Wang [2016]	Topic	news-event tweets	89	2,472
StackOverflow	2017	Xu et al. [2017]	Topic	programming Q&A titles	20	3,000
FewRel	2018	Han et al. [2018]	Relation	Wikipedia sentences	64	4,480
SNIPS	2018	Coucke et al. [2018]	Intent	voice-assistant queries	7	3,010
CLINC(I)	2019	Larson et al. [2019]	Intent	assistant utterances	151	5,500
CLINC(D)	2019	Larson et al. [2019]	Domain	assistant utterances	10	4,500
Banking77	2020	Casanueva et al. [2020]	Intent	banking queries	77	1,979
FewEvent	2020	Deng et al. [2020]	Event type	event mentions	34	4,742
M-CID	2020	Arora et al. [2020]	Intent	COVID-19 queries	16	1,745
FewNerd	2021	Ding et al. [2021]	Entity type	entity mentions	58	3,789
MTOP(D)	2021	Li et al. [2021]	Domain	assistant commands	11	4,386
MTOP(I)	2021	Li et al. [2021]	Intent	assistant commands	102	4,386
ArXiv-S2S	2023	Muennighoff et al. [2023]	Topic	paper titles	93	3,674
MASSIVE(D)	2023	FitzGerald et al. [2023]	Domain	assistant utterances	18	2,974
MASSIVE(I)	2023	FitzGerald et al. [2023]	Intent	assistant utterances	59	2,974
Reddit-S2S	2023	Muennighoff et al. [2023]	Topic	post titles	49	6,000
StackEx-S2S	2023	Zhang et al. [2023b]	Topic	question titles	121	4,156

Table 6: Statistics on the 19 benchmark suite. Criterion is the main clustering criterion family, and k and N give the number of benchmark clusters and documents. CLINC, MTOP and MASSIVE appear twice, labelled by intent and by domain.

2015, MTEB, 2023a, Facebook AI, 2020, Held, 2021, MTEB, 2023b, Snips, 2018, Xu, 2017, Rakib, 2020].

A.2 Published Results by Method and Benchmark

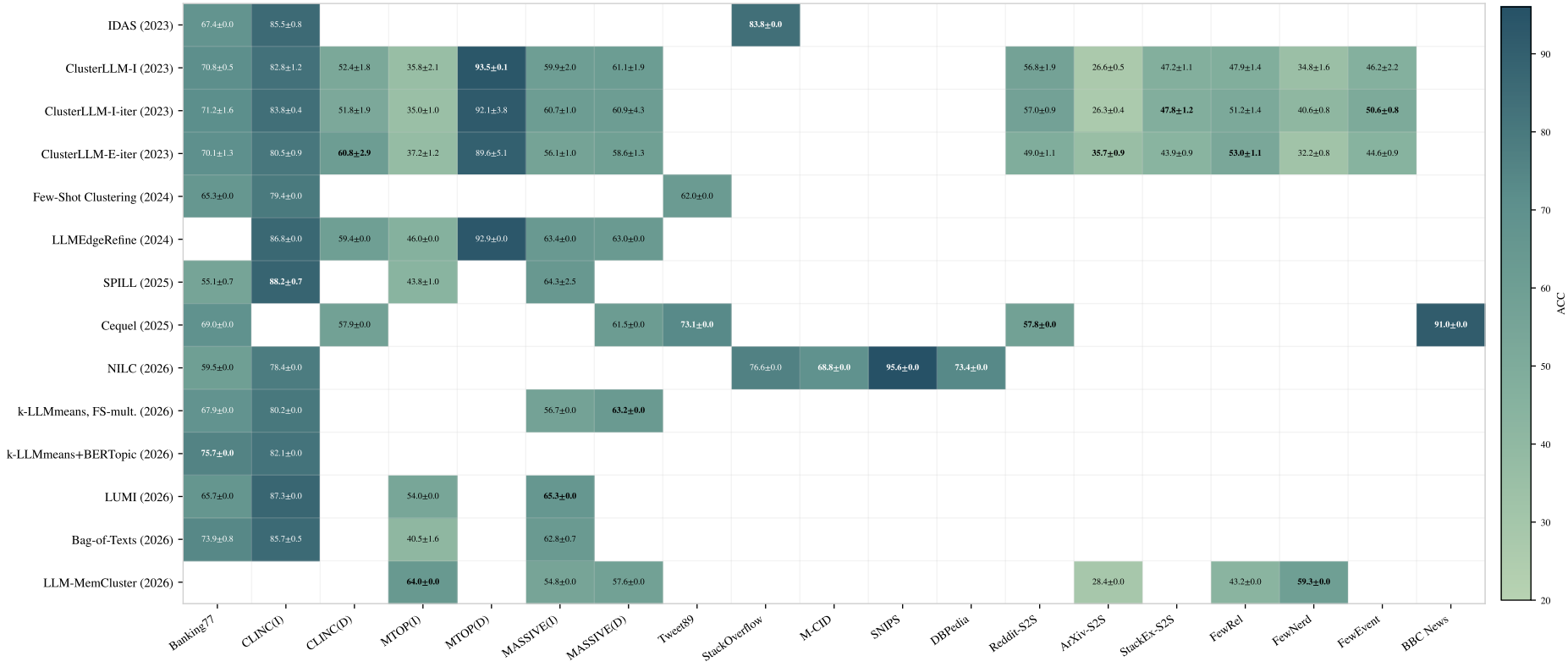


Figure 13: Every published ACC value in the surveyed literature, by method variant (rows) and benchmark (columns). Values are ± the standard deviation reported by the paper, or ±0.0 where none is published.

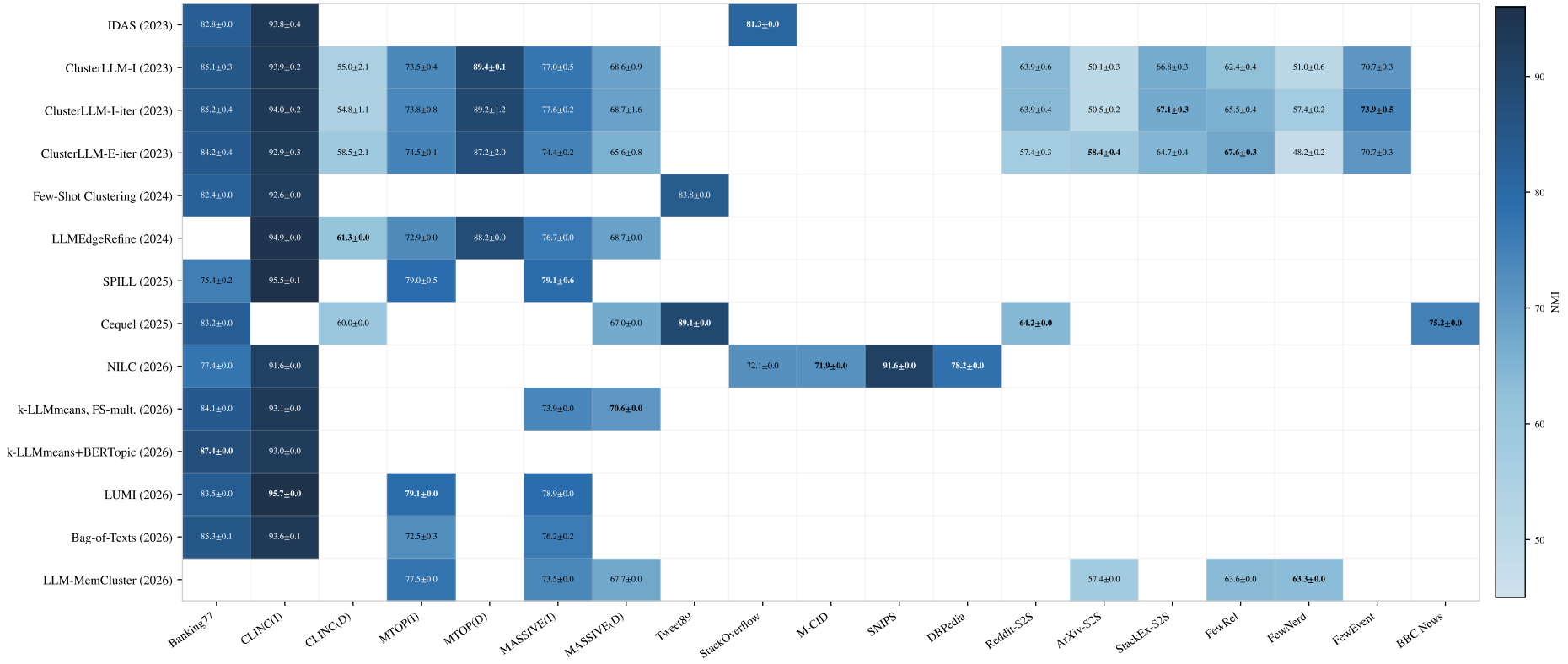


Figure 14: Every published NMI value in the surveyed literature, by method variant (rows) and benchmark (columns). Values are ± the standard deviation reported by the paper, or ±0.0 where none is published.

Appendix B

Experimental Configuration

B.1 Clustering Criteria

Benchmark	Clustering criterion
ArXiv-S2S	"which fine-grained arxiv subject category (e.g. cs.LG, math.AG, hep-th) the paper title belongs to"
BBC News	"which news topic area the article covers"
Banking77	"which banking customer-service intent the query expresses"
CLINC(D)	"which general domain of assistant use the utterance belongs to"
CLINC(I)	"which assistant-command intent the utterance expresses, that is what the user wants done"
DBPedia	"which type of entity the encyclopedia article describes"
FewEvent	"which type of event the trigger word stated at the end of the text ('The event trigger is ...') denotes"
FewNerd	"which entity type the entity named at the end of the text ('The entity type of X') belongs to"
FewRel	"which semantic relation holds between the two entities stated at the end of the text ('The relation between X and Y')"
M-CID	"which COVID-19-related user intent the utterance expresses"
MASSIVE(D)	"which domain of everyday assistant use the utterance belongs to"
MASSIVE(I)	"which assistant-command intent the utterance expresses, that is what the user wants done"
MTOP(D)	"which domain of assistant functionality the utterance belongs to"
MTOP(I)	"which assistant-command intent the utterance expresses, that is what the user wants done"
Reddit-S2S	"which subreddit community the post title was written for"
SNIPS	"which assistant-command intent the utterance expresses, that is what the user wants done"
StackEx-S2S	"which StackExchange community / site the question title was posted to"
StackOverflow	"which programming technology or tool the question is about"
Tweet89	"which specific news story or event the tweet is about"

Table 7: The one-sentence clustering criteria used for each benchmark throughout this thesis.

B.2 Matched Configurations for the State-of-the-Art Comparison

Method	Encoder	LLM	NMI record held on
Cequel	`instructor-large` (Dec 2022)	`gpt-4o-mini` (Jul 2024)	Tweet89, BBC News, Reddit-S2S
ClusterLLM-E-iter	`e5-large`‖ (Dec 2022) → `e5-large-v2` (May 2023)	`gpt-3.5-turbo` (Mar 2023)	ArXiv-S2S, FewRel
ClusterLLM-I	`instructor-large` (Dec 2022)	`gpt-3.5-turbo` (Mar 2023)	MTOP(D)
ClusterLLM-I-iter	`instructor-large` (Dec 2022)	`gpt-3.5-turbo` (Mar 2023)	StackEx-S2S, FewEvent
IDAS	`MTP`† (May 2022) → `paraphrase-mpnet` (May 2021)	`text-davinci-003`‡ (Nov 2022) → `gpt-3.5-turbo` (Mar 2023)	StackOverflow
k-LLMmeans	`text-embedding-3-small` (Jan 2024)	`gpt-4o` (May 2024)	Banking77, MASSIVE(D)
LLMEdgeRefine	`instructor-large` (Dec 2022)	`gpt-3.5-turbo` (Mar 2023)	CLINC(D)
LLM-MemCluster	none¶ → `gemini-embedding-001` (Jul 2025)	`gpt-4.1-mini` (Apr 2025)	FewNerd
LUMI	`instructor-large` (Dec 2022)	`gemma-2-9b-it` (Jun 2024)	CLINC(I), MTOP(I)
NILC	`USNID`† (Apr 2023) → `instructor-large` (Dec 2022)	`gpt-4o-mini` (Jul 2024)	M-CID, SNIPS, DBPedia
SPILL	`gemma-2-9b-it`* (Jun 2024) → `instructor-large` (Dec 2022)	`gemma-2-9b-it` (Jun 2024)	MASSIVE(I)

An arrow (→) marks a component that could not be rerun. The model above the arrow produced the published record and the model below it is the substitute this work used.

‖ `e5-large` was superseded by its authors' `e5-large-v2`.

† an encoder fine-tuned by that paper's authors (`MTP`, Zhang et al., 2022; `USNID`, Zhang et al., 2024b), with no public checkpoint available to rerun.

‡ `text-davinci-003` has been retired from the OpenAI API.

¶ LLM-MemCluster uses no encoder at all, while CLEAR requires one.

* SPILL embeds documents with its own language model's internal states, which is not available as a standalone encoder.

Table 8: For each record-holding method, the encoder and LLM behind its published results, each with its release month, and the benchmarks it holds the published NMI record on. Model references: Su et al., 2023, Wang et al., 2022, Reimers and Gurevych, 2019, OpenAI, 2024a, Lee et al., 2025b, Gemma Team, Google, 2024, Zhang et al., 2022, 2024b, Ouyang et al., 2022, OpenAI, 2023, 2024b, 2025.

B.3 Random Seeds

Random seeds are fixed throughout this work. The candidate test bed of research question 1 uses seeds 0 to 2, the judge replicates use run numbers 1 to 10, the granularity estimators of research question 2 use seeds 0 to 9, and CLEAR uses seeds 0 to 4.

Appendix C

Prompt Templates

C.1 The Codebook Judge

You are designing a codebook for organising a corpus of documents by ⟨*criterion*⟩.

Below are 300 documents sampled from the corpus. Read them all, then write the complete list of categories needed to organise this corpus by ⟨*criterion*⟩.
- Choose the granularity you judge most natural for this corpus and purpose; use however many categories the corpus needs.
- Cover every category you can see evidence for in the sample.
- Do NOT invent categories that are absent, and do not force a round number.
- Definitions must be one line and mutually exclusive where possible.
- Plain text only inside names and definitions: no double quotes, no backslashes.

Respond only with JSON:
{"categories": [{"id": 1, "name": "...", "definition": "..."}, ...]}

Documents:
1. ⟨*document 1*⟩
2. ⟨*document 2*⟩
... ⟨*all sampled documents*⟩

Figure 15: The codebook-discovery prompt. ⟨*criterion*⟩ is the criterion sentence (Table 7) and ⟨*document 1*⟩ etc. are the 300 sampled documents.

You are classifying one document by ⟨*criterion*⟩.

Categories:
1. ⟨*category 1 name*⟩ - ⟨*one-line definition*⟩
2. ⟨*category 2 name*⟩ - ⟨*one-line definition*⟩
... ⟨*all codebook categories*⟩

Which single category does the document belong to? If it truly fits none of them, answer 0.
Respond only with JSON: {"category": <number>}

Document: ⟨*document*⟩

Figure 16: The assignment prompt. ⟨*criterion*⟩ is the criterion sentence, ⟨*category 1 name*⟩ and ⟨*one-line definition*⟩ etc. are the discovery prompt's codebook categories with their definitions, ⟨*document*⟩ is the document being classified and the literal `<number>` is the chosen category id.

C.2 Granularity Estimators

A corpus of ⟨*N*⟩ documents is to be organised into categories by ⟨*criterion*⟩.

Below are ⟨*n*⟩ documents sampled from it. Read them, then estimate how many categories the FULL corpus needs. Choose the granularity you judge most natural.
Respond only with JSON: {"k": <integer>}

Documents:
1. ⟨*document 1*⟩
2. ⟨*document 2*⟩
... ⟨*all sampled documents*⟩

Figure 17: The prompt of the *Direct* granularity method. ⟨*criterion*⟩ is the criterion sentence, ⟨*N*⟩ is the full corpus size, ⟨*n*⟩ is the sample size, 300 throughout, and ⟨*document 1*⟩ etc. are the sampled documents. The literal `<integer>` is the estimate taken as $\hat{k}$.

A corpus of documents is to be organised into categories by ⟨*criterion*⟩. Different granularities of codebook can be valid.

Below are ⟨*n*⟩ documents sampled from the corpus. Propose THREE codebooks as lists of category names (names only): "coarse", "natural", "fine". Cover the whole corpus.

Respond only with JSON:
{"coarse": [...], "natural": [...], "fine": [...]}

Documents:
1. ⟨*document 1*⟩
2. ⟨*document 2*⟩
... ⟨*all sampled documents*⟩

Figure 18: The prompt of the *Probe* granularity method. ⟨*criterion*⟩ is the criterion sentence, ⟨*n*⟩ is the sample size, 300 throughout, and ⟨*document 1*⟩ etc. are the sampled documents. $\hat{k}$ is the size of the returned `natural` list.

Two documents from the same corpus. By ⟨*criterion*⟩, do they belong to the SAME category?
Respond only with JSON: {"same": true} or {"same": false}

Document A: ⟨*document A*⟩

Document B: ⟨*document B*⟩

Figure 19: The same-category vote prompt of the *Pairwise* and *Simpson* methods. ⟨*criterion*⟩ is the criterion sentence and ⟨*document A*⟩ and ⟨*document B*⟩ are the two sampled documents of the pair.

```
A corpus is being organised into categories by ⟨criterion⟩.  Categories found so far:
- ⟨category 1⟩
- ⟨category 2⟩
... ⟨all categories so far, or "(none yet)"⟩

Below are ⟨n⟩ new documents.  List ONLY the genuinely new categories (by ⟨criterion⟩)
needed for documents that fit none of the existing categories.  If none are needed,
respond {"new":  []}.
Respond only with JSON: {"new":  ["...", ...]}

Documents:
1.  ⟨document 1⟩
2.  ⟨document 2⟩
... ⟨all documents in the batch⟩
```

```
These category names were collected while organising one corpus by ⟨criterion⟩.
Merge duplicates and near-duplicates and return the final deduplicated list of
category names.
Respond only with JSON: {"categories":  ["...", ...]}

1.  ⟨category 1⟩
2.  ⟨category 2⟩
... ⟨all collected categories⟩
```

Figure 20: The two prompts of the *Sequential* method. Top: the accumulation prompt where the category list grows with each batch and reads "(none yet)" on the first. Bottom: the deduplication prompt, where $\hat{k}$ is the size of the returned list. ⟨*criterion*⟩ is the criterion sentence, ⟨*n*⟩ is the batch size, 20 throughout, ⟨*category 1*⟩ etc. are the category names collected so far and ⟨*document 1*⟩ etc. are the batch's document texts.

C.3 The CLEAR algortitm

```
For each document below, write a phrase of at most 15 words capturing ONLY
⟨criterion⟩.  Drop everything else - tone, detail, phrasing.  Keep the same order.
Return ONLY a JSON array of ⟨n⟩ strings.

Documents:
1.  ⟨document 1⟩
2.  ⟨document 2⟩
... ⟨all documents in the batch⟩
```

Figure 21: The rewrite prompt. ⟨*criterion*⟩ is the criterion sentence, ⟨*n*⟩ is the batch size, 25 throughout, and ⟨*document 1*⟩ etc. are the batch's document texts.

```
A corpus is organised by ⟨criterion⟩.  One category is:
⟨category name⟩ - ⟨one-line definition⟩
Write ⟨n⟩ short documents that would clearly belong to this category and read like
real corpus entries.
Return ONLY a JSON array of ⟨n⟩ strings.
```

Figure 22: The exemplar prompt. ⟨*criterion*⟩ is the criterion sentence, ⟨*category name*⟩ and ⟨*one-line definition*⟩ are the codebook category's name and definition and ⟨*n*⟩ is the number of example documents requested which is five throughout.

```
Documents grouped by ⟨criterion⟩.  Give ONE short category name (2-6 words) by
⟨criterion⟩.  Name only.
- ⟨central document 1⟩
- ⟨central document 2⟩
... ⟨eight most central documents⟩
```

Figure 23: The cluster naming prompt of the boundary repair stage. ⟨*criterion*⟩ is the criterion sentence and ⟨*central document 1*⟩ etc. are the eight documents closest to the cluster's centroid.

```
Assign each document below to exactly one category from the codebook, organising by
⟨criterion⟩.
CODEBOOK:
1.  ⟨category 1 name⟩
2.  ⟨category 2 name⟩
... ⟨all categories⟩
Use category id 0 if a document fits none of them.  Do not invent categories.
Return ONLY a JSON array with one entry per document, in order:
[{"doc":1,"category":<id>}, ...]
DOCUMENTS (⟨n⟩):
1.  ⟨document 1⟩
2.  ⟨document 2⟩
... ⟨all documents in the batch⟩
```

Figure 24: The batched classification prompt. ⟨*criterion*⟩ is the criterion sentence, ⟨*category 1 name*⟩ etc. are the codebook's category names, ⟨*n*⟩ is the batch size, ⟨*document 1*⟩ etc. are the batch's document texts and `<id>` is the category id the model returns.

Appendix D

Supplementary Results

D.1 Research Question 1

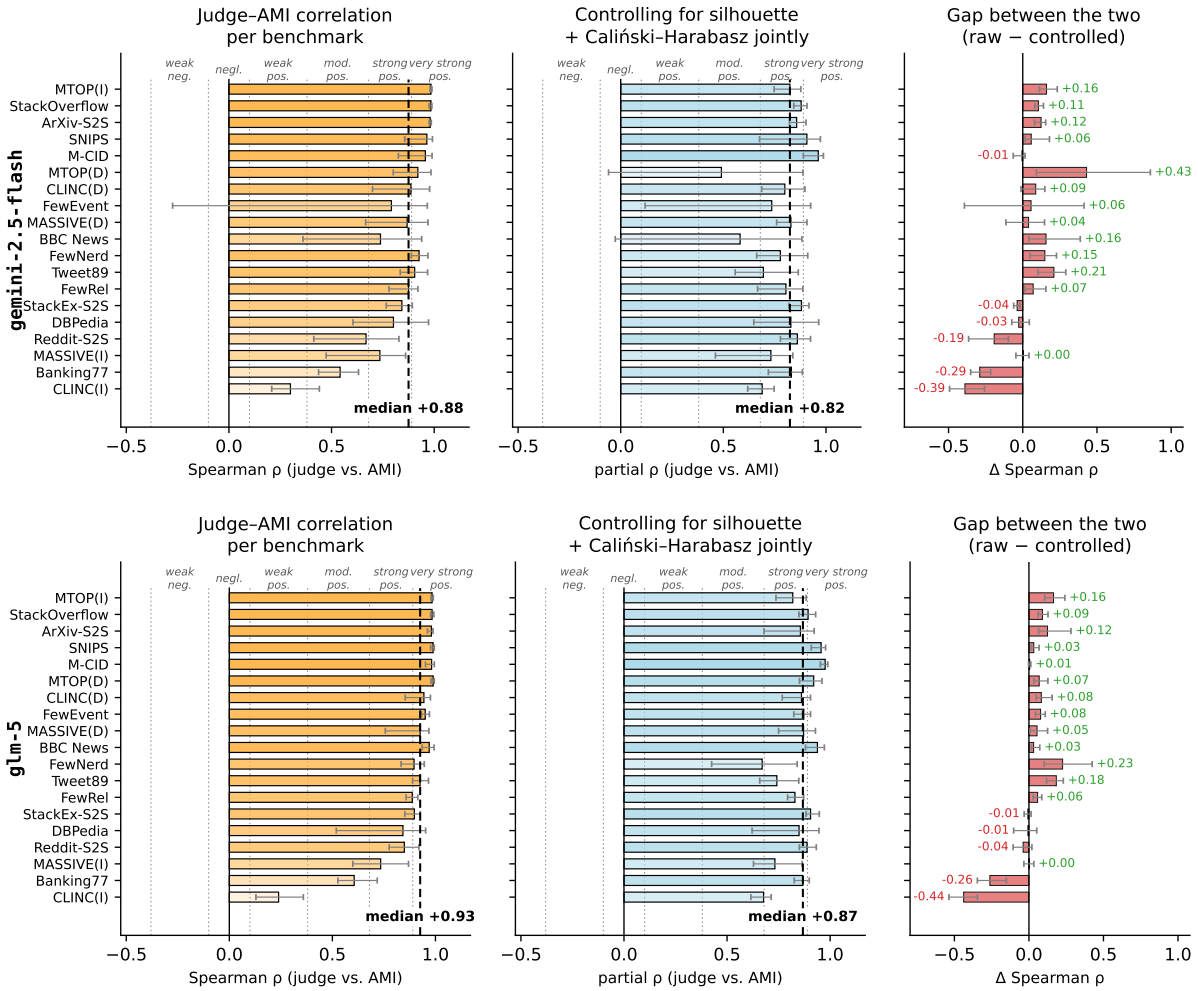


Figure 25: Per-benchmark judge validity on the generic-INSTRUCTOR test bed of clusterings with one row per judge (continued on the next page). Per row: the raw judge–AMI correlation (orange), the partial correlation controlling for the silhouette score and the Caliński–Harabasz index jointly (light blue), and the gap between the two (red). Bars are means over the ten runs, whiskers span the minimum and maximum run, hatched bars are not significant in a majority of runs, and the black dashed line marks the median.

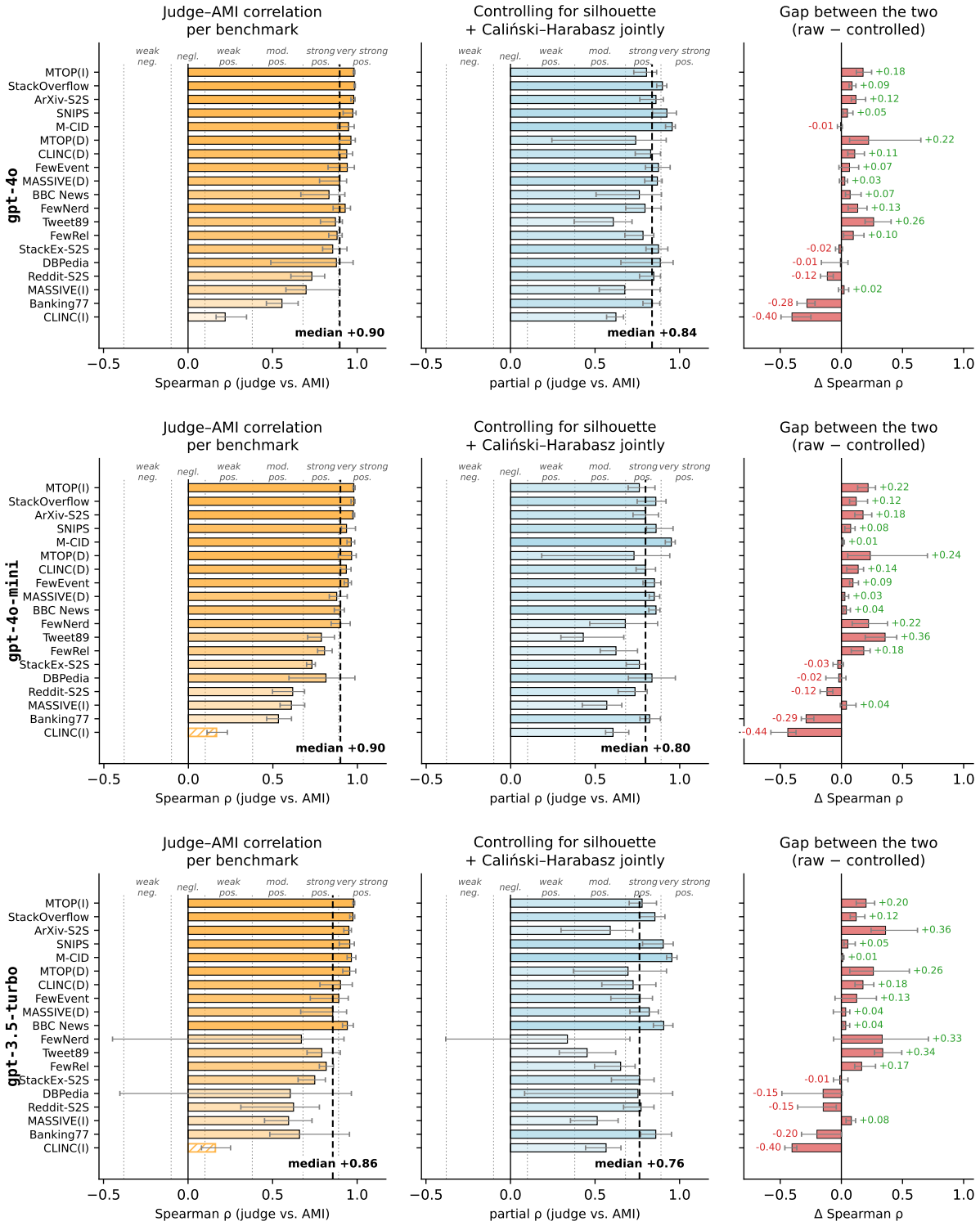


Figure 25: Continued: `gpt-4o`, `gpt-4o-mini` and `gpt-3.5-turbo`.

D.2 Research Question 2

Method	W	p
Codebook	60	0.267
Direct	73	0.395
Probe	94	0.984
Pairwise	75	0.441
Sequential	76	0.465
Simpson	86	0.738
Judge sweep	44	0.124
Merge convergence	72	0.374

Table 9: Two-sided Wilcoxon signed-rank tests of each estimator's 19 per-corpus absolute errors against the silhouette-argmax baseline (§3.2.3), with `gemini-2.5-flash` as the LLM.

Method	Within-1.5× ↑	Within ±10% ↑	Med. $\|\varepsilon\|$ ↓	CCC ↑	Spread ↓
Codebook	6/19	0/19	1.54	0.13	0.47
Direct	6/19	**3/19**	0.90	0.36	0.25
Probe	5/19	1/19	1.54	0.06	2.84
Pairwise	1/19	0/19	1.97	0.11	0.54
Sequential	0/19	0/19	2.55	0.04	0.58
Simpson	**10/19**	**3/19**	**0.58**	**0.66**	1.79
Judge sweep	7/19	2/19	1.40	0.34	1.09
Merge convergence	1/19	0/19	2.02	0.00	0.03
Silhouette-argmax (no LLM)	9/19	4/19	1.15	0.19	1.06

Table 10: Granularity estimation on the 19-benchmark suite with `gpt-4o-mini` as the LLM and generic-instruction `instructor-large` embeddings, with all statistics as defined in the statistical protocol.

Method	Within-1.5× ↑	Within ±10% ↑	Med. $\|\varepsilon\|$ ↓	CCC ↑	Spread ↓
Codebook	6/19	1/19	1.39	0.19	1.54
Direct	**8/19**	**3/19**	**0.83**	**0.51**	0.80
Probe	4/19	2/19	1.13	0.06	4.13
Pairwise	3/19	1/19	2.03	0.04	0.00
Sequential	1/19	0/19	3.01	0.06	0.82
Simpson	4/19	2/19	1.33	0.22	1.31
Judge sweep	7/19	**3/19**	1.13	0.23	1.35
Merge convergence	1/19	0/19	2.00	0.01	0.08
Silhouette-argmax (no LLM)	9/19	4/19	1.15	0.19	1.06

Table 11: Granularity estimation on the 19-benchmark suite with `gpt-3.5-turbo` as the LLM and generic-instruction `instructor-large` embeddings, with all statistics as defined in the statistical protocol.

D.3 Research Question 3

Benchmark	NMI	ACC
SNIPS	96.0 ± 1.3	98.2 ± 0.8
MTOP(D)	94.4 ± 0.7	97.0 ± 0.5
BBC News	92.7 ± 0.4	97.7 ± 0.2
Banking77	92.2 ± 1.4	82.6 ± 6.2
Tweet89	89.1 ± 0.6	73.0 ± 1.4
StackOverflow	87.7 ± 0.7	86.9 ± 2.1
DBPedia	84.8 ± 2.7	77.5 ± 7.9
CLINC(I)	83.8 ± 0.7	56.2 ± 3.1
MTOP(I)	81.5 ± 1.1	55.1 ± 3.8
M-CID	80.2 ± 1.1	73.2 ± 3.2
MASSIVE(I)	79.0 ± 0.8	64.7 ± 1.1
MASSIVE(D)	75.0 ± 0.5	70.6 ± 1.8
StackEx-S2S	73.8 ± 0.8	61.0 ± 1.4
Reddit-S2S	69.4 ± 0.2	66.3 ± 0.4
ArXiv-S2S	67.2 ± 0.3	53.5 ± 1.0
FewEvent	66.0 ± 0.8	47.8 ± 2.2
FewRel	65.1 ± 2.0	44.3 ± 6.1
FewNerd	64.1 ± 0.9	44.9 ± 0.8
CLINC(D)	59.1 ± 2.5	53.1 ± 4.2
Mean	79.0	68.6

Table 12: Per-benchmark clustering quality of CLEAR (`gemini-embedding-001` embeddings, `glm-5` as the LLM), as means and standard deviations over the five runs. These are the exact values for Figure 7.

Declaration of Generative AI Use

This declaration reports all use of generative AI in the production of this thesis, in accordance with UCL's policy on generative AI in assessment. The AI tool used was Anthropic's Claude family of Large Language Models (the Sonnet, Opus and Fable models), which were accessed through the Claude Code assistant between June and September 2026. No other generative AI tools were used.

The research questions, methodologies, the designs of the codebook judge and the CLEAR algorithm, the criterion sentences, the choice of benchmarks and statistical protocols, all experimental decisions and the interpretation of all results are fully of my own and no AI was used to make these decisions. Every AI-assisted output described below was reviewed by me. No research idea or citation from literature in this thesis originates from an AI response.

Software development

AI was used as a coding assistant to help with practical implementation and debugging the following components. In every case I specified the design and intended behaviour, reviewed the code, and verified its outputs. Generated code was reviewed by me and no third-party code was knowingly reproduced without proper licencing.

- The research question 1 pipeline: corpus setup and embedding scripts, candidate partition generation (§3.1.4), geometric metric computation (the label-free metrics of §2.3.1), the judge's codebook and assignment stages (Algorithm 1, Figures 15 and 16), the analysis-table builder, the five judge-replicate scripts (the ten-run protocol of §3.1.4), and the two figure scripts behind the judge figures (Figures 5 and 25).
- The research question 2 estimator implementations (§3.2.1), the silhouette baseline, the replication runs on `gpt-4o-mini` and `gpt-3.5-turbo` (Tables 10 and 11), and the analysis notebook behind the scoreboard and calibration figure (Table 2 and Figure 6).
- The CLEAR pipeline package (candidate pools, judge selection, boundary repair, caching, run orchestration: Figure 4) and its study, baseline and selector-comparison scripts (Figure 12).
- The analysis notebooks and plotting code behind every figure in this thesis.
- The public repository accompanying this thesis: its structure, the ready-to-use command-line tools for both algorithms.

Correctness of this code was established through seed-controlled reruns, cross-checks of computed statistics, and full regeneration of every figure and table from the result files. additionally, every single line of code was checked by me for correctness for things like correct implementation of formulae and the right data.

Data analysis assistance

AI assistance was used to execute and cross-check analyses that I specified, namely consistency audits of every number quoted in this thesis against the underlying run data, the token cost accounting of §5.2, and the Wilcoxon signed-rank tests of the k-means uplift (Figure 8). All these statistics were double checked by me. The choice of tests, the choice of analysis (statistical protocol) and all interpretation are fully of my own. Every single line of code for this data analysis was double checked for correctness by me. Additionally, every input data point into any analysis pipeline was double-checked to ensure the right data was being analysed and reported.

Thesis Writing Support

AI was used in an auditing capacity (review and revision) where it was used to check intermediate thesis drafts for mistakes such as spelling, grammar, inconsistencies in notation, duplicated-content review, correctness of captions, formatting, logical inconsistencies, scientific correctness, incorrect citations, UK English spelling, and general clarity. These types of mistakes in intermediate drafts were flagged by AI, but any rewriting or correcting of those mistakes was fully done by me.

Research support

AI assistance was used to in an assistive capacity to help in the literature review. All literature and claims were double checked by me. No technical content, theoretical claim or literature-review finding in this thesis was sourced from an AI answer instead of the cited literature.

AI Usage Log

The following is a more detailed log of AI usage throughout the project:

- **RQ1 pipeline:** Task: Implementing the protocol scripts of §3.1. Interaction: I described each stage (sample sizes, prompt structure, metrics, the ten-run replicate design) and iterated until the code matched the required protocol. Verification: seed-controlled reruns and re-computations of the reported correlations from the raw data files (and hand verifying the inputs).
- **RQ2 estimators:** Task: Implementing the eight estimators of §3.2.1, the silhouette baseline and both replications. Interaction: I specified each estimator's procedure (grids, sample sizes, vote counts, stopping rules). Verification: per-run estimates checked against the stored run files and the median aggregation recomputed independently.

- **CLEAR algorithm:** Task: Implemented candidate pools, judge selection, boundary repair (Figure 4), caching and orchestration. Interaction: I specified the mechanisms behind the eight candidates, the tie-break order and the repair acceptance rule, and iterated on caching edge cases. Verification: seed-controlled reruns reproduced identical results, and final metrics were recomputed from the saved labellings.

- **Notebooks and figures:** Task: Implemented the analysis notebooks and plotting code. Interaction: I specified each exhibit's content, layout and colour palette and iterated on the rendered output. Verification: Every plotted number checked against the underlying result files, and all notebooks were re-execute from the committed data.

- **Repository and tools:** Task: Structured the public repository and wrapped both algorithms as command-line tools. Interaction: I specified the layout and tool behaviour, built on the existing pipeline functions. Verification: End-to-end smoke runs on the example corpus and full notebook re-execution from the restructured layout.

- **Consistency audits:** Task: Verified every quoted number against the run data. Interaction: I asked for each statistic to be recomputed from the committed files and mismatches reported. Verification: I re-derived any flagged discrepancy myself before correcting the text.

- **Token cost accounting:** Task: Computed the expected costs of §5.2 (Table 4) and the suite totals. Interaction: I specified the what numbers to account for. Verification: I re-derived the cost-table entries myself and confirmed the measured totals against them.